

CS3230

Design and Analysis of Algorithms

Open-Book Examination Notes

Donald Kong
dkkl1137@gmail.com

S2 AY2025/26

Contents

Quick reference	4
Solving an algorithm problem: moves to consider	4
I Foundations	6
1 Mathematical Preliminaries	6
1.1 Notation and basic concepts	6
1.2 Key results	6
1.3 Reference identities	7
1.4 Worked examples	8
2 Asymptotic Analysis	8
2.1 Comparison table	8
2.2 Definitions	9
2.3 Key results	10
2.4 Growth-rate ordering	11
2.5 Worked examples	12
3 Recurrences and Master Theorem	13
3.1 Definitions	13
3.2 Substitution method	13
3.3 Recursion-tree method	14
3.4 Master Theorem	15
4 Correctness	17
4.1 Definitions	17
4.2 Loop invariants	18

4.3	Termination	20
4.4	Induction for recursive algorithms	21
II	Algorithm Design Paradigms	22
5	Divide and Conquer	22
5.1	Definitions	22
5.2	Key results	23
5.3	Worked examples	24
6	Sorting and Lower Bounds	26
6.1	Definitions	26
6.2	Comparison-based lower bound	27
6.3	Quicksort average-case analysis	28
6.4	Linear-time non-comparison sorts	30
6.5	Linear-time selection	31
7	Randomised Algorithms	32
7.1	Definitions	32
7.2	Freivalds' algorithm	33
7.3	Amplification by repetition	35
7.4	Randomised quicksort	36
8	Dynamic Programming	36
8.1	Definitions	37
8.2	Recipe and analysis	38
8.3	Worked examples	40
9	Greedy Algorithms	41
9.1	Definitions	42
9.2	Interval scheduling	43
9.3	Huffman coding	44
9.4	Minimum spanning trees	45
9.5	Approximation algorithms by greedy	47
10	Amortised Analysis	48
10.1	Definitions	49
10.2	The three methods	50
10.3	Binary counter	51
10.4	Dynamic array	52
III	Limits of Efficient Computation	54

11 Reductions and Intractability	54
11.1 Decision problems and complexity classes	54
11.2 Reductions	55
11.3 Properties of reductions	56
11.4 Worked example	58
12 NP-Completeness	58
12.1 Definitions	59
12.2 Cook–Levin theorem	59
12.3 Canonical reductions	60
12.4 Worked example	63
A Selected Problems	64
A.1 $\log f \in o(\log g)$ does not follow from $f \in o(g)$	64
A.2 Recurrence with two unequal recursive calls	64
A.3 H-index by unbounded (galloping) binary search	65
A.4 Two-scanner search with a moving target	65
A.5 Sorting pairs (x, y) by $x + n^y$ in $O(n)$	65
A.6 Existence proofs via the probabilistic method	65
A.7 Minimise $ S_X - 2S_Y $ via subset-sum DP	66
A.8 Interval cover: pick the latest finish among feasible starts	66
A.9 Repeated set unions cost $O(n \log n)$	66
A.10 Decision-to-search via self-reduction (Hamiltonian cycle, 3-SAT)	66
A.11 Reducing CLIQUE to Half Clique by padding	67
Index of Definitions	68

Quick reference

Logs and exponentials.

- $a^{\log_b c} = c^{\log_b a}$
- $\log_b(xy) = \log_b x + \log_b y$, $\log_b(x^k) = k \log_b x$
- $e^x \geq 1 + x$ for all x ; $\ln(1 + x) \leq x$ for $x > -1$

Series.

- Arithmetic: $\sum_{i=1}^n i = n(n+1)/2$
- Geometric: $\sum_{i=0}^{n-1} r^i = (r^n - 1)/(r - 1)$ for $r \neq 1$; $\sum_{i=0}^{\infty} r^i = 1/(1 - r)$ for $|r| < 1$
- Harmonic: $H_n = \sum_{i=1}^n 1/i = \ln n + \Theta(1)$

Stirling's approximation. $n! = \sqrt{2\pi n} (n/e)^n (1 + \Theta(1/n))$; in particular $\log_2(n!) = n \log_2 n - n \log_2 e + \Theta(\log n)$.

Probability.

- Linearity: $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ (always)
- Markov: $\mathbb{P}[X \geq a] \leq \mathbb{E}[X]/a$ for $X \geq 0$, $a > 0$
- Chebyshev: $\mathbb{P}[|X - \mathbb{E}[X]| \geq a] \leq \text{Var}(X)/a^2$
- Union bound: $\mathbb{P}[\bigcup_i A_i] \leq \sum_i \mathbb{P}[A_i]$

Master Theorem. For $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$, let $c^* = \log_b a$:

- Case 1: $f(n) = O(n^{c^* - \varepsilon})$ for some $\varepsilon > 0 \Rightarrow T(n) = \Theta(n^{c^*})$
- Case 2: $f(n) = \Theta(n^{c^*}) \Rightarrow T(n) = \Theta(n^{c^*} \log n)$
- Case 3: $f(n) = \Omega(n^{c^* + \varepsilon})$ and $af(n/b) \leq kf(n)$ for some $k < 1$ and large $n \Rightarrow T(n) = \Theta(f(n))$

Solving an algorithm problem: moves to consider

For an algorithm-design problem, try in order:

1. **Greedy** (section 9). Identify the locally optimal choice; attempt an exchange argument. Often the intended solution; cheap to try first.
2. **Divide and conquer** (section 5). Look for a way to split the input into independent subproblems of similar shape; analyse via Master Theorem (section 3).
3. **Dynamic programming** (section 8). Identify subproblems and a recurrence; check overlap; tabulate. Time = (state space) $\times$ (per-transition cost).

4. **Reduction** (section 11). Reduce to a known polynomial-time problem (e.g., max-flow, shortest paths) – or, if intractability is suspected, reduce a known NP-hard problem to it (section 12).
5. **Randomisation** (section 7). When deterministic structure is hard to exploit, try a Las Vegas or Monte Carlo strategy; analyse via linearity of expectation.

For a correctness proof: state a loop invariant or inductive hypothesis explicitly; show init / maintenance / termination (section 4).

For a complexity analysis: write down the recurrence; identify a , b , $f(n)$; apply the Master Theorem case-decision tree (fig. 2).

For an NP-hardness proof: pick a known NP-complete problem (section 12) and construct a polynomial-time mapping reduction to the target. State the mapping and prove correctness in both directions.

For a data-structure analysis: if costs vary across operations but the total over a sequence is bounded, this is amortised analysis – pick aggregate / accounting / potential and apply (section 10).

Part I

Foundations

1 Mathematical Preliminaries

Algorithm analysis throughout these notes draws on three small toolkits: discrete probability, elementary asymptotic identities, and a handful of closed-form sums. This chapter fixes notation and collects the working set used throughout the remainder of the notes.

1.1 Notation and basic concepts

A *probability space* consists of a finite sample space S and a probability measure $\mathbb{P}$ assigning to each event $A \subseteq S$ a value $\mathbb{P}(A) \in [0, 1]$, with $\mathbb{P}(S) = 1$ and $\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B)$ for disjoint A, B . Two events are *independent* if $\mathbb{P}(A \cap B) = \mathbb{P}(A) \mathbb{P}(B)$. A *random variable* X is a function $X : S \rightarrow \mathbb{R}$; its *expectation* is $\mathbb{E}[X] = \sum_x x \cdot \mathbb{P}(X = x)$. The *indicator* $\mathbb{1}_A$ of an event A is the random variable taking value 1 when A occurs and 0 otherwise, so that $\mathbb{E}[\mathbb{1}_A] = \mathbb{P}(A)$. The *variance* of X with mean μ is $\text{Var}[X] = \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - \mu^2$, and the *standard deviation* is $\sigma_X = \sqrt{\text{Var}[X]}$. Variance scales as $\text{Var}[aX] = a^2 \text{Var}[X]$ and is additive over independent summands: $\text{Var}[\sum_i X_i] = \sum_i \text{Var}[X_i]$ if the X_i are pairwise independent.

1.2 Key results

Theorem 1.1 (Linearity of expectation). *For any random variables $X_1, \dots, X_n$ on a common probability space and any constants $a_1, \dots, a_n \in \mathbb{R}$,*

$$\mathbb{E}\left[\sum_{i=1}^n a_i X_i\right] = \sum_{i=1}^n a_i \mathbb{E}[X_i].$$

The X_i need not be independent.

Theorem 1.2 (Markov's inequality). *Let X be a non-negative random variable. Then for every $t > 0$,*

$$\mathbb{P}(X \geq t) \leq \frac{\mathbb{E}[X]}{t}.$$

Remark 1.3. Markov requires $X \geq 0$; applied to a possibly-negative variable, the bound fails. It bounds only the upper tail and is informative only when $t > \mathbb{E}[X]$ (otherwise $\mathbb{E}[X]/t \geq 1$). For variables that may be negative, apply Markov to $|X|$, X^2 , or $X - \min$.

Theorem 1.4 (Chebyshev's inequality). *Let X be a random variable with finite mean $\mu = \mathbb{E}[X]$ and finite variance $\sigma^2 = \text{Var}[X]$. Then for every $t > 0$,*

$$\mathbb{P}(|X - \mu| \geq t) \leq \frac{\sigma^2}{t^2}.$$

Remark 1.5. The denominator is t^2 , not t : a frequent slip is to write $\text{Var}[X]/t$. Equivalently, $\mathbb{P}(|X - \mu| \geq k\sigma) \leq 1/k^2$.

Theorem 1.6 (Union bound). *For any events $A_1, A_2, \dots, A_n$ on a common probability space,*

$$\mathbb{P}\left(\bigcup_{i=1}^n A_i\right) \leq \sum_{i=1}^n \mathbb{P}(A_i).$$

Remark 1.7. The union bound is loose but free, and is often tight enough. For highly correlated or numerous events, an inclusion–exclusion or second-moment argument may be sharper.

Remark 1.8 (Why these four are the workhorses). Linearity (which does not require independence) converts expected-count problems into sums of indicator probabilities and underpins average-case running-time analyses. Markov turns a single moment bound into a tail bound; Chebyshev sharpens this when the second moment is available; the union bound concludes “with high probability nothing bad happens” from per-event failure probabilities.

1.3 Reference identities

The following identities are used without proof throughout the notes; they are collected here for reference.

Exponentials and logarithms. For all $a, b, c > 0$ and real m, n, x :

$$\begin{aligned} a^m a^n &= a^{m+n}, & (a^m)^n &= a^{mn}, & a^{-1} &= 1/a, \\ e^x &\geq 1 + x, & \ln x &\leq x - 1 \text{ for } x > 0, & a &= b^{\log_b a}, \\ \log_c(ab) &= \log_c a + \log_c b, & \log_b(a^c) &= c \log_b a, & \log_b a &= \frac{\log_c a}{\log_c b} = \frac{1}{\log_a b}, \\ a^{\log_b c} &= c^{\log_b a}. \end{aligned}$$

The base of a logarithm is irrelevant inside $\Theta(\cdot)$ since $\log_a n = \Theta(\log_b n)$ for any constant bases $a, b > 1$. Bases of *exponentials* are not asymptotically interchangeable: $4^n \notin \Theta(2^n)$.

Arithmetic and geometric series.

$$\begin{aligned} \sum_{i=1}^n i &= \frac{n(n+1)}{2}, & \sum_{i=1}^n i^2 &= \frac{n(n+1)(2n+1)}{6}, & \sum_{i=1}^n i^k &= \frac{n^{k+1}}{k+1} + O(n^k), \\ \sum_{i=0}^n r^i &= \frac{1-r^{n+1}}{1-r} \quad (r \neq 1), & \sum_{i=0}^{\infty} r^i &= \frac{1}{1-r} \quad (|r| < 1), & \sum_{i=0}^{\infty} i r^i &= \frac{r}{(1-r)^2} \quad (|r| < 1). \end{aligned}$$

Harmonic numbers. The n -th harmonic number is $H_n = \sum_{i=1}^n 1/i$. Comparing with $\int_1^n \frac{dx}{x}$ gives the tight envelope

$$\ln n \leq H_n \leq 1 + \ln n,$$

hence $H_n = \ln n + \Theta(1)$. (More precisely, $H_n = \ln n + \gamma + O(1/n)$, where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant.)

Stirling’s approximation.

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + O\left(\frac{1}{n}\right)\right), \quad \log(n!) \in \Theta(n \log n).$$

The second form is the version invoked in the $\Omega(n \log n)$ comparison-sorting lower bound.

1.4 Worked examples

Example 1.9 (Expected number of fixed points of a random permutation). Let π be a uniformly random permutation of $\{1, 2, \dots, n\}$, and let F be the number of *fixed points* of π , i.e. $F = |\{i : \pi(i) = i\}|$. We compute $\mathbb{E}[F]$.

Solution. Define indicator variables $X_i = \mathbb{1}_{\{\pi(i)=i\}}$ for $i = 1, \dots, n$, so that $F = \sum_{i=1}^n X_i$. By the indicator-expectation identity,

$$\mathbb{E}[X_i] = \mathbb{P}(\pi(i) = i).$$

Among the $n!$ permutations of $\{1, \dots, n\}$, exactly $(n-1)!$ fix position i (the remaining $n-1$ positions can be permuted freely), so $\mathbb{P}(\pi(i) = i) = (n-1)!/n! = 1/n$. By linearity of expectation (Theorem 1.1), which crucially does *not* require the X_i to be independent,

$$\mathbb{E}[F] = \sum_{i=1}^n \mathbb{E}[X_i] = \sum_{i=1}^n \frac{1}{n} = 1.$$

The expected number of fixed points is exactly 1, independent of n .

Remarks. The X_i are *not* independent: if $\pi(1) = 1$ then the conditional probability that $\pi(2) = 2$ is $1/(n-1) \neq 1/n$. Independence was never required; only linearity of expectation. This pattern – decompose a count into indicators, compute each marginal probability, sum – recurs throughout average-case running-time analyses.

2 Asymptotic Analysis

Asymptotic analysis treats the running time of an algorithm as a function $T(n)$ of the input size n , ignores constant factors and lower-order terms, and asks how $T(n)$ behaves as $n \rightarrow \infty$. The cost model is the RAM machine of section 1: each arithmetic, logical, comparison, branch, or memory-access instruction is charged $\Theta(1)$ time (exponentiation excepted), and $T(n)$ is the worst-case or average-case instruction count on inputs of size n . The Bachmann–Landau symbols O , Ω , Θ , o , ω are the standard vocabulary for stating such bounds.

2.1 Comparison table

The five asymptotic notations are conveniently organised by reading two parallel columns: the constants-form definition (use this when you need a proof from first principles) and the limit-form characterisation (use this when one of L'Hôpital's rule, Stirling, or known growth-rate facts evaluates the limit cleanly). The two columns agree whenever the limit exists; the constants-form is strictly more general.

Coarse bounds: O , Ω , Θ .

We say	if $\exists c, c_1, c_2, n_0 > 0$ s.t. $\forall n \geq n_0$,	if exists, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$
$f(n) \in O(g(n))$	$0 \leq f(n) \leq c \cdot g(n)$	$< \infty$
$f(n) \in \Omega(g(n))$	$0 \leq c \cdot g(n) \leq f(n)$	> 0
$f(n) \in \Theta(g(n))$	$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$	$\in (0, \infty)$

Strict bounds: o , ω . Note the universal quantifier on c and the strict inequality.

We say	if $\forall c > 0, \exists n_0 > 0$ s.t. $\forall n \geq n_0,$	if exists, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$
$f(n) \in o(g(n))$	$0 \leq f(n) < c \cdot g(n)$	$= 0$
$f(n) \in \omega(g(n))$	$0 \leq c \cdot g(n) < f(n)$	$= \infty$

2.2 Definitions

In every definition below, f and g are functions $\mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, assumed eventually non-negative. The clause $0 \leq \dots$ in each definition enforces this and is part of the definition.

Definition 2.1 (Big-O). $f \in O(g)$ if there exist constants $c > 0$ and $n_0 > 0$ such that

$$\forall n \geq n_0, \quad 0 \leq f(n) \leq c \cdot g(n).$$

Read: “ f grows at most as fast as g .” Equivalently, $O(g)$ is the set of functions eventually dominated by some constant multiple of g .

Remark 2.2 (Limit form of O). If $\lim_{n \rightarrow \infty} f(n)/g(n) < \infty$ (in particular, if the limit equals 0 or any finite positive number), then $f \in O(g)$. The converse fails when the limit does not exist: e.g., $f(n) = (2 + \sin n) \cdot n$ and $g(n) = n$ satisfy $f \in O(g)$ even though f/g oscillates and has no limit.

Remark 2.3 (Notation: $\in$ versus $=$). The customary notation $f = O(g)$ is a one-way membership, not an equality: it abbreviates $f \in O(g)$. Reading it as set inclusion makes statements like $O(n) = O(n^2)$ meaningful while the reversed $O(n^2) = O(n)$ is false. Writing $\in$ throughout avoids the asymmetry trap.

Definition 2.4 (Big-Omega). $f \in \Omega(g)$ if there exist constants $c > 0$ and $n_0 > 0$ such that

$$\forall n \geq n_0, \quad 0 \leq c \cdot g(n) \leq f(n).$$

Read: “ f grows at least as fast as g .”

Remark 2.5 (Limit form of Ω). If $\lim_{n \rightarrow \infty} f(n)/g(n) > 0$ (allowing ∞), then $f \in \Omega(g)$.

Remark 2.6 (Eventual non-negativity is part of the definition). The clause $0 \leq \dots$ in the constants-form definitions of O and Ω is silently dropped in many textbooks but must be checked when f or g can be negative or zero. In algorithm analysis the functions of interest are running times, which are non-negative, so this clause is automatic.

Definition 2.7 (Big-Theta). $f \in \Theta(g)$ if there exist constants $c_1, c_2 > 0$ and $n_0 > 0$ such that

$$\forall n \geq n_0, \quad 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n).$$

Equivalently, $f \in \Theta(g) \iff f \in O(g) \wedge f \in \Omega(g)$, i.e., $\Theta(g) = O(g) \cap \Omega(g)$. Read: “ f and g grow at the same rate.”

Remark 2.8 (Limit form of Θ). If $\lim_{n \rightarrow \infty} f(n)/g(n) = c$ for some $c \in (0, \infty)$, then $f \in \Theta(g)$.

Remark 2.9 (Both directions are required). Membership in $\Theta(g)$ requires both $f \in O(g)$ and $f \in \Omega(g)$; an upper bound alone does not suffice. For instance, $n \in O(n^2)$ but $n \notin \Theta(n^2)$.

Definition 2.10 (little-o). $f \in o(g)$ if for every constant $c > 0$ there exists $n_0 > 0$ such that

$$\forall n \geq n_0, \quad 0 \leq f(n) < c \cdot g(n).$$

Note the universal quantifier on c and the strict inequality. Read: “ f grows strictly slower than g .” We often write $f \ll g$ as shorthand for $f \in o(g)$.

Remark 2.11 (Limit form of o). $f \in o(g) \iff \lim_{n \rightarrow \infty} f(n)/g(n) = 0$ (assuming the ratio is eventually defined). Some textbooks use $\leq$ instead of $<$ in the constants-form; this changes only the choice of n_0 and yields the same set of functions.

Remark 2.12 (O versus o). The two definitions differ in two places: Big- O uses an existential constant and $\leq$, whereas little- o uses a universal constant and $<$. The distinction settles questions like “ $n^2 \in o(n^2)$?” (no: no $c < 1$ works) versus “ $n^2 \in O(n^2)$?” (yes, with $c = 1$).

Definition 2.13 (little- ω). $f \in \omega(g)$ if for every constant $c > 0$ there exists $n_0 > 0$ such that

$$\forall n \geq n_0, \quad 0 \leq c \cdot g(n) < f(n).$$

Read: “ f grows strictly faster than g .”

Remark 2.14 (Limit form of ω). $f \in \omega(g) \iff \lim_{n \rightarrow \infty} f(n)/g(n) = \infty$.

2.3 Key results

Theorem 2.15 (Equivalence of definitional and limit forms). Suppose f, g are eventually positive and $\lim_{n \rightarrow \infty} f(n)/g(n) = L$ exists in $[0, \infty]$. Then

- $L = 0 \iff f \in o(g)$;
- $L < \infty \implies f \in O(g)$;
- $L \in (0, \infty) \implies f \in \Theta(g)$;
- $L > 0 \implies f \in \Omega(g)$ (with $L = \infty$ giving $f \in \omega(g)$);
- $L = \infty \iff f \in \omega(g)$.

Proof. (sketch.) All five clauses are direct unwindings of the ε - N definition of a limit. We illustrate the first. Suppose $L = 0$. Fix any $c > 0$; by definition of limit there exists n_0 such that for all $n \geq n_0$, $|f(n)/g(n) - 0| < c$, i.e. $f(n) < c \cdot g(n)$, and $f(n) \geq 0$ by assumption. This is precisely the definition of $f \in o(g)$ (Definition 2.10). Conversely, if $f \in o(g)$, then for every $\varepsilon > 0$ there is n_0 with $f(n)/g(n) < \varepsilon$ for $n \geq n_0$, so the limit exists and equals 0. The other clauses are analogous: take $\varepsilon = L/2$ for the lower bound and $\varepsilon = 1$ together with $c = L + 1$ for the upper bound. The unidirectional clauses (involving only $\implies$) cannot be reversed because Big- O and Ω are insensitive to oscillation: a bounded oscillating ratio gives $f \in \Theta(g)$ without the limit existing.

Remark 2.16 (When the limit does not exist). The limit-form shortcuts only apply when $\lim_{n \rightarrow \infty} f(n)/g(n)$ exists. For oscillating ratios such as $f(n) = (2 + \sin n)n$ against $g(n) = n$, no limit exists and one must fall back to the constants-form definition.

Theorem 2.17 (O – Ω duality). For any eventually non-negative f and g ,

$$f \in O(g) \iff g \in \Omega(f).$$

The same swap converts o and ω : $f \in o(g) \iff g \in \omega(f)$.

Proof. Suppose $f \in O(g)$, witnessed by constants $c > 0$ and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for $n \geq n_0$. Dividing by $c > 0$ gives $0 \leq \frac{1}{c}f(n) \leq g(n)$ for $n \geq n_0$, which is exactly the definition of $g \in \Omega(f)$ (Definition 2.4) with constant $c' = 1/c$ and the same n_0 .

Conversely, if $g \in \Omega(f)$ with constants c' and n'_0 giving $c' \cdot f(n) \leq g(n)$, then $f(n) \leq \frac{1}{c'}g(n)$ for $n \geq n'_0$, so $f \in O(g)$ with $c = 1/c'$.

The little- o/ω case is identical, with the universal quantifier on the constant c being preserved by the same reciprocation: $\forall c > 0, f(n) < c \cdot g(n)$ iff $\forall c' > 0, \frac{1}{c'} f(n) < g(n)$ (relabel $c = 1/c'$).

Theorem 2.18 (Transitivity). *Let $A \in \{O, \Omega, \Theta, o, \omega\}$. Then*

$$f \in A(g) \text{ and } g \in A(h) \implies f \in A(h).$$

Proof. We show the case $A = O$; the others follow either by the same scaling argument or by Theorem 2.17. Suppose $f \in O(g)$, witnessed by c_1, n_1 with $f(n) \leq c_1 g(n)$ for $n \geq n_1$, and $g \in O(h)$, witnessed by c_2, n_2 with $g(n) \leq c_2 h(n)$ for $n \geq n_2$. Set $n_0 = \max(n_1, n_2)$ and $c = c_1 c_2$. For all $n \geq n_0$,

$$0 \leq f(n) \leq c_1 g(n) \leq c_1 c_2 h(n) = c \cdot h(n),$$

so $f \in O(h)$.

For $A = \Omega$, transit through Theorem 2.17: $f \in \Omega(g) \iff g \in O(f)$, etc., and apply the O case in reverse. For $A = \Theta$, combine the O and Ω cases. For $A = o$, the same product-of-constants argument works, but the universal quantifier on c is handled as follows: given any $c > 0$, choose $c_1 = c/c_2$ in the $f \in o(g)$ definition where c_2 is any fixed constant making $g(n) \leq c_2 h(n)$ eventually (such c_2 exists because $g \in o(h) \subseteq O(h)$). The argument for ω is dual.

Theorem 2.19 (Sum and max rules). *If $f \in O(g)$ and $f' \in O(g')$, then*

$$f + f' \in O(\max(g, g')) = O(g + g').$$

In particular, $O(g) + O(g') = O(\max(g, g'))$. The same identity holds with O replaced by Ω or Θ , with $\max$ replaced by $\min$ in the case of Ω if one wants the analogous lower-bound version.

Proof. Let c, n_1 witness $f \leq c \cdot g$ for $n \geq n_1$, and let c', n_2 witness $f' \leq c' \cdot g'$ for $n \geq n_2$. For $n \geq n_0 := \max(n_1, n_2)$,

$$0 \leq f(n) + f'(n) \leq c \cdot g(n) + c' \cdot g'(n) \leq (c + c') \cdot \max(g(n), g'(n)),$$

so $f + f' \in O(\max(g, g'))$ with constant $c + c'$. The equality $O(\max(g, g')) = O(g + g')$ is then immediate from the two-sided bound $\max(g, g') \leq g + g' \leq 2 \max(g, g')$ (assuming $g, g' \geq 0$), which keeps the same set of functions up to a constant factor.

The corollary used most often is: *a sum of finitely many terms is asymptotically the largest term.* So $3n^2 + 5n + 7 \log n \in \Theta(n^2)$ because $5n, 7 \log n \in O(n^2)$ and the leading $3n^2$ term is in $\Omega(n^2)$.

Corollary 2.20 (Reflexivity, symmetry of Θ). *For every f , $f \in O(f)$, $f \in \Omega(f)$, and $f \in \Theta(f)$. Furthermore $f \in \Theta(g) \iff g \in \Theta(f)$.*

Proof. Take $c = 1, n_0 = 1$ in each constants-form definition for reflexivity. Symmetry of Θ follows from $\Theta(g) = O(g) \cap \Omega(g)$ together with Theorem 2.17: $f \in \Theta(g) \iff f \in O(g) \wedge f \in \Omega(g) \iff g \in \Omega(f) \wedge g \in O(f) \iff g \in \Theta(f)$.

2.4 Growth-rate ordering

The functions one meets in algorithm analysis fall into a small number of asymptotic equivalence classes that are totally ordered by $\ll$ (where $f \ll g$ abbreviates $f \in o(g)$). The ladder below subsumes most concrete growth-rate comparisons.

$$1 \ll \log \log n \ll \log n \ll \log^c n \ll n^{1/k} \ll n \ll n \log n \ll n^c \ll c^n \ll n! \ll n^n$$

where $c > 1$ is any fixed constant and $k \geq 1$ is any fixed integer. In words, going left to right: constants, double-log, single-log, any constant power of log, any constant fractional power of n , linear, $n \log n$, any constant power of n (polynomial), any constant base raised to n (exponential), the factorial, and finally n^n .

Useful identities.

- Inside $\Theta(\cdot)$ the base of a logarithm is irrelevant: $\log_a n = \Theta(\log_b n)$ for any constant bases $a, b > 1$. Bases of *exponentials* are not interchangeable: $4^n \notin \Theta(2^n)$, since $4^n/2^n = 2^n \rightarrow \infty$.
- L'Hôpital's rule: if $f, g \rightarrow \infty$ then $\lim f/g = \lim f'/g'$. This is the standard tool for proving $f \in o(g)$ between, e.g., $n \log n$ and n^2 .
- Stirling: $n! = \sqrt{2\pi n}(n/e)^n(1 + O(1/n))$, so $\log(n!) \in \Theta(n \log n)$ and $n! \in \omega(c^n)$ for every constant $c > 1$.
- Logarithm-in-the-exponent identity: $a^{\log_b c} = c^{\log_b a}$. Useful for normalising expressions like $n^{\log_2 3}$ that arise from the master theorem.

2.5 Worked examples

Example 2.21 ($3n^2 + 5n \in \Theta(n^2)$ from first principles). We show both $3n^2 + 5n \in O(n^2)$ and $3n^2 + 5n \in \Omega(n^2)$.

Upper bound. For all $n \geq 1$, $5n \leq 5n^2$, so $3n^2 + 5n \leq 3n^2 + 5n^2 = 8n^2$. Take $c_2 = 8$, $n_0 = 1$.

Lower bound. For all $n \geq 1$, $3n^2 + 5n \geq 3n^2$. Take $c_1 = 3$, $n_0 = 1$.

Conclusion. $0 \leq 3n^2 \leq 3n^2 + 5n \leq 8n^2$ for all $n \geq 1$, so $3n^2 + 5n \in \Theta(n^2)$ by Definition 2.7. The same scheme handles any polynomial: for $p(n) = a_d n^d + \dots + a_0$ with $a_d > 0$, $p \in \Theta(n^d)$.

Example 2.22 ($\log^{100} n \in o(n^{0.1})$). **True.** Apply L'Hôpital's rule 100 times, or argue via the substitution $u = \log n$ (so $n = e^u$):

$$\frac{\log^{100} n}{n^{0.1}} = \frac{u^{100}}{e^{0.1u}} \xrightarrow{u \rightarrow \infty} 0,$$

since exponentials beat polynomials in u . Hence the limit-form characterisation gives $\log^{100} n \in o(n^{0.1})$. The takeaway: every constant power of $\log n$ is dominated by every constant fractional power of n , no matter how the constants compare.

Example 2.23 ($\log \log n \in \omega(n^{100})$). **False.** The relation goes the other way: $\log \log n \in o(n^{100})$, indeed $\log \log n \in o(n^\varepsilon)$ for every $\varepsilon > 0$. To see this, $\log \log n < \log n \in o(n^\varepsilon)$ by the previous example, so $\log \log n \in o(n^{100})$ by transitivity (Theorem 2.18). Equivalently, $n^{100} \in \omega(\log \log n)$ by duality (Theorem 2.17). The given statement asserts the wrong inequality.

Example 2.24 ($f \in O(g) \implies g \in \Omega(f)$). **True.** This is exactly the O – Ω duality theorem (Theorem 2.17). The proof is one line: if $f \leq c \cdot g$ eventually, then $\frac{1}{c}f \leq g$ eventually, witnessing $g \in \Omega(f)$. Note carefully that the converse statement “ $g \in \Omega(f) \implies f \in O(g)$ ” is also true (it is the same equivalence read in the other direction), but the analogous swap with Θ in place of O/Ω would only give $\Theta(f) \subseteq \Theta(g) \iff \Theta(g) \subseteq \Theta(f)$, the symmetric case.

3 Recurrences and Master Theorem

The running time of a recursive algorithm is naturally expressed by a *recurrence*: the cost on an input of size n in terms of the cost on the smaller inputs produced by its recursive calls, plus the local work outside those calls. This chapter collects four standard solving techniques – telescoping, substitution, the recursion tree, and the master theorem – and gives the master theorem the full treatment, since it dispatches most divide-and-conquer recurrences in a single line. The asymptotic vocabulary of section 2 is presupposed: solutions are stated up to $\Theta(\cdot)$, and the driving function $f(n)$ is described by a Θ , O , or Ω class.

3.1 Definitions

Definition 3.1 (Recurrence relation). A *recurrence relation* for a function $T : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ specifies $T(n)$ for small n (the *base case*) and, for larger n , expresses $T(n)$ as a function of T evaluated at smaller arguments. The *generic divide-and-conquer form* adopted in this chapter is

$$T(n) = aT(n/b) + f(n), \quad a \geq 1, b > 1, f(n) \in \Omega(1),$$

with $T(n) = \Theta(1)$ on a finite set of base cases. Here a is the number of recursive subproblems, n/b is the size of each subproblem (floors and ceilings are dropped without affecting the asymptotic answer), and $f(n)$ is the *driving function*, accounting for all work outside the recursive calls (typically the cost of splitting the input and combining the subsolutions). Linear recurrences such as $T(n) = T(n-1) + cn$ and recurrences with mixed splits such as $T(n) = T(n/3) + T(2n/3) + n$ also fall under the umbrella of Definition 3.1 but lie outside the master theorem’s scope.

Definition 3.2 (Substitution method). The *substitution method* solves a recurrence in two steps. (i) *Guess* a closed-form upper bound (or matching lower bound) for $T(n)$, typically of the form $T(n) \leq cg(n) - h(n)$ for some constants c and lower-order term h . (ii) *Verify* the guess by induction on n : confirm the base case and, in the inductive step, substitute the inductive hypothesis on $T(n/b)$ into the recurrence and check that the bound on $T(n)$ follows. The lower-order h is included to make the inductive step go through; without it the induction often fails by a constant slack (see Example 3.4).

Definition 3.3 (Recursion-tree method). The *recursion-tree method* visualises the unrolled recurrence as a rooted tree: the root is labelled $f(n)$ (the local cost at the top call), each node of cost $f(m)$ has a children of cost $f(m/b)$, and the leaves correspond to base cases. The total cost $T(n)$ equals the sum of all node labels plus the cost at the leaves. Each level of the tree contributes a geometric or polynomial sum, and the dominant level identifies the asymptotic order of $T(n)$. Concretely, level i (root is level 0) contains a^i nodes each of cost $f(n/b^i)$, the tree has height $\log_b n$, and the number of leaves is $a^{\log_b n} = n^{\log_b a}$. The exponent $d := \log_b a$ is called the *critical exponent* and governs which of the master theorem’s three cases applies.

3.2 Substitution method

The substitution method is the most general – and therefore the most error-prone – technique. It is the one to reach for when the recurrence falls outside the master theorem (mixed splits, subtractive recurrences, recurrences with unusual driving functions). The single most common error is to drop the lower-order term in the guess: trying to prove $T(n) \leq cg(n)$ directly often fails because the inductive step leaves an unabsorbable additive residual. Strengthening the hypothesis

to $T(n) \leq cg(n) - h(n)$ for a suitably chosen lower-order h gives the inductive step exactly the slack needed to close. Example 3.4 illustrates the failure mode and the fix.

Example 3.4 ($T(n) = 2T(n/2) + n$ proves $T(n) = O(n \log n)$). We show by induction that the mergesort recurrence $T(n) = 2T(\lfloor n/2 \rfloor) + n$ with $T(1) = 1$ satisfies $T(n) \leq cn \log n$ for some constant $c > 0$ and all $n \geq n_0$.

Guess. $T(n) \leq cn \log n$.

Inductive step. Assume $T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \log \lfloor n/2 \rfloor \leq c(n/2) \log(n/2)$. Then

$$T(n) \leq 2 \cdot c(n/2) \log(n/2) + n = cn(\log n - 1) + n = cn \log n - cn + n \leq cn \log n$$

provided $-cn + n \leq 0$, i.e. $c \geq 1$. So the guess holds for any $c \geq 1$ in the inductive step.

Base case. For $n_0 = 2$, $T(2) = 2T(1) + 2 = 4$ and $c \cdot 2 \log 2 = 2c$, so $c \geq 2$ suffices. Choosing $c = 2$ closes the induction; hence $T(n) \leq 2n \log n = O(n \log n)$. A symmetric argument with $T(n) \geq cn \log n$ gives the matching lower bound, so $T(n) = \Theta(n \log n)$.

Why the lower-order term is needed. Guessing $T(n) \leq c \cdot n$ fails: the inductive step yields $T(n) \leq cn + n = (c+1)n$, which is not of the form cn – the induction hypothesis must be *exactly* the form one is trying to prove, including constants. The remedy is to subtract a lower-order term: strengthening the guess to $T(n) \leq cn \log n - dn$ absorbs the additive $+n$ in the inductive step. Conversely, guessing $T(n) \leq cn^2$ instead of $cn \log n$ “works” but is not tight: the substitution closes for any $c \geq 1$ but the bound is asymptotically wasteful.

3.3 Recursion-tree method

The recursion tree is the right tool when one wants insight into *which level dominates* – the root, the leaves, or a uniform contribution across all $\log_b n$ levels. It also doubles as a heuristic for guessing the right ansatz for substitution.

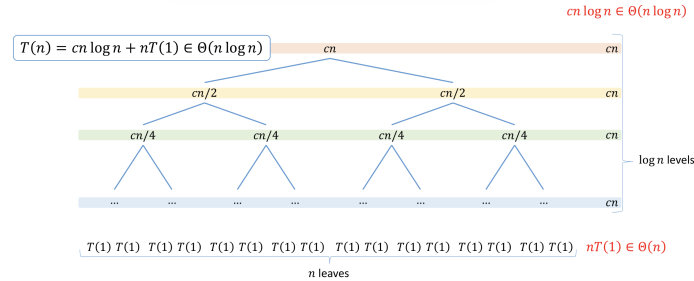


Figure 1: A recursion tree for $T(n) = aT(n/b) + f(n)$. Level i contains a^i nodes, each contributing $f(n/b^i)$; the tree has $\log_b n$ levels and $n^{\log_b a}$ leaves.

Example 3.5 (Recursion tree for $T(n) = 2T(n/2) + cn$). Consider $T(n) = 2T(n/2) + cn$ with $T(1) = \Theta(1)$, drawn as in fig. 1. The root contributes cn . Each of the two children contributes $cn/2$, so level 1 totals $2 \cdot cn/2 = cn$. By induction, level i totals $2^i \cdot c(n/2^i) = cn$. The tree has $\log_2 n$ levels and n leaves, each of cost $T(1) = \Theta(1)$. Summing,

$$T(n) = \underbrace{cn \cdot \log_2 n}_{\text{splitting/combining work}} + \underbrace{n \cdot \Theta(1)}_{\text{base cases}} = \Theta(n \log n).$$

The cost is uniform across levels; the log factor is exactly the height. This is the canonical case-2 picture.

3.4 Master Theorem

The master theorem is a *look-up table* for divide-and-conquer recurrences. Given $T(n) = aT(n/b) + f(n)$ with $a \geq 1$ and $b > 1$, one computes the critical exponent $d = \log_b a$, compares $f(n)$ to n^d , and reads off the answer. The decision procedure is summarised in fig. 2; the case-3 branch additionally requires a *regularity condition* on f .

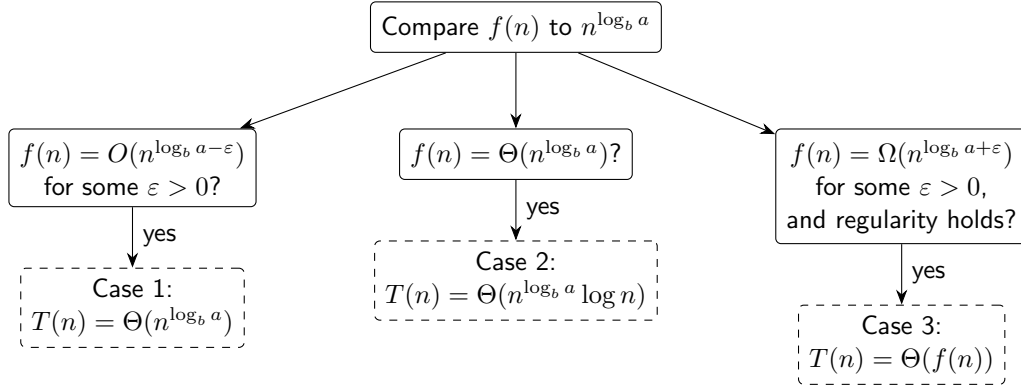


Figure 2: Master Theorem – case selection. The critical exponent $d = \log_b a$ counts leaves of the recursion tree; $f(n)$ is the per-call splitting/combining cost.

Theorem 3.6 (Master theorem). Let $T(n) = aT(n/b) + f(n)$ with constants $a \geq 1$, $b > 1$, and $f(n) \in \Omega(1)$. Set the critical exponent $d = \log_b a$. Then:

- (1) Leaves dominate. If $f(n) = O(n^{d-\epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^d)$.
- (2) Comparable. If $f(n) = \Theta(n^d \log^k n)$ for some constant $k \geq 0$, then $T(n) = \Theta(n^d \log^{k+1} n)$.
- (3) Root dominates. If $f(n) = \Omega(n^{d+\epsilon})$ for some constant $\epsilon > 0$, and the regularity condition $a f(n/b) \leq c f(n)$ holds for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$.

Proof. (sketch.) Unroll the recurrence to a tree of height $\log_b n$, in which level i contains a^i nodes each of cost $f(n/b^i)$, with $a^{\log_b n} = n^d$ leaves of cost $T(1)$. The total cost is

$$T(n) = \underbrace{\sum_{i=0}^{\log_b n - 1} a^i f(n/b^i)}_{\text{splitting/combining}} + \underbrace{n^d \cdot T(1)}_{\text{base cases}}.$$

The base-case total is $\Theta(n^d)$. The splitting/combining sum is the geometric series whose ratio between successive levels is a/b^e where e is the exponent of f .

Case 1. If $f(n) = O(n^{d-\epsilon})$, then $a^i f(n/b^i) \leq c \cdot n^{d-\epsilon} \cdot (a/b^{d-\epsilon})^i = c n^{d-\epsilon} \cdot b^{i\epsilon}$ (using $b^d = a$). Summing the geometric series in $b^\epsilon > 1$ over $\log_b n$ levels gives $O(n^{d-\epsilon} \cdot b^{\epsilon \log_b n}) = O(n^{d-\epsilon} \cdot n^\epsilon) = O(n^d)$. The leaf contribution $\Theta(n^d)$ dominates, so $T(n) = \Theta(n^d)$.

Case 2. With $f(n) = \Theta(n^d \log^k n)$, level i contributes $a^i \cdot \Theta((n/b^i)^d \log^k(n/b^i)) = \Theta(n^d \log^k(n/b^i))$, since $a^i (n/b^i)^d = n^d$. Summing over $i = 0, \dots, \log_b n - 1$ gives $\Theta(n^d) \cdot \sum_{i=0}^{\log_b n - 1} (\log n - i \log b)^k = \Theta(n^d \log^{k+1} n)$ (the standard sum $\sum_{j=1}^m j^k = \Theta(m^{k+1})$ for $k \geq 0$). The leaf cost $\Theta(n^d)$ is dominated by this, hence $T(n) = \Theta(n^d \log^{k+1} n)$. The familiar $k = 0$ subcase recovers $T(n) = \Theta(n^d \log n)$.

Case 3. The regularity condition $af(n/b) \leq cf(n)$ with $c < 1$ means the splitting cost shrinks geometrically as we descend the tree, so iteratively $a^i f(n/b^i) \leq c^i f(n)$. The total splitting/combining cost is at most $f(n) \sum_{i \geq 0} c^i = f(n)/(1 - c) = O(f(n))$, and at least $f(n)$ (the root alone). The leaf cost $n^d \cdot T(1)$ is $o(f(n))$ because $f(n) = \Omega(n^{d+\varepsilon})$. Hence $T(n) = \Theta(f(n))$.

Remark. Floors and ceilings can be ignored: for the recurrence $T(n) = aT(\lfloor n/b \rfloor) + f(n)$ the same conclusions hold, because the difference $\lfloor n/b \rfloor$ vs. n/b contributes only a lower-order perturbation. The condition $f(n) = \Omega(n^{d+\varepsilon})$ in case 3 is in fact *redundant* given the regularity condition (the regularity condition by itself implies a polynomial separation), but stating both is the standard convention.

Remark 3.7 (Hypotheses, gaps, and fallbacks). The form of Theorem 3.6 above requires $a \geq 1$, $b > 1$, and a non-negative driving function f . Recurrences like $T(n) = (1/2)T(n/2) + n$ ($a < 1$) and subtractive recurrences like $T(n) = T(n-1) + n$ or $T(n) = T(n-1) + T(1) + cn$ (the worst-case quicksort recurrence) are *not* of the form $aT(n/b) + f(n)$ and must be solved by telescoping or a direct recursion-tree sum – the worst-case quicksort recurrence, for instance, gives $\Theta(n^2)$, not $\Theta(n \log n)$.

Beyond hypothesis failures, even within the theorem's scope the standard statement does not cover every regime. The case-2 clause of Theorem 3.6 is restricted to $k \geq 0$. The *extended master theorem* handles the remaining $\log^k$ exponents: for $f(n) = \Theta(n^d \log^k n)$ with $k > -1$ the conclusion $T(n) = \Theta(n^d \log^{k+1} n)$ still holds; with $k = -1$, $T(n) = \Theta(n^d \log \log n)$; with $k < -1$, $T(n) = \Theta(n^d)$. These are extensions beyond Theorem 3.6. For exotic driving functions outside every $\log^k$ -class – for example $f(n) = 2^{\sqrt{\log n}}$ in $T(n) = T(n/2) + f(n)$ – the recursion-tree method computes the actual sum directly. The Akra–Bazzi theorem generalises the master theorem to mixed-split recurrences such as $T(n) = T(n/3) + T(2n/3) + n$ and to a wider class of driving functions, at the cost of an integral evaluation in place of the case look-up.

Remark 3.8 (Case 3's regularity condition is non-negotiable). The polynomial-separation clause $f(n) \in \Omega(n^{d+\varepsilon})$ alone does not suffice for case 3. Even when polynomial separation holds, the regularity condition $af(n/b) \leq cf(n)$ for some $c < 1$ is a separate requirement that must be verified independently: it encodes the geometric decay of the per-level cost as one descends the recursion tree, and without it the proof of case 3 breaks down. Always verify regularity explicitly with a concrete $c < 1$, as illustrated by the verification step in Example 3.11.

Example 3.9 (Master theorem case 1: $T(n) = 9T(n/3) + n$). Here $a = 9$, $b = 3$, so $d = \log_3 9 = 2$. The driving function is $f(n) = n = O(n^{2-1}) = O(n^{d-\varepsilon})$ for $\varepsilon = 1 > 0$. We are in case 1 of Theorem 3.6, so $T(n) = \Theta(n^d) = \Theta(n^2)$. Intuitively the work is leaf-dominated: there are $9^{\log_3 n} = n^2$ leaves, each contributing $\Theta(1)$, which already accounts for $\Theta(n^2)$ work; the per-level $f(n)$ is too small to add anything beyond a constant factor.

Example 3.10 (Master theorem case 2: $T(n) = 2T(n/2) + n$ (mergesort)). Here $a = 2$, $b = 2$, so $d = \log_2 2 = 1$. The driving function is $f(n) = n = \Theta(n^1 \log^0 n) = \Theta(n^d \log^k n)$ with $k = 0 \geq 0$. Case 2 of Theorem 3.6 gives $T(n) = \Theta(n^d \log^{k+1} n) = \Theta(n \log n)$. This reproduces the recursion-tree analysis of Example 3.5: every level of the tree contributes cn , there are $\log n$ levels, and the leaves contribute another $\Theta(n)$ subsumed by the level sum. Mergesort, the standard divide-and-conquer presentation of any $O(n)$ -work merge step, the in-place mergesort, and Strassen's matrix multiplication all have case-2-style recurrences.

Example 3.11 (Master theorem case 3: $T(n) = 3T(n/4) + n \log n$). Here $a = 3$, $b = 4$, so $d = \log_4 3 \approx 0.793$. The driving function is $f(n) = n \log n$. We check case 3.

Polynomial separation. For any ε with $0 < \varepsilon < 1 - \log_4 3 \approx 0.207$, $f(n) = n \log n = \Omega(n^1) = \Omega(n^{d+\varepsilon})$, since $n \log n / n^{d+\varepsilon} = n^{1-d-\varepsilon} \log n \rightarrow \infty$ as $n \rightarrow \infty$. So $f(n) \in \Omega(n^{d+\varepsilon})$.

Regularity. Need $a f(n/b) \leq c f(n)$ for some $c < 1$. Compute

$$a f(n/b) = 3 \cdot (n/4) \log(n/4) = (3/4)n (\log n - 2) \leq (3/4)n \log n = (3/4) f(n).$$

So the regularity condition holds with $c = 3/4 < 1$. Both clauses of case 3 are satisfied; Theorem 3.6 concludes $T(n) = \Theta(f(n)) = \Theta(n \log n)$. The work is root-dominated: at the top level we already pay $n \log n$, and the sum across all $\log_4 n$ levels is bounded by $(1 + 3/4 + (3/4)^2 + \dots) \cdot f(n) = O(f(n))$.

4 Correctness

An algorithm is *correct* on a given input if, when executed, it halts and produces the specified output, and *correct* (without qualification) if it is correct on every legal input. Correctness has two faces: *partial correctness* (if the algorithm halts, the output satisfies the post-condition) and *termination*; together these constitute *total correctness*. Iterative algorithms admit a uniform partial-correctness template – the loop-invariant theorem – with termination argued separately by exhibiting a non-negative integer variant. Recursive algorithms are handled by strong induction on input size.

4.1 Definitions

Definition 4.1 (Loop invariant). A *loop invariant* for a loop L is a predicate I on the program state such that I holds at the start of every iteration of L (equivalently, every time control reaches the loop guard). The invariant is the inductive hypothesis used to prove partial correctness of the loop: it must be (a) true on entry, (b) preserved by each pass of the loop body, and (c) strong enough that, combined with the negated guard, it implies the desired post-condition when the loop exits. Choosing the right invariant is the only creative step in a loop-invariant proof; everything else is mechanical.

Definition 4.2 (Partial correctness). An algorithm $\mathcal{A}$ is *partially correct* with respect to a precondition P and post-condition Q if, for every input satisfying P , if $\mathcal{A}$ halts on that input then the output satisfies Q . Partial correctness says nothing about whether the algorithm halts; an algorithm that loops forever on every input is vacuously partially correct.

Definition 4.3 (Total correctness). An algorithm $\mathcal{A}$ is *totally correct* with respect to (P, Q) if, for every input satisfying P , $\mathcal{A}$ halts and the output satisfies Q . Equivalently, total correctness is partial correctness (Definition 4.2) plus termination. The two are typically established by separate arguments: the loop invariant or strong induction handles partial correctness, while a termination measure (Definition 4.4) handles termination.

Definition 4.4 (Termination measure / variant). A *termination measure* (or *loop variant*) for a loop L is a function μ from the program state into $\mathbb{N}$ (or, more generally, into a well-founded set such as $(\mathbb{N}, <)$) such that:

- (i) $\mu \geq 0$ at every point in the execution of L ;
- (ii) μ *strictly decreases* on every pass of the loop body, i.e. if μ_t is the value at the start of iteration t , then $\mu_{t+1} < \mu_t$.

Since a strictly decreasing sequence of non-negative integers cannot be infinite, the existence of such a μ proves that L executes only finitely many iterations. For recursive procedures the analogue is a measure on the input that strictly decreases on every recursive call; well-foundedness then forbids infinite recursion. Typical choices: $\mu = n - i$ for a **for** loop running from i to n ; $\mu = \text{ub} - \text{lb} + 1$ for binary search; $\mu = |V \setminus R|$ for Dijkstra's main loop.

Remark 4.5 (Well-founded, not merely decreasing). The codomain of μ must be well-founded; strict decrease alone is not enough. A strictly decreasing sequence in $\mathbb{Q}$ or $\mathbb{R}$ can be infinite $(1, 1/2, 1/4, \dots)$, so picking $\mathbb{N}$ (or any well-founded order) is essential. Conversely, “the value of i never repeats” does not suffice – i must *decrease* (or progress monotonically toward a fixed bound).

Definition 4.6 (Strong induction). *Strong induction* (also *course-of-values induction*) on $\mathbb{N}$: to prove that a predicate $P(n)$ holds for every $n \geq 0$, it suffices to prove

$$\forall n \in \mathbb{N}, (\forall k < n, P(k)) \implies P(n).$$

The base case $P(0)$ is the instance of this implication at $n = 0$, where the antecedent $\forall k < 0, P(k)$ is vacuous. Strong induction differs from ordinary induction only in the inductive hypothesis: instead of assuming $P(n - 1)$ alone, we may assume $P(k)$ for *all* $k < n$. The two principles are logically equivalent, but strong induction is the natural fit when a recursive procedure makes calls of varying smaller sizes (e.g. $\text{Fib}(n)$ calls both $\text{Fib}(n - 1)$ and $\text{Fib}(n - 2)$).

Definition 4.7 (Structural induction). *Structural induction* generalises strong induction from $\mathbb{N}$ to any *inductively defined* (algebraic) structure: lists, trees, formulas, etc. To prove that a predicate P holds for every element of such a structure, one proves P for each base constructor (e.g. the empty list, a leaf), and proves P for each recursive constructor under the assumption that P holds for every immediate substructure (e.g. the tail of a list, the children of an internal node). Termination of recursion on the structure is automatic because the substructures are strictly smaller in the well-founded ordering induced by the constructors. In algorithm analysis, most correctness proofs reduce to strong induction (Definition 4.6) on the integer input size; structural induction is the right tool whenever the input is naturally a tree or a recursive data type.

4.2 Loop invariants

The loop-invariant pattern reduces partial correctness of an iterative algorithm to a three-line induction.

Theorem 4.8 (Loop-invariant theorem). *Let L be a loop with guard G and let I be a predicate on the program state. Suppose:*

- (i) (Initialisation) *I holds at the start of the first iteration;*
- (ii) (Maintenance) *for every iteration t , if I and G hold at the start of t , then I holds at the start of iteration $t + 1$ (equivalently, at the end of iteration t);*
- (iii) (Termination) *the loop performs only finitely many iterations.*

Then, when L exits, the conjunction $I \wedge \neg G$ holds. Consequently, if $I \wedge \neg G$ entails the desired post-condition, the loop is partially correct, and – given (iii) – totally correct.

Proof. Let I_t denote “ I holds at the start of iteration t ”. We prove by induction on t that I_t holds for every $t = 1, 2, \dots, T$, where T is the total number of iterations (finite by (iii)). The base case $t = 1$ is exactly (i). For the inductive step, suppose I_t holds. Either iteration t is the last one (the

guard becomes false, and I_t remains true at exit, so $I_{T+1} := I\text{-at-exit}$ holds), or iteration t executes the body and control returns to the guard; by (ii) this implies I_{t+1} . By induction, I holds at the moment the loop exits. At that moment the guard is false (else the loop would have continued), so $I \wedge \neg G$ holds, as claimed. The post-condition then follows by the assumed entailment $I \wedge \neg G \models Q$.

Remark 4.9 (Three obligations $\leftrightarrow$ induction). The structure of Theorem 4.8 is exactly mathematical induction on iteration count: initialisation is the base case, maintenance is the inductive step, and the “termination” clause names the conclusion drawn at exit (not to be confused with proving the loop terminates – that is a separate obligation handled by a variant, Definition 4.4). Some authors fold the post-condition entailment into a fourth bullet called “termination”; we bundle “invariant survives plus guard fails $\Rightarrow$ correct output” into a single observation.

Remark 4.10 (Common loop-invariant pitfalls). Three slips recur. First, partial correctness is not total correctness: the loop-invariant theorem gives only conditional correctness, so a separate termination argument (a variant in $\mathbb{N}$ that strictly decreases, Definition 4.4) is still owed. Second, the invariant must survive *exit*: it is $I \wedge \neg G$ at the moment the guard fails that must imply the post-condition, so a candidate I too weak to do so when conjoined with $\neg G$ is not yet finished. Third, the boundary case of zero iterations matters: if the body is never executed (e.g. $n = 0$ in selection sort, or $\text{lb} > \text{ub}$ in binary search), initialisation and the negated guard alone must already entail the post-condition, typically by appealing to vacuous truth (an empty prefix is sorted, an empty range contains no element) – but this should be stated explicitly.

Example 4.11 (Selection sort – finding the right invariant). We prove the partial correctness of `SELECTIONSORT`, the sorting algorithm given in algorithm 1.

Algorithm 1 `SELECTIONSORT`($A[1..n]$).

```

1: for  $j = 1$  to  $n - 1$  do
2:   Select  $s \in \{j, j + 1, \dots, n\}$  such that  $A[s]$  is a smallest entry in  $A[j..n]$ .
3:   Swap  $A[j]$  and  $A[s]$ .
4: end for

```

Choice of invariant. Three candidate invariants “at the start of iteration j ” suggest themselves:

- (a) “ A is sorted.” *Fails initialisation*: a generic input is not sorted at the start of $j = 1$.
- (b) “ $A[1..j - 1]$ is sorted.” Initialisation and termination check out (at exit $j = n$, so $A[1..n - 1]$ is sorted, and one more element being out of place would break the post-condition only if $A[n]$ were smaller than $A[n - 1]$). *Fails maintenance*: nothing forces $A[j - 1] \leq A[j]$, since the prefix sorted-ness of $A[1..j - 1]$ does not constrain the relationship between the prefix and the suffix.
- (c) “ $x \leq y$ for all $x \in A[1..j - 1]$ and $y \in A[j..n]$.” *Fails termination*: at exit, this only says the largest of the first $n - 1$ entries is $\leq A[n]$, which does not imply that the prefix itself is sorted.

The fix is to conjoin (b) and (c). The correct invariant I_j is:

$$A[1..j - 1] \text{ is sorted} \quad \wedge \quad x \leq y \text{ for all } x \in A[1..j - 1], y \in A[j..n].$$

Initialisation. At $j = 1$, the prefix $A[1..0]$ is empty, so both conjuncts hold vacuously.

Maintenance. Suppose I_j holds at the start of iteration j . Iteration j picks $s \in \{j, \dots, n\}$ with $A[s] = \min A[j..n]$ and swaps $A[j]$ and $A[s]$. After the swap:

- $A[1..j-1]$ is unchanged and remains sorted.
- The new $A[j]$ equals the old $A[s] = \min A[j..n]$, which by the second conjunct of I_j is $\geq$ every $x \in A[1..j-1]$ (since each such x was $\leq$ every entry in $A[j..n]$). Hence $A[j-1] \leq A[j]$, so $A[1..j]$ is sorted.
- The new $A[j]$ is also $\leq$ every entry remaining in $A[j+1..n]$ (it was the minimum of $A[j..n]$). Combined with the second conjunct of I_j , every entry of $A[1..j]$ is $\leq$ every entry of $A[j+1..n]$.

Both conjuncts of I_{j+1} hold.

Termination (post-condition). The loop exits with $j = n$, at which point I_n asserts that $A[1..n-1]$ is sorted and every entry of $A[1..n-1]$ is $\leq A[n]$. Together this means $A[1..n]$ is sorted, which is the post-condition. (Termination of the loop itself is immediate: the **for** loop runs $n-1$ times.)

Why each candidate failed. The lesson is that an invariant must be *strong enough* to imply the post-condition at exit and *weak enough* to be true at entry; (b) was too weak at the right (termination), while (a) was too strong at the left (initialisation). Conjoining (b) and (c) is the standard fix: a sorted prefix together with “every prefix entry $\leq$ every suffix entry” is exactly the invariant that survives both endpoints. The methodological point generalises: pick the invariant *before* writing the proof. Reverse-engineering one from the maintenance step is a common error. The candidate-checking exercise above – write several candidates, mark which of (init / maint / term) each one satisfies, and conjoin until all three boxes are ticked – is the right discipline.

4.3 Termination

Partial correctness is independent of termination, and termination is usually proved separately by exhibiting a variant.

Example 4.12 (Binary search – termination via a strictly decreasing variant). Consider `BINARYSEARCH(A, lb, ub, x)` in algorithm 2, which decides whether x occurs in the sorted slice $A[lb..ub]$.

Algorithm 2 `BINARYSEARCH(A, lb, ub, x)`.

```

1: if  $lb > ub$  then return NO
2: end if
3:  $mid \leftarrow \lfloor (lb + ub)/2 \rfloor$ 
4: if  $x = A[mid]$  then return YES
5: else if  $x > A[mid]$  then return BINARYSEARCH(A, mid + 1, ub, x)
6: else if  $x < A[mid]$  then return BINARYSEARCH(A, lb, mid - 1, x)
7: end if

```

Termination measure. Let $n = ub - lb + 1$ be the size of the searched slice. Set $\mu = \max(n, 0)$; then $\mu \in \mathbb{N}$.

μ strictly decreases at every recursive call. Let $m = \lfloor (lb + ub)/2 \rfloor$, so $lb \leq m \leq ub$ when the call is made (the early return at $lb > ub$ ensures $n \geq 1$ in this branch). The two recursive calls have new slice sizes

$$n_{\text{right}} = ub - (m + 1) + 1 = ub - m, \quad n_{\text{left}} = (m - 1) - lb + 1 = m - lb.$$

Both are $\leq \lfloor n/2 \rfloor \leq n - 1 < n$ (when $n \geq 1$), so μ strictly decreases.

$\mu \geq 0$. Immediate from the definition.

Conclusion. A strictly decreasing sequence in $\mathbb{N}$ cannot be infinite, so the recursion bottoms out after at most μ_0 levels, where μ_0 is the initial value. In fact μ at least halves each level, so the recursion depth is $O(\log n)$, matching the well-known $O(\log n)$ time bound. The same variant $\text{ub} - \text{lb} + 1$ also works for the half-open-interval variant of binary search; the choice of $+1$ vs -1 affects only the off-by-one shift in the base case.

4.4 Induction for recursive algorithms

For a recursive procedure $\mathcal{A}$, correctness on every input reduces to two obligations: (a) the no-recursion (base) branches are correct, and (b) for every recursive branch, *if* all recursive calls return correct answers, then so does the surrounding code. The combination is precisely strong induction (Definition 4.6) on input size.

Theorem 4.13 (Strong induction principle). *Let $P(n)$ be a predicate on $\mathbb{N}$. If*

$$\forall n \in \mathbb{N}, (\forall k < n, P(k)) \implies P(n),$$

then $P(n)$ holds for every $n \in \mathbb{N}$.

Proof. Define $Q(n) := \forall k \leq n, P(k)$. We show $Q(n)$ for all n by ordinary induction. *Base* $n = 0$: the hypothesis at $n = 0$ has the vacuous antecedent $\forall k < 0, P(k)$, so it directly yields $P(0)$, hence $Q(0)$. *Step*: suppose $Q(n)$, i.e. $P(k)$ for all $k \leq n$; equivalently $P(k)$ for all $k < n + 1$. The hypothesis at $n + 1$ then gives $P(n + 1)$, so $Q(n + 1)$ holds. By ordinary induction $Q(n)$ holds for every n , hence so does $P(n)$.

The same argument works for any well-founded relation in place of $<$ on $\mathbb{N}$, which is what justifies structural induction (Definition 4.7) on inductively defined data types.

Remark 4.14 (When to reach for strong induction, and how many base cases). If the recursive call is on a single argument with size exactly $n - 1$ (e.g. a tail-recursive linear scan), ordinary induction suffices. Strong induction is the right tool when the call is on a smaller-by-an-arbitrary-amount argument (binary search's $\lfloor n/2 \rfloor$) or when there are multiple recursive calls (Fibonacci, merge sort). A recursion that refers to both $f(n - 1)$ and $f(n - 2)$ also requires base cases at *both* $n = 0$ and $n = 1$; otherwise the first recursive step appeals to $f(-1)$.

Example 4.15 (Recursive Fibonacci – correctness by strong induction). Let $\text{Fib}(n)$ be defined by $\text{Fib}(0) = 0$, $\text{Fib}(1) = 1$, and $\text{Fib}(n) = \text{Fib}(n - 1) + \text{Fib}(n - 2)$ for $n \geq 2$. Consider the recursive algorithm

$$\text{FIB}(n) := \begin{cases} n & \text{if } n \leq 1, \\ \text{FIB}(n - 1) + \text{FIB}(n - 2) & \text{otherwise.} \end{cases}$$

We show that $\text{FIB}(n) = \text{Fib}(n)$ for all $n \geq 0$ by strong induction (Theorem 4.13) on n .

Base cases. For $n = 0$: $\text{FIB}(0) = 0 = \text{Fib}(0)$. For $n = 1$: $\text{FIB}(1) = 1 = \text{Fib}(1)$. Both are the $n \leq 1$ branch and involve no recursion. Two base cases are needed because the recursive branch refers to $\text{FIB}(n - 2)$ and $\text{FIB}(n - 1)$ simultaneously.

Inductive step. Fix $n \geq 2$ and assume $\text{FIB}(k) = \text{Fib}(k)$ for every k with $0 \leq k < n$. Since $n \geq 2$ the algorithm executes the recursive branch:

$$\text{FIB}(n) = \text{FIB}(n - 1) + \text{FIB}(n - 2) \stackrel{\text{IH}}{=} \text{Fib}(n - 1) + \text{Fib}(n - 2) = \text{Fib}(n),$$

where the second equality uses the inductive hypothesis at the two strictly smaller indices $n - 1$ and $n - 2$ (both ≥ 0 because $n \geq 2$), and the third is the recurrence defining Fib. By Theorem 4.13, $\text{FIB}(n) = \text{Fib}(n)$ for every $n \geq 0$.

Termination. Each recursive call strictly decreases the input ($n \mapsto n - 1$ and $n \mapsto n - 2$, both non-negative once $n \geq 2$), and the base branch returns at $n \leq 1$. The variant $\mu = n$ is non-negative and strictly decreases on every recursive call, so by Definition 4.4 the recursion terminates. Total correctness follows.

Why strong (not ordinary) induction. The recursive branch invokes the algorithm at *two* smaller arguments, $n - 1$ and $n - 2$. The inductive hypothesis must cover both, which is exactly the strength advantage of strong induction over $P(n - 1) \Rightarrow P(n)$. The same pattern applies to merge sort (recursion on the two halves of size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$, both $< n$ for $n \geq 2$), to recursive binary search (induction on slice size $n = \text{ub} - \text{lb} + 1$, base case $n \leq 0$ returning **NO**, inductive step splitting on x vs $A[\text{mid}]$ and invoking strong induction on a strictly smaller slice), and to fast exponentiation $\text{power}(x, n) = \text{power}(x, \lfloor n/2 \rfloor)^2 \cdot x^{n \bmod 2}$.

Part II

Algorithm Design Paradigms

5 Divide and Conquer

Divide-and-conquer (D&C) refines the brute-force / iterative approach by reducing an instance of size n to several instances of the *same problem* on strictly smaller inputs, recursing, and stitching the sub-answers back together. Running times then satisfy a recurrence of the canonical shape $T(n) = aT(n/b) + f(n)$ solved by the Master Theorem (Theorem 3.6), and correctness follows by strong induction on n . The asymptotic cost is determined by a competition between the number of subproblems a and the combine cost $f(n)$, a principle exploited by Strassen's algorithm and many others.

5.1 Definitions

Definition 5.1 (Divide-and-conquer paradigm). A *divide-and-conquer* algorithm for a problem with input size n proceeds in three steps:

- (1) **Divide.** Split the instance into $a \geq 1$ subinstances, each of size at most n/b for some constant $b > 1$ (so subinstances are *strictly smaller*).
- (2) **Conquer.** Solve each subinstance recursively. When n falls below a constant threshold the recursion is terminated by a direct base-case computation in $\Theta(1)$ time.
- (3) **Combine.** Stitch the a subinstance answers into an answer for the original instance.

Writing $f(n)$ for the time spent on divide+combine on an instance of size n , the worst-case running time satisfies the canonical recurrence

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq n_0, \\ aT(n/b) + f(n) & \text{if } n > n_0, \end{cases}$$

for the constant base-case threshold n_0 . The recursion-tree shape is determined by (a, b) and the leaf-vs-root cost balance is determined by $f(n)$ versus $n^{\log_b a}$ (Theorem 3.6).

Remark 5.2 (Why $b > 1$ strictly). The strict inequality $b > 1$ is what makes D&C terminate: each call is on an input strictly smaller than its parent, so after $\log_b n$ levels the recursion bottoms out. A recurrence such as $T(n) = 2T(n/1) + n$ or $T(n) = T(n) + 1$ does not describe a D&C algorithm and does not terminate; this is the same reason recursive correctness proofs require the induction to be on a well-founded measure. When setting up the recurrence, always verify $n/b < n$ for the chosen b .

Remark 5.3 (State the combine cost explicitly). A common slip is to write $T(n) = aT(n/b)$ with no $f(n)$ term, which silently asserts that the divide and combine steps cost $O(1)$. They rarely do: for naive matrix multiplication the combine is $\Theta(n^2)$, for Strassen's algorithm it is also $\Theta(n^2)$ (eighteen block additions), and for the closest-pair-of-points problem it is $\Theta(n)$ via the strip argument. Always identify $f(n)$ before invoking the Master Theorem.

5.2 Key results

Theorem 5.4 (Correctness template for D&C). *Let $\mathcal{A}$ be a D&C algorithm in the form of Definition 5.1, with base case correct by direct inspection and a combine step which, given correct answers for the a subinstances, produces a correct answer for the parent instance. Then $\mathcal{A}$ is correct on every input.*

Proof. Strong induction on n , the size of the input.

Base case. For $n \leq n_0$, the algorithm returns the value computed by the direct base-case routine, which is correct by hypothesis.

Inductive step. Fix $n > n_0$ and assume (strong inductive hypothesis) that $\mathcal{A}$ is correct on every input of size $< n$. On an input of size n , $\mathcal{A}$ first divides into a subinstances each of size at most $n/b < n$. By the inductive hypothesis, the recursive calls return correct answers for these a subinstances. By the assumption on the combine step, the algorithm then produces a correct answer for the parent. Hence $\mathcal{A}$ is correct on the input of size n .

By strong induction, $\mathcal{A}$ is correct on every input.

Remark 5.5 (Match the base case to the algorithm). The strong-induction proof above only goes through if the base case in the proof matches what the algorithm actually does. If the recursion bottoms out at threshold $n_0 > 1$ but the analysis writes $T(1) = T(0) = \Theta(1)$ to argue termination, the recurrence is still numerically correct but the inductive step has no anchor at $n = n_0$. State the base case at the size the algorithm actually terminates at.

Proposition 5.6 (Worst-case time of a D&C algorithm). *Let $\mathcal{A}$ be a D&C algorithm whose running time satisfies the canonical recurrence $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$, f eventually positive. Let $c^* = \log_b a$. Then by the Master Theorem (Theorem 3.6):*

- if $f(n) \in O(n^{c^* - \varepsilon})$ for some $\varepsilon > 0$, then $T(n) \in \Theta(n^{c^*})$ (leaves dominate);
- if $f(n) \in \Theta(n^{c^*})$, then $T(n) \in \Theta(n^{c^*} \log n)$ (balanced);
- if $f(n) \in \Omega(n^{c^* + \varepsilon})$ for some $\varepsilon > 0$ and the regularity condition $af(n/b) \leq kf(n)$ holds for some $k < 1$ and all large n , then $T(n) \in \Theta(f(n))$ (root dominates).

Proof. (*sketch.*) This is exactly the Master Theorem (Theorem 3.6), which is proved in section 3 by summing the per-level work in the recursion tree. Each of the $\log_b n$ levels has a^k nodes at depth k , each doing work $f(n/b^k)$, so the total work is

$$T(n) = \sum_{k=0}^{\log_b n - 1} a^k f(n/b^k) + \Theta(a^{\log_b n}).$$

The leaf term equals $\Theta(n^{\log_b a}) = \Theta(n^{c^*})$. Whether the geometric-like sum on the left is dominated by its first term, last term, or grows uniformly across levels, is exactly the case split on $f(n)$ versus n^{c^*} .

Remark 5.7 (The two levers of D&C optimisation). Proposition 5.6 makes the two design levers explicit. To improve a D&C algorithm one can either (i) reduce a (fewer subproblems) which decreases $c^* = \log_b a$ and so shrinks the leaf term, or (ii) reduce $f(n)$ which only helps if $f(n)$ already dominates (Case 3) or matches (Case 2) the leaf term. In Case 1 the leaves already swamp the combine, so making the combine cheaper does not change the asymptotic cost. The recurrence $T(n) = 8T(n/2) + n^3$ illustrates this: original is $\Theta(n^3 \log n)$ (Case 2); dropping f from n^3 to n^2 gives $\Theta(n^3)$ via Case 1 (where f no longer matters), and dropping a from 8 to 7 also gives $\Theta(n^3)$ but via Case 3, where regularity must be checked.

Remark 5.8 (Regularity and the divide step). Case 3 of Theorem 3.6 requires the regularity condition $af(n/b) \leq kf(n)$ for some constant $k < 1$ and all large n . For polynomial f this is automatic; for f involving logarithms or other slowly-varying factors it must be verified, otherwise the Akra–Bazzi method (section 3) is needed. A separate trap is the cost of the divide step itself: for closest-pair and similar geometric problems, sorting points by x -coordinate inside every recursive call would push the recurrence to $T(n) = 2T(n/2) + n \log n$, giving $\Theta(n \log^2 n)$ instead of the optimal $\Theta(n \log n)$. The standard fix — pre-sort once, then pass sorted lists down through the recursion — is itself a D&C-design pattern.

5.3 Worked examples

Example 5.9 (Fast exponentiation by repeated squaring). Problem. Given a positive integer a and a non-negative integer n , compute a^n (in modular arithmetic, so each multiplication is $\Theta(1)$ in the word RAM model).

Naive approach. The recurrence $a^n = a^{n-1} \cdot a$ gives $T(n) = T(n-1) + \Theta(1)$, hence $T(n) \in \Theta(n)$. This is iteration, not D&C: the subinstance is of size $n-1$, not n/b .

D&C approach. Use the identity

$$a^n = \begin{cases} (a^{\lfloor n/2 \rfloor})^2 & \text{if } n \text{ is even,} \\ (a^{\lfloor n/2 \rfloor})^2 \cdot a & \text{if } n \text{ is odd.} \end{cases}$$

Divide: reduce computing a^n to a single subproblem of computing $a^{\lfloor n/2 \rfloor}$. *Conquer:* recurse to obtain $x = a^{\lfloor n/2 \rfloor}$. *Combine:* return $x \cdot x$ (or $x \cdot x \cdot a$ if n is odd), $\Theta(1)$ work.

Pseudocode.

```
function FASTEXP( $a, n$ )
  if  $n = 0$  then return 1
  end if
```

```

 $x \leftarrow \text{FASTEEXP}(a, \lfloor n/2 \rfloor)$ 
if  $n$  is even then return  $x \cdot x$ 
else return  $x \cdot x \cdot a$ 
end if
end function

```

Correctness. By Theorem 5.4: base case $n = 0$ returns $1 = a^0$. For $n > 0$, by strong induction the recursive call returns $x = a^{\lfloor n/2 \rfloor}$ correctly; the combine step then returns $x^2 = a^{2\lfloor n/2 \rfloor}$ which equals a^n when n is even and a^{n-1} when n is odd, in which case multiplying by a gives a^n .

Time. The recurrence is $T(n) = T(n/2) + \Theta(1)$, so $a = 1$, $b = 2$, $f(n) = \Theta(1)$, $c^* = \log_2 1 = 0$. Since $f(n) = \Theta(n^0) = \Theta(n^{c^*})$, Case 2 of Proposition 5.6 (Theorem 3.6) gives

$$T(n) \in \Theta(n^0 \log n) = \Theta(\log n).$$

This is an exponential improvement over the iterative $\Theta(n)$. The same idea, applied to a 2×2 companion matrix $M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$, gives an $O(\log n)$ -time algorithm for the n -th Fibonacci number via $M^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$.

Example 5.10 (Strassen's matrix multiplication, $\Theta(n^{\log_2 7})$). **Problem.** Given two $n \times n$ matrices A and B (assume n is a power of 2 for cleanliness), compute $C = A \cdot B$.

Naive D&C. Block-partition each matrix into four $(n/2) \times (n/2)$ blocks:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad B = \begin{pmatrix} e & f \\ g & h \end{pmatrix}, \quad C = \begin{pmatrix} r & s \\ t & u \end{pmatrix},$$

where the entries $a, \dots, h, r, \dots, u$ are themselves $(n/2) \times (n/2)$ matrices and

$$r = ae + bg, \quad s = af + bh, \quad t = ce + dg, \quad u = cf + dh.$$

Divide+combine cost: 4 block-additions of $(n/2)$ -square matrices, $\Theta(n^2)$ total. *Conquer:* 8 block-multiplications of $(n/2)$ -square matrices. The recurrence is $T(n) = 8T(n/2) + \Theta(n^2)$, with $c^* = \log_2 8 = 3$ and $f(n) = \Theta(n^2) = O(n^{3-1})$ in Case 1, hence $T(n) \in \Theta(n^3)$. No improvement over the standard triple-loop algorithm.

Strassen's trick. Following Proposition 5.6, the leaves dominate ($f(n) \ll n^{c^*}$), so the only way to improve the asymptotic is to reduce $a = 8$. Strassen (1969) computes $C = AB$ using only *seven* block-multiplications, at the cost of 18 block-additions/subtractions:

$$\begin{aligned} P_1 &= a(f - h), & P_5 &= (a + d)(e + h), \\ P_2 &= (a + b)h, & P_6 &= (b - d)(g + h), \\ P_3 &= (c + d)e, & P_7 &= (a - c)(e + f), \\ P_4 &= d(g - e), \end{aligned}$$

and then

$$r = P_5 + P_4 - P_2 + P_6, \quad s = P_1 + P_2, \quad t = P_3 + P_4, \quad u = P_5 + P_1 - P_3 - P_7.$$

Correctness. A direct expansion verifies each formula; for instance,

$$\begin{aligned} P_5 + P_4 - P_2 + P_6 &= (a + d)(e + h) + d(g - e) - (a + b)h + (b - d)(g + h) \\ &= ae + ah + de + dh + dg - de - ah - bh + bg + bh - dg - dh \\ &= ae + bg = r, \end{aligned}$$

and similarly for s, t, u . Note: the P_i are products of $(n/2)$ -square matrices, so they do *not* commute with each other, but each P_i is a single matrix product, so the algebraic identities only require associativity and distributivity, both of which matrix arithmetic satisfies. Combined with Theorem 5.4, the algorithm is correct.

Time. Now $a = 7$, $b = 2$, $f(n) = \Theta(n^2)$ (the 18 block-additions). So $c^* = \log_2 7 \approx 2.807$ and $f(n) = \Theta(n^2) = O(n^{c^* - \varepsilon})$ for any $\varepsilon < c^* - 2$. Case 1 of Proposition 5.6 gives

$$T(n) \in \Theta(n^{\log_2 7}) = \Theta(n^{2.807\dots}).$$

Remark. Strassen's algorithm is the proof-of-concept that fast matrix multiplication is possible; the matrix-multiplication exponent ω has been pushed down repeatedly (Coppersmith–Winograd 1990: $\omega < 2.376$; current record approximately 2.371552), but Strassen's $\log_2 7$ remains the standard textbook example because the construction is fully explicit.

6 Sorting and Lower Bounds

The sorting problem takes an array $A = (a_1, \dots, a_n)$ and a total order $\leq$ and returns a permutation $(a'_1, \dots, a'_n)$ with $a'_1 \leq a'_2 \leq \dots \leq a'_n$. Two questions organise the chapter: a decision-tree argument together with Stirling's approximation forces every comparison-based algorithm to use $\Omega(n \log n)$ comparisons in the worst case, and quicksort's expected running time on a uniformly random input permutation is $\Theta(n \log n)$ even though its worst case is $\Theta(n^2)$. Lifting the comparison-only restriction unlocks linear-time methods (counting sort, radix sort) whose running time depends on the size of the alphabet rather than on $n!$. Distinct keys are assumed throughout unless stated otherwise.

6.1 Definitions

Definition 6.1 (Comparison-based sort). A sorting algorithm is *comparison-based* if the only operation it performs on input keys is to compare two of them with one of $<, \leq, =, >, \geq$, branching on the boolean outcome. Arithmetic on keys, hashing of keys, bit-level inspection of keys, and array indexing by key value are all forbidden. Equivalently, the algorithm's behaviour on input $(a_1, \dots, a_n)$ depends only on the order type of the input, i.e. on which of the $n!$ relative orderings the input realises. Insertion sort, selection sort, merge sort, heap sort and quicksort are all comparison-based; counting sort and radix sort are not.

Definition 6.2 (In-place sort). A sorting algorithm is *in-place* if, beyond the input array, it uses only $O(1)$ or $O(\log n)$ auxiliary memory. Insertion sort and heap sort are in-place ($O(1)$ extra). A standard recursive quicksort is in-place ($O(\log n)$ stack depth, provided one always recurses on the smaller side first). The textbook merge sort, which allocates a length- n scratch array, is *not* in-place.

Definition 6.3 (Stable sort). A sorting algorithm is *stable* if it preserves the relative order of equal keys: whenever $A[i] = A[j]$ and $i < j$ in the input, the element originally at index i appears before the element originally at index j in the output. Insertion sort and (carefully implemented) merge sort are stable; the standard in-place quicksort partition is not. Stability matters when keys carry payload data (e.g. sorting records by one field while preserving an earlier sort by another field) and is the structural assumption that makes radix sort correct.

Definition 6.4 (Decision tree for a comparison-based sorting algorithm). Fix a comparison-based sorting algorithm $\mathcal{A}$ and an input length n . The *decision tree* of $\mathcal{A}$ on inputs of length n is a rooted binary tree in which:

- each internal node is labelled with a comparison “ $a_i : a_j$ ” that $\mathcal{A}$ might perform between two specific input positions;
- the two outgoing edges are labelled $\leq$ and $>$ (or analogous), corresponding to the two branches taken by $\mathcal{A}$ after that comparison;
- each leaf is labelled with the permutation $\mathcal{A}$ outputs along the unique root-to-leaf path that reaches it.

A specific input traces a unique root-to-leaf path; the number of comparisons made on that input equals the depth of the leaf. Worst-case running time is therefore at least the height of the tree.

6.2 Comparison-based lower bound

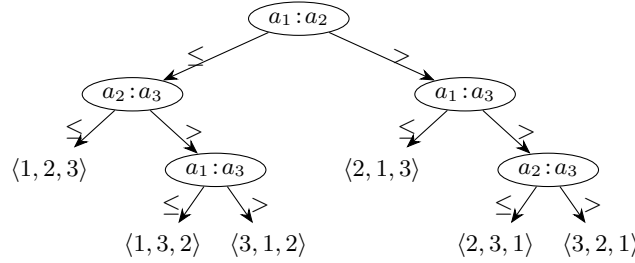


Figure 3: A comparison-sort decision tree for $n = 3$: every leaf is a permutation of the input; every internal node is a comparison.

Theorem 6.5 (Comparison-sorting lower bound). Any deterministic comparison-based sorting algorithm makes at least $\lceil \log_2(n!) \rceil = \Omega(n \log n)$ comparisons in the worst case on inputs of length n . Consequently its worst-case running time is $\Omega(n \log n)$.

Proof. Let $\mathcal{A}$ be any deterministic comparison-based sorting algorithm and let T be its decision tree on inputs of length n (Definition 6.4). We claim T has at least $n!$ leaves.

For each of the $n!$ orderings of the input there is a corresponding input – e.g. take the input to be the corresponding permutation of $(1, 2, \dots, n)$ – on which $\mathcal{A}$ must produce a *distinct* output permutation, namely the inverse permutation that sorts the input. Inputs that are different orderings cannot reach the same leaf, since a leaf labels a single permutation and $\mathcal{A}$ is correct on every input. Hence T has at least $n!$ leaves.

A binary tree of height h has at most 2^h leaves, so any tree with $\geq n!$ leaves satisfies $2^h \geq n!$, i.e. $h \geq \log_2(n!)$. By Stirling’s approximation (section 1, $\log(n!) \in \Theta(n \log n)$, more precisely $\log_2(n!) = n \log_2 n - n \log_2 e + O(\log n)$),

$$h \geq \log_2(n!) = n \log_2 n - n \log_2 e + O(\log n) \in \Omega(n \log n).$$

The worst-case running time of $\mathcal{A}$ is at least the worst-case number of comparisons, which is the height of the tree, hence $\Omega(n \log n)$.

Corollary 6.6 (Sharp counting consequence). *There is no comparison-based algorithm that sorts every 5-element array using ≤ 6 comparisons.*

Proof. (*sketch.*) A binary tree of height ≤ 6 has at most $2^6 = 64$ leaves, but $5! = 120 > 64$, so it cannot label every permutation. (Tight: a custom algorithm sorting 5 elements with 7 comparisons does exist.)

Remark 6.7 (Scope of the bound). The lower bound applies only to algorithms restricted to comparisons; quoting “sorting requires $\Omega(n \log n)$ ” as if it were universal is a textbook trap. Algorithms that look at bits of keys or use keys as array indices (counting sort, radix sort, bucket sort) escape it – the decision-tree model does not apply because their behaviour depends on key *values*, not just on the order type of the input. Merge sort and heap sort match the lower bound and are therefore *asymptotically optimal* among comparison-based sorts. The leaf-counting step of the proof rests on *correctness*: distinct input orderings demand distinct output permutations, so distinct leaves – a tree with fewer than $n!$ leaves would have to be wrong on some input. The same decision-tree counting argument generalises: merging k sorted arrays of length n each has $(kn)!/(n!)^k$ possible interleavings, giving the lower bound $\log_2((kn)!/(n!)^k) \in \Theta(kn \log k)$.

6.3 Quicksort average-case analysis

Quicksort picks a pivot from the array, partitions the array into elements $\leq$ pivot and elements $\geq$ pivot in $\Theta(n)$ time, and recurses on the two sides. With the pivot chosen as $A[1]$, the worst-case input is an already-sorted array (or its reverse), giving the recurrence $T(n) = T(n-1) + T(0) + cn$ and hence $T(n) \in \Theta(n^2)$. The point of the average-case analysis is to show that, over a uniformly random input permutation (equivalently, a random pivot), the expected running time is $\Theta(n \log n)$.

Theorem 6.8 (Average-case running time of quicksort). *Assume distinct keys. Let $A(n)$ denote the average running time of quicksort over the $n!$ equally likely input permutations of n keys. Then $A(n) \in O(n \log n)$.*

Proof. Conditional on the pivot being the j -th smallest of the n keys, the partition produces sub-arrays of sizes $j-1$ and $n-j$. Quicksort is comparison-based, so its run depends only on the order type of the input. If the input permutation is uniformly random then (i) the pivot $A[1]$ is the j -th smallest with probability $1/n$ for each $j \in [n]$, and (ii) conditional on this, the order types of the two sub-arrays are each uniformly random and independent (the partition only compares the pivot against each non-pivot, so it never compares two non-pivots with each other and cannot bias their relative order). Hence

$$A(n) = cn + \frac{1}{n} \sum_{j=1}^n [A(j-1) + A(n-j)] = cn + \frac{2}{n} \sum_{j=0}^{n-1} A(j),$$

with base case $A(0) = A(1) = 0$. To solve, multiply by n and subtract the same identity at $n-1$ to telescope:

$$nA(n) - (n-1)A(n-1) = c(2n-1) + 2A(n-1) \iff nA(n) - (n+1)A(n-1) = c(2n-1).$$

Dividing both sides by $n(n+1)$,

$$\frac{A(n)}{n+1} - \frac{A(n-1)}{n} = \frac{c(2n-1)}{n(n+1)} < \frac{2c}{n}.$$

Telescoping from n down to 1,

$$\frac{A(n)}{n+1} < \frac{A(1)}{2} + 2c \sum_{k=2}^n \frac{1}{k} = O(1) + 2c(H_n - 1) = O(\log n),$$

where we used the harmonic-sum envelope $H_n = \ln n + \Theta(1)$ from section 1. Multiplying by $n+1$ gives $A(n) \in O(n \log n)$, as claimed.

Remark 6.9 (Average case vs. randomised quicksort). The same bound $O(n \log n)$ holds for the *randomised* variant in which the pivot is chosen uniformly at random from the current sub-array on every recursive call – regardless of input order. The two analyses are essentially identical because the only place randomness enters is the per-call distribution of the pivot’s rank, and that distribution is $\text{Unif}([n])$ in both settings. The per-input “with high probability” guarantee is taken up in section 7. Two caveats apply when quoting the result. First, “average-case running time” is meaningless without specifying a distribution; the standard convention is uniform over the $n!$ orderings, and other conventions yield other answers. Second, the unqualified claim “quicksort is $O(n \log n)$ ” is wrong: on a sorted (or reverse-sorted) input with first-element pivoting, the worst case $\Theta(n^2)$ from the recurrence $T(n) = T(n-1) + cn$ is realised.

Remark 6.10 (In-place quicksort). The standard recursive quicksort can be made in-place by recursing on the smaller side first and handling the larger side via tail recursion; this caps the recursion depth at $O(\log n)$ in the worst case, giving $O(\log n)$ auxiliary space. The partition step itself, however, is not stable: equal-keyed elements can swap. If stability is required, either fall back to a stable algorithm (insertion or carefully implemented merge sort) or attach the original index as a tiebreaker.

Example 6.11 (Quicksort via indicator variables and linearity). An alternative proof of Theorem 6.8 that avoids recurrences entirely. Let $a_1 < a_2 < \dots < a_n$ be the sorted keys and let X be the total number of comparisons quicksort makes on a uniformly random input permutation. Two specific keys a_i and a_j ($i < j$) are compared by the algorithm *at most once*, because the only comparisons happen between a pivot and a non-pivot, after which the pivot leaves the recursion. Define the indicator

$$X_{ij} = \mathbb{1}_{\{a_i \text{ and } a_j \text{ are compared at some point}\}}, \quad i < j.$$

Then $X = \sum_{i < j} X_{ij}$ and, by linearity of expectation (Theorem 1.1),

$$\mathbb{E}[X] = \sum_{1 \leq i < j \leq n} \mathbb{P}(a_i \text{ and } a_j \text{ are compared}).$$

Computing the per-pair probability. Consider the first pivot among $\{a_i, a_{i+1}, \dots, a_j\}$ (in execution order). Until that moment, all $j-i+1$ of these keys have stayed together in the same recursive sub-array, since any earlier pivot is either smaller than a_i (sending them all right) or larger than a_j (sending them all left). When the first pivot from this set is finally chosen:

- if the pivot is a_i or a_j , then a_i and a_j are compared (pivot against non-pivot);
- if the pivot is one of the $j-i-1$ keys strictly between, then a_i goes left and a_j goes right and they are never compared again.

By symmetry, each of the $j - i + 1$ keys is equally likely to be that first pivot, so

$$\mathbb{P}(a_i, a_j \text{ compared}) = \frac{2}{j - i + 1}.$$

Summing. Substituting,

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} = \sum_{i=1}^{n-1} \sum_{k=2}^{n-i+1} \frac{2}{k} \leq 2 \sum_{i=1}^{n-1} H_n = 2(n-1)H_n \in O(n \log n),$$

using $H_n = \ln n + \Theta(1)$. Since the running time is $\Theta(\mathbb{E}[X]) + \Theta(n) = \Theta(\mathbb{E}[X])$ once partition overhead is accounted for, we recover Theorem 6.8. The crucial structural fact that makes the indicator argument work is that each *pair* is compared at most once, so X_{ij} is genuinely an indicator and not a count.

6.4 Linear-time non-comparison sorts

The $\Omega(n \log n)$ lower bound disappears as soon as the algorithm is allowed to look at key values directly. Both algorithms below assume keys come from a small integer range and use the keys themselves as array indices.

Theorem 6.12 (Counting sort). *Suppose the input array $A[1..n]$ has keys drawn from $\{1, 2, \dots, k\}$. Counting sort sorts A in $O(n + k)$ time and $O(n + k)$ space. Implemented carefully, it is stable.*

Proof. Counting sort works in three passes:

- (i) **Count:** traverse A once and compute $\text{count}[v] = \text{number of indices with } A[i] = v$, for each $v \in \{1, \dots, k\}$. This is $O(n + k)$ (initialise the array of size k , then n increments).
- (ii) **Prefix sum:** replace $\text{count}[v]$ by $\sum_{u \leq v} \text{count}[u]$, the number of keys $\leq v$. After this pass, $\text{count}[v]$ is the index of the *last* cell that should hold value v . This is $O(k)$.
- (iii) **Place:** traverse A from right to left; for each $A[i]$, write it to position $\text{count}[A[i]]$ of the output array B and decrement $\text{count}[A[i]]$. This is $O(n)$.

Total: $O(n + k)$. Stability is guaranteed by the right-to-left placement pass: among indices with the same key v , those processed later (smaller i) are written to smaller output positions, preserving original order.

Remark 6.13 (Input regime and placement orientation). The $O(n + k)$ bound is linear only when $k = O(n)$; for $k = n^{10}$ counting sort is $\Omega(n^{10})$, useless. The placement orientation also matters: the right-to-left pass with prefix sums interpreted as “last free index” is the formulation given above, while a left-to-right pass with the count array interpreted as “next free index” is also stable. Mixing the two – right-to-left with “next free” or vice versa – silently breaks stability, which then breaks radix sort downstream.

Example 6.14 (Counting sort on a small input). Let $A = (3, 1, 2, 1, 3, 2, 1)$ with $n = 7$ and alphabet $\{1, 2, 3\}$ ($k = 3$).

Count pass. $\text{count} = (3, 2, 2)$ – three 1s, two 2s, two 3s.

Prefix sum. $\text{count} = (3, 5, 7)$ – the last cell holding a 1 is index 3; the last cell holding a 2 is index 5; the last cell holding a 3 is index 7.

Place pass (right to left, output array $B[1..7]$).

i	$A[i]$	action	state of B
7	1	$B[3] \leftarrow 1, \text{count}[1] \leftarrow 2$	$(-, -, 1, -, -, -)$
6	2	$B[5] \leftarrow 2, \text{count}[2] \leftarrow 4$	$(-, -, 1, -, 2, -)$
5	3	$B[7] \leftarrow 3, \text{count}[3] \leftarrow 6$	$(-, -, 1, -, 2, -, 3)$
4	1	$B[2] \leftarrow 1, \text{count}[1] \leftarrow 1$	$(-, 1, 1, -, 2, -, 3)$
3	2	$B[4] \leftarrow 2, \text{count}[2] \leftarrow 3$	$(-, 1, 1, 2, 2, -, 3)$
2	1	$B[1] \leftarrow 1, \text{count}[1] \leftarrow 0$	$(1, 1, 1, 2, 2, -, 3)$
1	3	$B[6] \leftarrow 3, \text{count}[3] \leftarrow 5$	$(1, 1, 1, 2, 2, 3, 3)$

The final output is $B = (1, 1, 1, 2, 2, 3, 3)$. Note how the right-to-left order is what gives stability: the 1 at $A[7]$ ends up at $B[3]$, while the 1 at $A[2]$ ends up at $B[1]$, preserving the input order $A[2] \prec A[4] \prec A[7]$.

Theorem 6.15 (Radix sort). *Suppose each key in $A[1..n]$ has d digits drawn from an alphabet of size k . Radix sort sorts A in $O(d(n+k))$ time.*

Proof. (*sketch.*) Run counting sort d times – once per digit – starting from the *least significant digit* and proceeding to the most significant. Each pass is $O(n+k)$ by Theorem 6.12, giving total $O(d(n+k))$. Correctness rests on the inductive invariant: after passing on digits $1, \dots, t$, the array is sorted by the lexicographic order of those t low-order digits. The induction step uses *stability* crucially: when sorting on digit $t+1$, two keys with the same digit- $(t+1)$ value retain their relative order from the previous pass, so they remain sorted on digits $1, \dots, t$. Without stable counting sort, radix sort is wrong.

Remark 6.16 (When is linear-time sorting actually linear?). For radix sort applied to n machine-word integers in $\{0, \dots, n^c - 1\}$, choosing alphabet $k = n$ gives $d = c$ digits and total $O(c \cdot 2n) = O(n)$. For arbitrary integers in a range up to U , $d = \log_k U$ and the running time is $O((n+k) \log_k U)$, optimised at $k = n$ to $O(n \log_n U)$. The catch is the k term: counting sort with alphabet $k = n^{10}$ is $\Omega(n^{10})$, useless. Bucket sort is a third member of this family: it scatters keys into buckets, sorts each bucket, and concatenates; under the assumption that keys are i.i.d. uniform on an interval, it runs in $O(n)$ expected time, but the assumption is essential – on adversarial input it degrades to the time of the bucket sub-sort.

6.5 Linear-time selection

Selection asks: given $A[1..n]$ and $k \in [n]$, find the k -th smallest element. Sorting and indexing takes $O(n \log n)$; the median-of-medians algorithm of Blum–Floyd–Pratt–Rivest–Tarjan does it in $\Theta(n)$ in the worst case.

Theorem 6.17 (Median-of-medians selection). *There is a deterministic, comparison-based algorithm $\text{SELECT}(A, k)$ that returns the k -th smallest element of $A[1..n]$ in $O(n)$ time in the worst case.*

Proof. (*sketch.*) The algorithm works in five phases.

- (i) Divide A into $\lceil n/5 \rceil$ groups of 5 (the last possibly smaller). Sort each group in $O(1)$ time, and let m_g be its median.
- (ii) Recursively call SELECT on the array $(m_1, m_2, \dots)$ of medians to find their median p . This takes $T(\lceil n/5 \rceil)$ time.

- (iii) Partition A around p into $A_{<}, \{p\}, A_{>}$ in $O(n)$ time.
- (iv) If $|A_{<}| = k - 1$, return p . Otherwise recurse into $A_{<}$ or $A_{>}$ with an updated rank.

Why $O(n)$. The pivot p is the median of the $\lceil n/5 \rceil$ group-medians, so at least half of those medians are $\geq p$, and each such group contributes at least 3 elements $\geq p$ (the median and the two larger). Hence at least $3\lceil n/10 \rceil \geq 3n/10$ elements of A are $\geq p$, and symmetrically at least $3n/10$ are $\leq p$. Therefore each of $|A_{<}|, |A_{>}|$ is at most $7n/10$, giving the recurrence

$$T(n) \leq T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\left\lfloor \frac{7n}{10} \right\rfloor\right) + O(n).$$

The two fractions sum to $9/10 < 1$, which is the structural reason the recurrence solves to $T(n) = O(n)$. To verify, guess $T(n) \leq cn$ and substitute: $T(n) \leq c(n/5) + c(7n/10) + an = (9c/10)n + an = cn - (cn/10 - an)$, which is $\leq cn$ for $c \geq 10a$. The base case is $O(1)$ for $n \leq 5$. The constant 5 is the smallest odd group size that makes the two fractions sum to less than one; group size 3 would give $T(n/3) + T(2n/3) + O(n) = O(n \log n)$.

Remark 6.18 (Randomised quickselect). Replacing the deterministic median-of-medians pivot with a uniformly random pivot gives a much simpler algorithm with expected running time $O(n)$ but $\Theta(n^2)$ worst case. The expected analysis mirrors Example 6.11 (each pair is compared at most once, sum of $2/(j - i + 1)$ over relevant pairs).

7 Randomised Algorithms

A *randomised* algorithm reads a stream of unbiased random bits in addition to its input, so its running time and correctness are random variables. The pay-off is simple, fast algorithms whose deterministic counterparts are slower (Freivalds' $O(n^2)$ matrix-product check vs. $\Theta(n^{2.37\dots})$ for re-multiplying) or substantially more intricate; the price is a controlled error probability, driven down by independent repetition. The probabilistic guarantees here hold *for every input*, with the expectation taken over the algorithm's coin flips — distinct from the average-case analysis of section 2 where the input itself is random.

7.1 Definitions

Definition 7.1 (Randomised algorithm). A *randomised algorithm* $\mathcal{A}$ is one that, in addition to its input x , reads a sequence of independent fair random bits r . Its output $\mathcal{A}(x; r)$ and running time $T_{\mathcal{A}}(x; r)$ are therefore functions of both x and r , and we treat them as random variables over the choice of r (with x fixed). Probabilistic statements about $\mathcal{A}$, such as “ $\mathbb{P}_r[\mathcal{A}(x; r) \text{ is correct}] \geq 1/2$,” are required to hold *for every* x ; this is what distinguishes randomised analysis from average-case analysis, where the input x is itself drawn from a distribution.

Definition 7.2 (Las Vegas algorithm). A *Las Vegas* algorithm always outputs the correct answer; its running time is a random variable, and we measure efficiency by the *expected* running time. The canonical example is randomised quicksort (Proposition 7.14): the output is the correctly sorted array on every run, but the running time depends on the random pivot choices and is $\Theta(n^2)$ in the worst case while $O(n \log n)$ in expectation.

Definition 7.3 (Monte Carlo algorithm). A *Monte Carlo* algorithm runs in a fixed (or worst-case bounded) time but may produce an incorrect answer. We measure efficiency by the worst-case

running time and quality by an *error probability* $\varepsilon < 1/2$, with the guarantee

$$\forall x, \quad \mathbb{P}_r[\mathcal{A}(x; r) \neq f(x)] \leq \varepsilon,$$

where f is the function being computed. Freivalds' algorithm (Theorem 7.8) is the canonical example.

Definition 7.4 (One-sided error). A Monte Carlo algorithm for a decision problem has *one-sided error* if it errs on only one of the two answers. Concretely, with $\varepsilon < 1$ and the convention that the algorithm may err only on YES-instances:

$$f(x) = \text{NO} \implies \mathbb{P}[\mathcal{A}(x) = \text{NO}] = 1, \quad f(x) = \text{YES} \implies \mathbb{P}[\mathcal{A}(x) = \text{YES}] \geq 1 - \varepsilon.$$

Freivalds' algorithm has one-sided error of this form, with the roles of YES/NO swapped: when $AB = C$ the algorithm is always right; only when $AB \neq C$ can it err.

Definition 7.5 (Two-sided error). A Monte Carlo algorithm has *two-sided error* if it can err on either answer, but with total error bounded:

$$\forall x, \quad \mathbb{P}[\mathcal{A}(x) \neq f(x)] \leq \varepsilon,$$

for some $\varepsilon < 1/2$. The error gap $1/2 - \varepsilon$ is what makes amplification by majority vote work; without it, repeated runs cannot improve confidence.

Definition 7.6 (Expected running time). The *expected running time* of a randomised algorithm $\mathcal{A}$ on input x is

$$\mathbb{E}[T_{\mathcal{A}}(x)] = \sum_r T_{\mathcal{A}}(x; r) \cdot \mathbb{P}[r],$$

i.e. the expectation of the running-time random variable over the algorithm's internal randomness. The *worst-case expected running time* is $\max_x \mathbb{E}[T_{\mathcal{A}}(x)]$, and this is the quantity bounded by, e.g. “randomised quicksort runs in expected $O(n \log n)$ time”: the bound is over the random pivot choices, uniformly over all inputs of size n .

Remark 7.7 (Las Vegas $\leftrightarrow$ Monte Carlo conversion). A Las Vegas algorithm with expected running time T can be converted to a Monte Carlo algorithm by truncation: run for $100T$ steps and output a default answer if it has not finished. By Markov's inequality (Theorem 1.2), the truncation aborts with probability at most $1/100$, giving a Monte Carlo algorithm with error $\leq 1/100$ and worst-case time $100T$. The reverse conversion (Monte Carlo $\rightarrow$ Las Vegas) requires an efficient way to *check* an answer's correctness: keep rerunning the Monte Carlo algorithm until the check succeeds. The Markov argument is also the standard route from an expected $O(n \log n)$ bound to a “runs in $O(n \log n)$ with probability ≥ 0.99 ” statement: $\mathbb{E}[T] \leq t$ gives $\mathbb{P}[T \geq 100t] \leq 1/100$.

7.2 Freivalds' algorithm

The matrix-product verification problem: given three $n \times n$ matrices A, B, C , decide whether $AB = C$. Computing AB deterministically and comparing entrywise costs $\Theta(n^{2.37\dots})$ via the best-known matrix-multiplication algorithms (or $O(n^{2.807\dots})$ via Strassen, $\Theta(n^3)$ for the schoolbook algorithm); Freivalds' algorithm settles the verification (not multiplication) question in $O(n^2)$ time with bounded error.

Algorithm (Freivalds). On input A, B, C :

- (1) Sample $v \in \{0, 1\}^n$ uniformly: each coordinate v_i is independently 0 or 1 with probability $1/2$.
- (2) Compute $w = A(Bv) - Cv$ via three matrix–vector products in $O(n^2)$ total time.
- (3) If $w = 0$, output “ $AB = C$ ”; else output “ $AB \neq C$ ”.

Theorem 7.8 (Correctness of Freivalds’ algorithm). *For every $A, B, C \in \mathbb{R}^{n \times n}$, one round of Freivalds’ algorithm satisfies:*

- if $AB = C$, the algorithm outputs “ $AB = C$ ” with probability 1 (no error);
- if $AB \neq C$, the algorithm outputs the wrong answer “ $AB = C$ ” with probability at most $1/2$.

The algorithm runs in $O(n^2)$ time, has one-sided error (Definition 7.4), and is a Monte Carlo algorithm (Definition 7.3).

Proof. If $AB = C$, then $ABv = Cv$ for every vector v , so $w = 0$ deterministically and the algorithm answers correctly with probability 1.

Now suppose $AB \neq C$, and write $D = AB - C$. By assumption D has at least one non-zero entry; fix indices (i^*, j^*) with $D_{i^*, j^*} \neq 0$. The algorithm errs iff $Dv = 0$, which forces in particular the i^* -th coordinate to vanish:

$$(Dv)_{i^*} = \sum_{k=1}^n D_{i^*, k} v_k = 0.$$

Isolate the j^* -term and condition on the other coordinates: for any fixing of $\{v_k : k \neq j^*\}$, write $S = \sum_{k \neq j^*} D_{i^*, k} v_k$ (a fixed real number once we condition). The equation $(Dv)_{i^*} = 0$ then becomes

$$D_{i^*, j^*} v_{j^*} = -S.$$

Since $D_{i^*, j^*} \neq 0$, this equation has at most one solution $v_{j^*} \in \{0, 1\}$ (in fact exactly one if $-S/D_{i^*, j^*} \in \{0, 1\}$, and zero otherwise). As v_{j^*} is independent of $\{v_k : k \neq j^*\}$ and uniform over $\{0, 1\}$, conditional on the other coordinates we have

$$\mathbb{P}[(Dv)_{i^*} = 0 \mid \{v_k\}_{k \neq j^*}] \leq \frac{1}{2}.$$

This bound holds for *every* fixing of the other coordinates, so by the principle of deferred decisions (Remark 7.9) the unconditional probability also satisfies $\mathbb{P}[(Dv)_{i^*} = 0] \leq 1/2$. Since $\{Dv = 0\} \subseteq \{(Dv)_{i^*} = 0\}$, the error probability $\mathbb{P}[Dv = 0]$ is also at most $1/2$.

The running time is the cost of three matrix–vector products, $O(n^2)$.

Remark 7.9 (Principle of deferred decisions). The step “ $\mathbb{P}[\mathcal{E} \mid X = x] \leq p$ for every x implies $\mathbb{P}[\mathcal{E}] \leq p$ ” is the *principle of deferred decisions*: instead of analysing all the random choices simultaneously, freeze all but one and analyse the remaining one conditionally; if the conditional bound is uniform, it lifts to an unconditional bound. Formally,

$$\mathbb{P}[\mathcal{E}] = \sum_x \mathbb{P}[\mathcal{E} \mid X = x] \mathbb{P}[X = x] \leq p \sum_x \mathbb{P}[X = x] = p.$$

The technique is used pervasively in the analysis of randomised algorithms whenever “the worst random coordinate is the one revealed last”.

Example 7.10 (Freivalds on a 2×2 counter-example). Let

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad C = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix},$$

so that $AB = I_2 \neq C$ and $D = AB - C = \begin{pmatrix} 0 & -1 \\ 0 & 0 \end{pmatrix}$. The four possible vectors $v \in \{0, 1\}^2$ each have probability $1/4$:

v	Dv	algorithm output
$(0, 0)$	$(0, 0)$	“ $AB = C$ ” (wrong)
$(0, 1)$	$(-1, 0)$	“ $AB \neq C$ ” (correct)
$(1, 0)$	$(0, 0)$	“ $AB = C$ ” (wrong)
$(1, 1)$	$(-1, 0)$	“ $AB \neq C$ ” (correct)

The error probability on this instance is exactly $2/4 = 1/2$, attaining the upper bound of Theorem 7.8. The bad coordinate is $j^* = 2$: among the two choices $v_2 \in \{0, 1\}$, exactly one of them ($v_2 = 0$) makes $-v_2 = 0$. This matches the proof: “at most one of the two choices of v_{j^*} kills the i^* -th row”, and the value of v_1 is irrelevant because column 1 of D is zero.

7.3 Amplification by repetition

Theorem 7.11 (Error amplification). *Let $\mathcal{A}$ be a Monte Carlo algorithm with one-sided error $\leq 1/2$ for a decision problem (as in Freivalds, where the algorithm errs only when the true answer is NO, and even then with probability $\leq 1/2$). Run $\mathcal{A}$ independently k times on the same input and output YES iff every run output YES. The resulting algorithm has error probability at most 2^{-k} . In particular, to drive the error below $f \in (0, 1)$ it suffices to take*

$$k = \left\lceil \log_2 \frac{1}{f} \right\rceil$$

independent rounds, multiplying the running time by a $\log(1/f)$ factor.

Proof. On a YES-instance every run outputs YES with probability 1, so the combined algorithm is correct with probability 1.

On a NO-instance, let $\mathcal{E}_i$ be the event that the i -th run errs (outputs YES). The combined algorithm errs iff $\bigcap_{i=1}^k \mathcal{E}_i$; by independence of the runs,

$$\mathbb{P}\left[\bigcap_{i=1}^k \mathcal{E}_i\right] = \prod_{i=1}^k \mathbb{P}[\mathcal{E}_i] \leq \prod_{i=1}^k \frac{1}{2} = 2^{-k}.$$

Setting $2^{-k} \leq f$ rearranges to $k \geq \log_2(1/f)$, giving the round count above.

For two-sided error, replace “every run YES” by majority vote and bound the probability that more than half the runs err using a Chernoff bound; the cruder union bound (Theorem 1.6), $\mathbb{P}[\bigcup_i \mathcal{E}_i] \leq \sum_i \mathbb{P}[\mathcal{E}_i]$, also gives a useful but weaker $k \cdot \varepsilon$ statement.

Remark 7.12 (Independence and the one-sided-error structure). The bound 2^{-k} uses $\mathbb{P}[\bigcap_i \mathcal{E}_i] = \prod_i \mathbb{P}[\mathcal{E}_i]$, which requires the rounds to be independent: reusing the same random vector v across all rounds of Freivalds leaves the error at $1/2$ forever, so each round must draw fresh random bits. The

AND-rule (output YES iff every run says YES) further relies on the one-sided structure of the error: a Freivalds-style algorithm errs only on NO-instances (where $AB \neq C$), so a single run answering NO is conclusive and a single YES carries the round's $\leq 1/2$ uncertainty. If errors could land on either side independently, the AND-rule would be invalid and majority vote (with a Chernoff or union-bound analysis) would be required. The union bound (Theorem 1.6) is loose but free, and is the workhorse whenever the events of interest are not independent.

Example 7.13 (Concrete amplification numbers). With Freivalds' single-round error of $1/2$ and k independent rounds, the error probability is at most 2^{-k} :

k	error bound 2^{-k}	total time
1	0.5	$O(n^2)$
10	$\leq 9.8 \times 10^{-4}$	$O(n^2)$
20	$\leq 9.5 \times 10^{-7}$	$O(n^2)$
50	$\leq 8.9 \times 10^{-16}$	$O(n^2)$

At $k = 20$ rounds the error is below 10^{-6} , far smaller than the probability of a hardware bit-flip during execution; at $k = 50$ the error is below the standard double-precision floating-point epsilon. The total time stays $O(n^2)$ because k is a constant, comfortably beating $\Theta(n^{2.37\dots})$ for re-multiplication. Conversely, to achieve a target error $f = 10^{-9}$, take $k = \lceil \log_2(10^9) \rceil = 30$ rounds.

7.4 Randomised quicksort

Proposition 7.14 (Randomised quicksort). *The Las Vegas algorithm Randomised Quicksort (partition around a pivot chosen uniformly at random from the current sub-array, recurse on the two sides) sorts n distinct numbers in expected time $O(n \log n)$, uniformly over all input arrays. The full analysis – via indicator variables $X_{i,j} = \mathbb{1}[a_i \text{ is compared with } a_j]$ with $\mathbb{P}[X_{i,j} = 1] = 2/(j - i + 1)$, summing through linearity of expectation (Theorem 1.1) and a harmonic-sum bound – is given in Theorem 6.8.*

Remark 7.15 (Randomised vs. average-case). The expected running-time bound for Randomised Quicksort holds for *every* input array; the expectation is over the algorithm's coin flips. By contrast, the classical “average-case $O(n \log n)$ for deterministic Quicksort with first-element pivot” bound holds when the input is uniformly random; on a sorted input that algorithm is $\Theta(n^2)$. The randomisation moves the worst-case bad input from the input distribution to the algorithm's randomness, which is under our control: an adversary who picks the input cannot also pick the pivots.

8 Dynamic Programming

Dynamic programming (DP) refines divide-and-conquer (section 5) for problems where the naive recursion recomputes the same subinstance exponentially many times. The fix is to enumerate the polynomial-size set of distinct subinstances exactly once, in an order that respects the recursive dependency, and store each answer in a table for $O(1)$ lookup. The two structural properties to check are *optimal substructure* (the recurrence) and *overlapping subproblems* (the polynomial state space); given those, the implementation is either top-down memoisation or bottom-up tabulation, with the same asymptotic cost.

8.1 Definitions

Definition 8.1 (Subproblem). A *subproblem* of a problem instance P is an instance P' of the same problem on a strictly smaller (or otherwise simpler, in the well-founded order on states) input. For DP one fixes a parameterisation of inputs by a tuple of integers (the *state*, see Definition 8.7); each tuple in the chosen range is a subproblem, and the original problem is the subproblem at the largest state.

Definition 8.2 (Optimal substructure). A problem has *optimal substructure* if an optimal solution to any instance can be expressed as a function (typically a min, max, or sum) of optimal solutions to a fixed, finite collection of strictly smaller subproblems. Equivalently, removing the “last decision” from an optimal solution leaves an optimal solution to the residual subproblem; this is the property one verifies by a *cut-and-paste* argument: assume some sub-solution were sub-optimal, paste in a better one, and contradict optimality of the parent (cf. the LCS proof in Example 8.16 and the knapsack proof in Example 8.17).

Remark 8.3 (Optimal substructure does not imply a greedy algorithm). Both DP and greedy require optimal substructure, so observing it does not justify a greedy algorithm. Greedy additionally requires the *greedy-choice property* – that some specific local choice extends to a global optimum. Coin change (Example 8.18) has optimal substructure but not the greedy-choice property under arbitrary denominations, so DP works and greedy does not; see section 9.

Definition 8.4 (Overlapping subproblems). A recursive solution exhibits *overlapping subproblems* if the (unmemoised) recursion tree contains the same subproblem at exponentially many internal nodes, while the total number of *distinct* subproblems is only polynomial in the input size. This is the property that makes DP profitable: without overlap, memoisation buys nothing and the problem is naturally solved by D&C (section 5); with overlap, memoisation collapses an exponential recursion tree to a polynomial DAG.

Definition 8.5 (Memoisation, top-down DP). *Memoisation* is the implementation of a recursive solution in which the result of every call $\text{SOLVE}(s)$ is cached in a table $T[s]$ on first computation, and returned from the cache on subsequent calls. The control flow is *top-down*: one invokes SOLVE at the original instance, and the cache is filled lazily by the recursion in whatever order it descends. Each distinct state s is solved exactly once.

Definition 8.6 (Tabulation, bottom-up DP). *Tabulation* is the implementation in which the table $T[s]$ is filled iteratively by a (nested) loop that visits every state s in an order consistent with the dependency DAG of the recurrence (every predecessor of s is computed before s). The control flow is *bottom-up*: base cases first, then a topological sweep. No recursion stack is used.

Definition 8.7 (State space). The *state space* of a DP formulation is the set of tuples $s = (s_1, \dots, s_d)$ that index the subproblems, together with their ranges. The state space size $|S|$ is the product of the per-coordinate ranges; for an array DP indexed by $i \in \{0, \dots, n\}$ it is $|S| = n + 1$, for a 2-D table $(i, j) \in \{0, \dots, n\} \times \{0, \dots, m\}$ it is $|S| = (n + 1)(m + 1)$, and so on. Choosing the state space is the central design decision: too coarse and the recurrence cannot close, too fine and the running time blows up.

Remark 8.8 (Diagnosing a too-coarse state). If the recurrence cannot be expressed as a function of $T[s']$ for $s' \prec s$, the state is missing a coordinate. Knapsack with just $T[i]$ (forgetting the capacity) does not close; longest increasing subsequence with $T[i] = \text{“length of LIS of } A[1..i]\text{”}$ (without “ending at i ”) does not close because the answer for $i + 1$ depends on whether the LIS ending at $i + 1$ extends an earlier one. The remedy is always to add the coordinate the recurrence wants to read.

Definition 8.9 (Transition). A *transition* is the recurrence relation expressing $T[s]$ as a function of one or more $T[s']$ with s' a strictly smaller state. The cost of a single transition is the time to evaluate the right-hand side once the predecessor entries are available; this is typically $O(1)$ (constant number of array lookups, e.g. LCS, knapsack) but can be $O(n)$ (a min/max over n predecessors, e.g. matrix-chain multiplication, coin change with k denominations) or larger.

8.2 Recipe and analysis

A complete DP solution comprises the following components:

- (i) **Define the state.** Give the high-level meaning of $T[s]$ as a precise English sentence stating what the entry stores (e.g. “ $T[i, j]$ is the length of the LCS of $A[1..i]$ and $B[1..j]$ ”).
- (ii) **Write the recurrence.** State the transition $T[s]$ in terms of $T[s']$ for predecessor states $s' \prec s$, together with the bounds under which each branch applies. Justify it by an optimal-substructure (Definition 8.2) argument.
- (iii) **Base cases.** Specify $T[s]$ for the smallest states (typically $s = 0$ or one coordinate = 0). The recurrence is meaningless without them; the most common error in 2-D DPs is omitting the boundary row or column.
- (iv) **Order of computation.** Choose a topological order on the dependency DAG (fig. 4); in 1-D this is just “increasing i ”, in 2-D it is row-major or column-major depending on how the recurrence reads. Top-down memoisation is forgiving because the recursion discovers the order; bottom-up tabulation requires nominating a topological order explicitly, otherwise the loop reads uninitialised cells.
- (v) **Time and space analysis.** Use Proposition 8.11.

A useful diagnostic is the distinction between a *recurrence relation*, which states $T[s]$ in terms of predecessor entries, and a *closed-form formula*, which evaluates $T[s]$ directly. DP requires only the former; conflating the two is a frequent source of incorrect bottom-up code.

Theorem 8.10 (Correctness template for DP). *Let $\mathcal{A}$ be a DP algorithm with a finite state space S equipped with a strict well-founded order $\prec$, base values $T[s_0]$ correct by direct inspection for the $\prec$ -minimal states s_0 , and a transition $T[s] = \Phi(T[s'] : s' \prec s)$ that is correct in the sense “if every $T[s']$ with $s' \prec s$ holds the true optimum at s' , then Φ produces the true optimum at s .” Then $T[s]$ holds the true optimum at every $s \in S$ on termination.*

Proof. (*sketch.*) Strong induction on the well-founded order $\prec$ on S . For each $\prec$ -minimal s_0 , $T[s_0]$ is correct by hypothesis on the base cases. For s with $\prec$ -predecessors only, assume (strong inductive hypothesis) that $T[s']$ is correct for every $s' \prec s$; by hypothesis on the transition, Φ produces the true optimum at s , so $T[s]$ is correct after it is filled. Since $\prec$ is well-founded and S is finite, induction terminates and every state is correct. The argument is structurally identical to the strong-induction proof for D&C correctness (Theorem 5.4); the only change is that the inductive hypothesis ranges over the predecessor states of the DP recurrence rather than over input sizes. The optimal-substructure clause (Definition 8.2) is exactly what makes the transition-correctness hypothesis verifiable by a cut-and-paste argument.

Proposition 8.11 (Time complexity of a DP algorithm). *Let $\mathcal{A}$ be a DP algorithm with state space S and per-state transition cost $\tau(s)$ (the time to evaluate Φ at s once predecessor entries are available). Then the running time of bottom-up tabulation is*

$$T(n) = \Theta\left(\sum_{s \in S} \tau(s)\right) = \Theta(|S| \cdot \tau_{\max})$$

when τ is uniformly $O(\tau_{\max})$. Top-down memoisation has the same worst-case bound; it can be strictly faster on instances where the recursion never reaches some states.

Proof. (sketch.) Bottom-up: the outer loop visits each $s \in S$ exactly once, and the body costs $\Theta(\tau(s))$ (table writes are $\Theta(1)$ in the RAM model). Summing gives $\sum_s \tau(s)$. Top-down: each state is computed at most once thanks to the cache; cache lookups and the bookkeeping per call are $\Theta(1)$, and aggregated over all calls (including the cache-hit calls that no longer recurse) the total is again $\Theta(\sum_s \tau(s))$ in the worst case. The $\tau_{\max}$ form is just $|S| \cdot \tau_{\max}$ as the trivial upper bound; tight analyses substitute the true $\tau(s)$ summation. This is the DP analogue of Proposition 5.6: instead of a recursion tree resolved by the master theorem (Theorem 3.6), the recursion DAG is summed directly.

Remark 8.12 (Counting the per-state cost). The bound is $|S| \cdot \tau_{\max}$, not $|S|$ alone: a recurrence whose right-hand side is itself a min or max over k predecessors contributes $\tau = \Theta(k)$, not $\Theta(1)$. Coin change (Example 8.18) is $\Theta(nk)$, not $\Theta(n)$, for exactly this reason.

Remark 8.13 (Subset state spaces are exponential). Some problems require a state space indexed by subsets $S \subseteq \{1, \dots, n\}$ – the canonical example is the Held–Karp DP for the travelling salesman problem – and so live in $\Theta(2^n \cdot n^c)$ time and $\Theta(2^n)$ space, not in $n^{O(1)}$. The state-space size during analysis flags this immediately; reporting such an algorithm as polynomial is a category error.

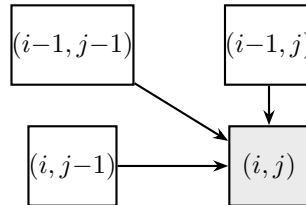


Figure 4: Dependency DAG for the LCS DP table: cell (i, j) depends on $(i-1, j-1)$, $(i-1, j)$, and $(i, j-1)$. Any topological order of this DAG is a valid order of computation; the standard row-major sweep (i outer, j inner) and column-major sweep both work.

Remark 8.14 (DP versus D&C). The decisive distinction with D&C (section 5) is whether the recursive subinstances overlap. For mergesort-style D&C, the two halves of the input are disjoint and each is solved independently; the recursion tree has no shared nodes and the master theorem (Theorem 3.6) bounds the running time directly. For DP, the recursion tree of the naive recursive formulation has shared nodes – exponentially many – and the table is precisely the device that collapses those duplicates into a polynomial-size DAG. A useful diagnostic: if the recursion tree of the recursive formulation has $\leq |S|$ distinct nodes and the rest are duplicates, DP gives a polynomial algorithm; if every node is distinct, only D&C is available.

8.3 Worked examples

Example 8.15 (Fibonacci numbers $F(n) \bmod m$). **Problem.** Given n, m , compute $F(n) \bmod m$ where $F(0) = 0$, $F(1) = 1$, $F(k) = F(k-1) + F(k-2)$ for $k \geq 2$.

State space. $T[i] \in \{0, 1, \dots, m-1\}$ for $i \in \{0, 1, \dots, n\}$, where $T[i] = F(i) \bmod m$. Size $|S| = n+1$.

Recurrence and base cases.

$$T[i] = \begin{cases} 0 & i = 0, \\ 1 & i = 1, \\ (T[i-1] + T[i-2]) \bmod m & i \geq 2. \end{cases}$$

Order. Increasing i .

Time. $|S| = n+1$, $\tau = O(1)$, so $\Theta(n)$ by Proposition 8.11. Compare to the naive recursive algorithm, whose running time satisfies $T_{\text{rec}}(n) = T_{\text{rec}}(n-1) + T_{\text{rec}}(n-2) + \Theta(1)$ and so $T_{\text{rec}}(n) \in \Theta(\phi^n)$ with $\phi = (1 + \sqrt{5})/2$.

Correctness. Optimal substructure is the defining recurrence itself, applied to the residue ring $\mathbb{Z}/m\mathbb{Z}$. Strong induction on i via Theorem 8.10.

Example 8.16 (Longest common subsequence (LCS)). **Problem.** Given sequences $A[1..n]$ and $B[1..m]$, find the length of the longest sequence C that is a subsequence of both. (C is a *subsequence* of A if C is obtained from A by deleting zero or more elements while preserving order.)

State space. $L[i, j] \in \{0, 1, \dots, \min(i, j)\}$ for $(i, j) \in \{0, \dots, n\} \times \{0, \dots, m\}$, where $L[i, j]$ is the length of an LCS of $A[1..i]$ and $B[1..j]$. Size $|S| = (n+1)(m+1)$.

Recurrence and base cases.

$$L[i, j] = \begin{cases} 0 & i = 0 \text{ or } j = 0, \\ L[i-1, j-1] + 1 & i, j \geq 1 \text{ and } A[i] = B[j], \\ \max(L[i-1, j], L[i, j-1]) & i, j \geq 1 \text{ and } A[i] \neq B[j]. \end{cases}$$

Order. Row-major: for $i = 1, \dots, n$, for $j = 1, \dots, m$, fill $L[i, j]$. Each entry depends only on $(i-1, j-1)$, $(i-1, j)$, $(i, j-1)$ (fig. 4), all already computed.

Time. $|S| = (n+1)(m+1)$, $\tau = O(1)$, so $\Theta(nm)$ by Proposition 8.11. Space $\Theta(nm)$, reducible to $\Theta(\min(n, m))$ by keeping only the previous row.

Correctness sketch. Cut-and-paste. If $A[i] = B[j]$, any LCS of $A[1..i]$, $B[1..j]$ may end with the matched pair (else we could append $A[i]$ and grow the LCS by one), so $L[i, j] = L[i-1, j-1] + 1$. If $A[i] \neq B[j]$, an LCS cannot use both as its last symbol; whichever last symbol it does use is absent from the corresponding prefix, giving $L[i, j] = \max(L[i-1, j], L[i, j-1])$. Strong induction over $\prec = "(i', j') \prec (i, j) \iff i' < i \vee (i' = i \wedge j' < j)"$ via Theorem 8.10.

Example 8.17 (0/1 knapsack). **Problem.** Given n items with positive integer weights $w_1, \dots, w_n$ and values $v_1, \dots, v_n$, and a capacity W , find a subset $S \subseteq \{1, \dots, n\}$ maximising $\sum_{i \in S} v_i$ subject to $\sum_{i \in S} w_i \leq W$.

State space. $m[i, j]$ for $(i, j) \in \{0, \dots, n\} \times \{0, \dots, W\}$, where $m[i, j]$ is the maximum total value achievable using a subset of items $\{1, \dots, i\}$ with total weight at most j . Size $|S| = (n+1)(W+1)$.

Recurrence and base cases.

$$m[i, j] = \begin{cases} 0 & i = 0 \text{ or } j = 0, \\ \max(m[i-1, j], m[i-1, j-w_i] + v_i) & i \geq 1 \text{ and } w_i \leq j, \\ m[i-1, j] & i \geq 1 \text{ and } w_i > j. \end{cases}$$

The two arguments of the max are “don’t take item i ” and “take item i ”.

Order. Increasing i outer, increasing j inner.

Time. $|S| = (n+1)(W+1)$, $\tau = O(1)$, so $\Theta(nW)$ by Proposition 8.11. This is *pseudo-polynomial*: it is polynomial in the numerical value W , which is exponential in the bit-length $\log W$ of the input. Knapsack is in fact NP-hard (section 12), and no truly polynomial algorithm is known.

Correctness sketch. Optimal substructure by cut-and-paste: in the optimum at (i, j) , either item i is excluded (then the chosen subset of $\{1, \dots, i-1\}$ must itself be optimal at $(i-1, j)$, else paste in a better subset and improve the parent), or item i is included (then the residual choice from $\{1, \dots, i-1\}$ uses weight $\leq j - w_i$ and must be optimal at $(i-1, j - w_i)$). The two alternatives give the two arguments of the max. Apply Theorem 8.10.

Example 8.18 (Coin change: minimum number of coins). Problem. Given denominations $d_1, \dots, d_k$ (positive integers) and a target amount n , find the minimum number of coins (with repetition allowed) summing to exactly n , or report infeasibility.

State space. $M[j]$ for $j \in \{0, 1, \dots, n\}$, where $M[j]$ is the minimum number of coins summing to exactly j (with $M[j] = \infty$ if no such combination exists). Size $|S| = n+1$.

Recurrence and base cases.

$$M[j] = \begin{cases} 0 & j = 0, \\ 1 + \min\{M[j - d_i] : i \in \{1, \dots, k\}, d_i \leq j\} & j \geq 1. \end{cases}$$

If no $d_i \leq j$ (or all such recurrences yield ∞), $M[j] = \infty$.

Order. Increasing j .

Time. $|S| = n+1$, $\tau(j) = O(k)$ (the inner min ranges over k denominations), so $\Theta(nk)$ by Proposition 8.11.

Correctness sketch. Optimal substructure: if $M[j] = t$ is realised by coins $d_{i_1}, \dots, d_{i_t}$ summing to j , then $M[j - d_{i_t}] = t - 1$ – otherwise a strictly better combination for $j - d_{i_t}$ would, by appending d_{i_t} , beat $M[j]$. Cut-and-paste, then Theorem 8.10. Note: the natural greedy (“always take the largest coin $\leq j$ ”) is *not* correct in general. Denominations $\{1, 10, 25\}$ with $n = 30$ illustrate this: greedy takes $25 + 1 + 1 + 1 + 1 + 1 = 6$ coins, but DP finds $10 + 10 + 10 = 3$ coins. Greedy works only for “canonical” coin systems; see section 9 for the structural condition (matroid / exchange) it requires.

9 Greedy Algorithms

A *greedy* algorithm builds an optimal solution by committing, at each step, to a single locally optimal choice and then recursing on the one subproblem that remains. Compared with dynamic programming (section 8), which explores every relevant subproblem and combines their solutions, greedy collapses the recursion tree into a single path: at each node only one child is visited. The payoff is dramatic running times, typically $O(n \log n)$, but not every problem with optimal substructure

admits a greedy choice – the classical sanity check is fractional knapsack (greedy works) versus 0/1 knapsack (only DP, in pseudo-polynomial time). Correctness proofs come in two flavours: a *cut-and-paste* argument for optimal substructure, and an *exchange* (or “stays-ahead”) argument for the greedy choice.

9.1 Definitions

Definition 9.1 (Greedy algorithm). A *greedy algorithm* for an optimisation problem is an algorithm that, at each step, makes the choice that looks best at the moment – a locally optimal choice – and then recursively solves the single subproblem that remains, without ever revisiting earlier decisions. The paradigm has three ingredients:

- (i) **Cast** the problem so that one decision reduces it to one subproblem of the same form.
- (ii) **Prove** the greedy-choice property (Definition 9.2): some optimal solution makes the locally optimal choice, so committing to it is safe.
- (iii) **Combine** the greedy choice with an optimal solution to the subproblem (using optimal substructure, Definition 9.3) to obtain an optimal solution to the original problem.

The hallmark is that a *locally* optimal choice is *globally* optimal – a property that must be *proved*, not assumed. A useful sanity check before trusting a candidate greedy: test it on a small adversarial instance; 0/1 knapsack famously *has* optimal substructure (DP works) yet *lacks* a usable greedy-choice property, so optimal substructure alone never suffices.

Definition 9.2 (Greedy-choice property). A problem has the *greedy-choice property* (with respect to a candidate greedy choice g) if there exists an optimal solution to the original problem that makes the choice g . Equivalently: any optimal solution can be *exchanged* for one that contains g without increasing the cost (resp. decreasing the value). This is what makes the local commitment safe: even though we cannot enumerate all optima, we know that at least one of them agrees with the greedy choice, so reducing to the subproblem after committing to g loses no optimal solution. Contrast with DP, where every locally feasible choice may need to be considered.

Definition 9.3 (Optimal substructure (greedy flavour)). A problem has *optimal substructure* for a greedy algorithm if, after committing to the greedy choice g , an optimal solution to the original instance contains an optimal solution to the residual subproblem – the instance obtained by removing g (and possibly some bookkeeping). Formally: if S^* is optimal and $S^* \ni g$, then $S^* \setminus \{g\}$ is optimal for the residual subproblem. This differs from the DP-flavoured optimal substructure (section 8) in two ways: (i) only *one* subproblem appears, not several; (ii) it is paired with the greedy-choice property to justify recursing on that single subproblem rather than enumerating all candidate decompositions. The proof template is a *cut-and-paste argument*: assume the residual were sub-optimal, paste in the supposedly better residual to get a better solution to the original, contradiction.

Definition 9.4 (Exchange argument). An *exchange argument* is the standard technique for proving the greedy-choice property. Given an arbitrary optimal solution OPT that does *not* contain the greedy choice g , one constructs a new solution OPT' by “exchanging” some element of OPT for g (or by inserting g in place of an element of OPT) and shows that OPT' is feasible and $\text{cost}(\text{OPT}') \leq \text{cost}(\text{OPT})$ (resp. $\text{value}(\text{OPT}') \geq \text{value}(\text{OPT})$). Iterating the exchange transforms OPT into an optimal solution containing g . A close relative is the *stays-ahead* argument, in which

one inducts on the prefix of the greedy solution and shows it is at least as good as the corresponding prefix of any optimal solution; the two are interchangeable in practice.

9.2 Interval scheduling

Example 9.5 (Setup). The *interval scheduling* problem: given n activities $a_1, \dots, a_n$ with start and finish times (s_i, f_i) , select a maximum-cardinality subset of pairwise non-overlapping activities (i.e. for any two selected activities, one finishes before the other starts). The greedy algorithm EARLIEST-FINISH-FIRST (EFF) sorts the activities by finish time and scans left-to-right, greedily selecting an activity iff its start time is at least the finish time of the most recently selected activity.

Theorem 9.6 (Correctness of earliest-finish-first scheduling). *The EARLIEST-FINISH-FIRST greedy algorithm returns a maximum-cardinality set of pairwise non-overlapping activities, in $O(n \log n)$ time (dominated by the sort).*

Proof. The algorithm clearly returns a feasible (non-overlapping) set $G = \{g_1, g_2, \dots, g_k\}$ ordered by finish time. We show $|G|$ matches any optimum by an exchange/stays-ahead argument. Let $O = \{o_1, o_2, \dots, o_m\}$ be *any* optimal solution, also ordered by finish time. We claim that for every $i \leq k$,

$$f(g_i) \leq f(o_i),$$

i.e. the i -th greedy pick finishes no later than the i -th optimal pick. The proof is by induction on i .

Base $i = 1$. Greedy picks the globally earliest-finishing activity, so $f(g_1) \leq f(o_1)$.

Inductive step. Suppose $f(g_{i-1}) \leq f(o_{i-1})$. Then o_i starts at $s(o_i) \geq f(o_{i-1}) \geq f(g_{i-1})$, so o_i was a candidate when greedy chose g_i (it does not overlap any previously chosen g_j). Greedy chose the activity of *earliest* finish time among such candidates, so $f(g_i) \leq f(o_i)$.

Now suppose for contradiction that $|G| < |O|$, i.e. $k < m$. Then o_{k+1} exists, with $s(o_{k+1}) \geq f(o_k) \geq f(g_k)$ (using the claim at $i = k$). Hence o_{k+1} does not overlap any of $g_1, \dots, g_k$ and was a candidate for the $(k+1)$ -th greedy pick – but greedy terminated, contradiction. Therefore $|G| \geq |O|$, so $|G|$ is optimal.

The running time is $O(n \log n)$ to sort by finish time plus a single $O(n)$ scan, hence $O(n \log n)$ overall.

Remark 9.7 (Stays-ahead vs. exchange framing). For interval scheduling the cleanest argument is *stays-ahead*: induct on the prefix index and show that the i -th greedy pick finishes no later than the i -th pick of any optimum. For Huffman coding and fractional knapsack, the cleanest is a direct *exchange* (swap two leaves, swap two items). Both establish the greedy-choice property; pick the framing that matches the structure of the problem. Whichever framing is chosen, the exchange step is load-bearing: state OPT, exhibit the modification that produces an OPT' containing the greedy choice, and verify $\text{cost}(\text{OPT}') \leq \text{cost}(\text{OPT})$ (resp. $\geq$). Hand-waving (“intuitively the heaviest item should go first”) is not a substitute.

Example 9.8 (Earliest-finish-first on a small input). Consider activities (start, finish): $a_1 = (1, 4)$, $a_2 = (3, 5)$, $a_3 = (0, 6)$, $a_4 = (5, 7)$, $a_5 = (3, 9)$, $a_6 = (5, 9)$, $a_7 = (6, 10)$, $a_8 = (8, 11)$, $a_9 = (8, 12)$, $a_{10} = (2, 14)$, $a_{11} = (12, 16)$.

Step 1. Sort by finish time (already sorted above).

Step 2. Sweep left-to-right with $f_{\text{last}} = -\infty$:

- $a_1 = (1, 4)$: $1 \geq -\infty$, select. $f_{\text{last}} = 4$.
- $a_2 = (3, 5)$: $3 < 4$, skip. $a_3 = (0, 6)$: $0 < 4$, skip.
- $a_4 = (5, 7)$: $5 \geq 4$, select. $f_{\text{last}} = 7$.
- $a_5 = (3, 9), a_6 = (5, 9), a_7 = (6, 10)$: all start before 7, skip.
- $a_8 = (8, 11)$: $8 \geq 7$, select. $f_{\text{last}} = 11$.
- $a_9 = (8, 12)$: $8 < 11$, skip. $a_{10} = (2, 14)$: skip.
- $a_{11} = (12, 16)$: $12 \geq 11$, select. $f_{\text{last}} = 16$.

Output: $\{a_1, a_4, a_8, a_{11}\}$, of size 4. By Theorem 9.6 this is optimal. Note that the alternative greedy “pick the activity of shortest duration first” would fail (e.g. a short central activity can preclude two longer non-overlapping ones).

9.3 Huffman coding

The *prefix coding* problem: given an alphabet $A = \{a_1, \dots, a_n\}$ with frequencies $f(a_i)$, find an encoding $\gamma : A \rightarrow \{0, 1\}^*$ in which no codeword is a prefix of another, minimising the average bit length $\text{ABL}(\gamma) = \sum_{x \in A} f(x) |\gamma(x)|$. By the standard correspondence, prefix codes on n alphabets are in bijection with full binary trees on n leaves (each leaf labelled by an alphabet, each codeword being the root-to-leaf 0/1 path label), and $\text{ABL}(\gamma) = \sum_{x \in A} f(x) \text{depth}_T(x)$ where T is the corresponding tree. *Huffman’s algorithm* repeatedly merges the two lowest-frequency alphabets:

Algorithm 3 HUFFMAN(A, f)

- 1: **if** $|A| = 1$ **then return** a single leaf.
 - 2: **end if**
 - 3: Let $a_1, a_2 \in A$ be the two alphabets of *smallest* frequency.
 - 4: Replace a_1, a_2 by a fresh symbol $a_{1,2}$ with $f(a_{1,2}) = f(a_1) + f(a_2)$.
 - 5: Recursively call HUFFMAN on $\{a_3, \dots, a_n, a_{1,2}\}$ to obtain a tree T' .
 - 6: In T' , replace the leaf $a_{1,2}$ by an internal node with children a_1 and a_2 . **return** the tree.
-

Using a min-heap keyed by frequency, the running time is $O(n \log n)$.

Theorem 9.9 (Correctness of Huffman coding). HUFFMAN(A, f) produces a prefix code of minimum average bit length.

Proof. (*sketch.*) The proof has two ingredients.

Tree-shape invariant. The binary tree of any optimal prefix code is *full* (every internal node has exactly two children): an internal node with one child could be deleted, shrinking all codewords below it and strictly decreasing ABL, contradicting optimality.

Greedy-choice (exchange). Order the alphabets so that $f(a_1) \leq f(a_2) \leq \dots \leq f(a_n)$. We claim there is an optimal tree in which a_1 and a_2 are siblings at the deepest level. Take any optimal tree T . Pick a deepest leaf; by the full-tree property its sibling is also a leaf – call them x, y . Swapping a_1 with x and a_2 with y can only weakly decrease ABL, because the lowest-frequency symbols are being moved to the deepest positions: the change in ABL is $(f(x) - f(a_1))(d(a_1) - d(x)) + (f(y) - f(a_2))(d(a_2) - d(y)) \leq 0$ since $f(x) \geq f(a_1)$, $d(x) \geq d(a_1)$, and similarly for y .

Optimal substructure (cut-and-paste). Let $A' = (A \setminus \{a_1, a_2\}) \cup \{a_{1,2}\}$ with $f(a_{1,2}) = f(a_1) + f(a_2)$, and let T' be any tree for A' . Building T from T' by attaching a_1, a_2 as children of $a_{1,2}$ gives $\text{ABL}(T) = \text{ABL}(T') + f(a_1) + f(a_2)$ (since a_1, a_2 inherit depth $d(a_{1,2}) + 1$, contributing $(f(a_1) + f(a_2))(d(a_{1,2}) + 1) = f(a_{1,2})d(a_{1,2}) + (f(a_1) + f(a_2))$). The additive constant is independent of T' , so an optimal T' produces an optimal T . Cut-and-paste: if a better T'' existed, replacing siblings a_1, a_2 by a merged leaf gives a tree on A' better than T' – contradiction.

Combining the two: greedy-choice ensures the two least-frequent alphabets can be made siblings; optimal substructure then reduces the problem to A' , on which the algorithm recurses. Induction on n closes the proof.

Example 9.10 (Huffman tree construction). Frequencies $f(a) = 0.45$, $f(b) = 0.18$, $f(c) = 0.15$, $f(d) = 0.12$, $f(e) = 0.10$.

Step 1. Two smallest: $d(0.12), e(0.10)$. Merge into de with frequency 0.22. Forest: $\{a : 0.45, b : 0.18, c : 0.15, de : 0.22\}$.

Step 2. Two smallest: $c(0.15), b(0.18)$. Merge into bc with frequency 0.33. Forest: $\{a : 0.45, bc : 0.33, de : 0.22\}$.

Step 3. Two smallest: $de(0.22), bc(0.33)$. Merge into $bcde$ with frequency 0.55. Forest: $\{a : 0.45, bcde : 0.55\}$.

Step 4. Merge a and $bcde$ into the root.

Reading 0 = left, 1 = right yields the optimal codes $a \mapsto 0$, $b \mapsto 100$, $c \mapsto 101$, $d \mapsto 110$, $e \mapsto 111$, giving $\text{ABL} = 0.45 \cdot 1 + (0.18 + 0.15 + 0.12 + 0.10) \cdot 3 = 0.45 + 1.65 = 2.10$ bits/symbol. (Compare with fixed-length $\lceil \log_2 5 \rceil = 3$.)

9.4 Minimum spanning trees

Example 9.11 (Setup). Given a connected undirected graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}_{>0}$ (assume distinct for simplicity), find a *spanning tree* – a subset $T \subseteq E$ with $|T| = |V| - 1$ that connects all vertices – of minimum total weight $w(T) = \sum_{e \in T} w(e)$.

The greedy structure of MST rests on a single deep fact, the *cut property*, which is itself the greedy-choice property in disguise.

Theorem 9.12 (Cut property). Let T be the (unique) MST of G , let $A \subsetneq V$, and let $(u, v) \in E$ be the minimum-weight edge crossing the cut $(A, V \setminus A)$. Then $(u, v) \in T$.

Proof. Suppose for contradiction that $(u, v) \notin T$. Since T is a spanning tree, there is a unique u -to- v path in T . This path begins in A (at u) and ends in $V \setminus A$ (at v), so it must contain at least one edge $e = (x, y)$ that crosses the cut, with $x \in A$, $y \in V \setminus A$. Now consider $T' = T \setminus \{e\} \cup \{(u, v)\}$:

- T' has $|V| - 1$ edges (we removed one and added one).
- Removing e disconnects T into two components, one containing $A \cap T$ vertices reachable from u side and the other containing the $V \setminus A$ side; adding (u, v) reconnects them, so T' is connected.
- Hence T' is a spanning tree.
- By choice of (u, v) as the *minimum* crossing edge, $w(u, v) < w(e)$ (using distinctness), so $w(T') < w(T)$.

This contradicts T being the MST. Therefore $(u, v) \in T$.

Remark 9.13 (Tied edge weights). The proof above uses distinct edge weights so that the swap strictly decreases the total weight. With ties the MST need not be unique; the cut property still holds with “minimum” read as “one of the minimum” crossing edges, and Kruskal/Prim still produce *some* MST.

The cut property is the engine that powers both standard MST algorithms below: Prim builds the tree by repeatedly applying the cut property to the cut $(A, V \setminus A)$ where A is the set of vertices already attached; Kruskal applies it implicitly by adding the globally-cheapest non-cycle-forming edge.

Algorithm 4 PRIM(G, w, r)

```

1:  $A \leftarrow \{r\}, T \leftarrow \emptyset.$ 
2: while  $A \neq V$  do
3:   Let  $e = (u, v)$  be the minimum-weight edge with  $u \in A, v \in V \setminus A.$ 
4:    $T \leftarrow T \cup \{e\}, A \leftarrow A \cup \{v\}.$ 
5: end while
6: return  $T.$ 

```

Theorem 9.14 (Correctness of Prim). PRIM returns the MST of G . With a Fibonacci-heap-backed priority queue, it runs in $O(|E| + |V| \log |V|)$ time.

Proof. (*sketch.*) Loop invariant: after each iteration, T is a subset of the MST. Initially $T = \emptyset \subseteq$ MST. Each iteration adds the minimum-weight edge crossing the cut $(A, V \setminus A)$, which by the cut property (Theorem 9.12) is in the MST. The invariant is preserved. The loop terminates when $|T| = |V| - 1$, at which point T is a spanning tree contained in the MST – hence T is the MST.

Algorithm 5 KRUSKAL(G, w)

```

1:  $T \leftarrow \emptyset.$  Sort  $E$  in increasing order of weight.
2: for each edge  $e = (u, v) \in E$  in sorted order do
3:   if  $T \cup \{e\}$  is acyclic then  $T \leftarrow T \cup \{e\}.$ 
4:   end if
5: end for
6: return  $T.$ 

```

Theorem 9.15 (Correctness of Kruskal). KRUSKAL returns the MST of G . With union-find, it runs in $O(|E| \log |V|)$ time.

Proof. (*sketch.*) At the moment edge $e = (u, v)$ is considered, suppose adding it would not create a cycle. Let A be the connected component of u in the current T . Then $v \notin A$, and every edge in E already crossing $(A, V \setminus A)$ has weight $\geq w(e)$ (all lighter edges have been considered, and by the no-cycle test would have been added if they connected A to $V \setminus A$, but they did not – they were either within A or rejected for forming a cycle elsewhere; a careful argument shows e is in fact the lightest crossing edge). By the cut property, $e \in$ MST. Conversely, any rejected edge forms a cycle with already-chosen edges of strictly smaller weight, so it cannot belong to the MST (cycle property, dual of cut property). Hence the algorithm outputs exactly the MST.

Example 9.16 (Prim’s algorithm step-by-step). Take a graph with vertices $\{v_1, \dots, v_7\}$ and edges of weights $\{3, 5, 6, 7, 8, 9, 10, 12, 14, 15\}$ (unique). Concretely, edges are $v_2v_3 = 3$, $v_4v_5 = 5$, $v_1v_4 = 6$, $v_5v_6 = 7$, $v_2v_5 = 8$, $v_5v_7 = 9$, $v_3v_6 = 10$, $v_1v_5 = 12$, $v_2v_4 = 14$, $v_6v_7 = 15$. Start Prim from v_6 .

Iter	A before	cut edges (weight)	min edge	T after
1	$\{v_6\}$	$v_5v_6 = 7$, $v_3v_6 = 10$, $v_6v_7 = 15$	$v_5v_6 = 7$	$\{v_5v_6\}$
2	$\{v_5, v_6\}$	$v_4v_5 = 5$, $v_2v_5 = 8$, $v_5v_7 = 9$, $v_3v_6 = 10$	$v_4v_5 = 5$	$\{v_5v_6, v_4v_5\}$
3	$+v_4$	$v_1v_4 = 6$, $v_2v_5 = 8$, $v_2v_4 = 14, \dots$	$v_1v_4 = 6$	$+v_1v_4$
4	$+v_1$	$v_2v_5 = 8$, $v_5v_7 = 9$, $v_3v_6 = 10, \dots$	$v_2v_5 = 8$	$+v_2v_5$
5	$+v_2$	$v_2v_3 = 3$, $v_5v_7 = 9, \dots$	$v_2v_3 = 3$	$+v_2v_3$
6	$+v_3$	$v_5v_7 = 9$, $v_6v_7 = 15$	$v_5v_7 = 9$	$+v_5v_7$

After iteration 6 all 7 vertices are in A , and $T = \{v_5v_6, v_4v_5, v_1v_4, v_2v_5, v_2v_3, v_5v_7\}$ has $|V| - 1 = 6$ edges and total weight $3 + 5 + 6 + 7 + 8 + 9 = 38$. (Kruskal would produce the same edges, in the order 3, 5, 6, 7, 8, 9, rejecting weights 10, 12, 14, 15 as cycle-forming.)

9.5 Approximation algorithms by greedy

When a problem is NP-hard, polynomial-time algorithms are not expected to compute the exact optimum. A common workaround is to settle for a solution provably within a constant factor of the optimum, computed by a polynomial-time greedy algorithm. Two canonical examples are SET-COVER (greedy gives an $O(\log n)$ -approximation) and VERTEX-COVER via maximal matching (greedy gives a 2-approximation). The latter has the cleanest proof and is treated in detail below.

Definition 9.17 (Approximation ratio). For an optimisation problem and a constant $\alpha \geq 1$, an algorithm $\mathcal{A}$ is an α -approximation algorithm if, for every instance I with optimum value $\text{OPT}(I)$,

$$\text{OPT}(I) \leq \mathcal{A}(I) \leq \alpha \cdot \text{OPT}(I) \quad (\text{minimisation}),$$

$$\frac{\text{OPT}(I)}{\alpha} \leq \mathcal{A}(I) \leq \text{OPT}(I) \quad (\text{maximisation}),$$

and $\mathcal{A}$ runs in polynomial time. Smaller α is better; $\alpha = 1$ recovers exact algorithms. The ratio may be a function of the input size, e.g. $\alpha = O(\log n)$ for set cover.

Example 9.18 (Vertex cover via maximal matching). VERTEX-COVER: given an undirected graph $G = (V, E)$, find a smallest $C \subseteq V$ such that every edge has at least one endpoint in C . The decision version is NP-complete. The greedy approach: compute any *maximal matching* M (greedily add edges with both endpoints unmatched until no such edge remains – not maximum, just maximal), and output the set of matched vertices.

Theorem 9.19 (Maximal matching gives a 2-approximation for Vertex Cover). *Let M be any maximal matching in G and let $C(M)$ be the set of $2|M|$ endpoints of M ’s edges. Then $C(M)$ is a vertex cover, and $|C(M)| \leq 2 \cdot \text{VC}(G)$, where $\text{VC}(G)$ is the minimum vertex cover size. In particular, the greedy algorithm is a polynomial-time 2-approximation for VERTEX-COVER.*

Proof. $C(M)$ is a vertex cover. If some edge (u, v) had neither endpoint in $C(M)$, then u, v are both unmatched in M , so $M \cup \{(u, v)\}$ is a matching, contradicting maximality of M . Hence every edge has an endpoint in $C(M)$.

Lower bound. The edges of M are pairwise disjoint, so any vertex cover must contain at least one endpoint of each edge in M – i.e. $\text{VC}(G) \geq |M|$.

Upper bound. By construction, $|C(M)| = 2|M|$.

Combining,

$$|C(M)| = 2|M| \leq 2 \cdot \text{VC}(G),$$

which is the 2-approximation bound. The maximal matching M can be computed in $O(|V| + |E|)$ time by a single edge-scan, so the algorithm runs in linear time.

Remark 9.20 (Greedy need not optimise locally). The maximal-matching algorithm picks edges *arbitrarily*: there is no local optimisation at each step. “Greedy” here means “one-shot, no back-tracking”; it does not require picking the locally best item. Conflating the two leads to spurious objections that the algorithm “isn’t really greedy”.

Example 9.21 (A small vertex-cover run). Take the path $P_4 = v_1 - v_2 - v_3 - v_4$ (3 edges). Greedy maximal matching scans edges in some order; suppose it picks (v_1, v_2) first. Now v_1, v_2 are matched, so the candidate edges (v_2, v_3) and (v_3, v_4) : the first has v_2 already matched – skip; the second, v_3, v_4 both unmatched – add. Matching $M = \{(v_1, v_2), (v_3, v_4)\}$, $|M| = 2$.

The output cover is $C(M) = \{v_1, v_2, v_3, v_4\}$ of size 4. The optimal vertex cover is $\{v_2, v_3\}$ of size 2, so the approximation ratio achieved on this instance is $4/2 = 2$ – exactly the worst case allowed by the theorem. Note that with a different edge order greedy might pick (v_2, v_3) first, yielding $M = \{(v_2, v_3)\}$ and cover $\{v_2, v_3\}$ of size 2 (optimal). So the analysis is tight against the worst-case greedy run, but the algorithm is still polynomial-time and *never* worse than $2 \cdot \text{OPT}$.

Remark 9.22 (Other greedy approximations). SET-COVER (cover $\{1, \dots, n\}$ by the fewest subsets from a family $\mathcal{S}$) admits a parallel treatment: the greedy algorithm that repeatedly picks the set covering the largest number of as-yet-uncovered elements is an $O(\log n)$ -approximation. The standard analysis tracks the uncovered set: since OPT sets cover everything, by pigeonhole some set in the optimum covers at least a $1/\text{OPT}$ fraction of whatever is currently uncovered, and greedy picks at least as good. After k rounds at most $(1 - 1/\text{OPT})^k n \leq n e^{-k/\text{OPT}}$ elements remain, which drops below 1 once $k > \text{OPT} \ln n$. Hence greedy terminates within $\text{OPT} \cdot (1 + \ln n) = O(\log n) \cdot \text{OPT}$ rounds (the tight constant is the n -th harmonic number H_n). The ratios are tight in a strong sense: no polynomial-time $(\sqrt{2} - \varepsilon)$ -approximation exists for VERTEX-COVER unless $\text{P} = \text{NP}$, and INDEPENDENT-SET admits no constant-factor approximation under the same hypothesis – a striking contrast given that VERTEX-COVER and INDEPENDENT-SET are complementary.

Remark 9.23 (When greedy fails: 0/1 vs. fractional knapsack). Sorting by value-per-weight and greedily packing is optimal for *fractional* knapsack but *wrong* for 0/1 knapsack: with items $(w, v) = (1, 2), (5, 9)$ and $W = 5$, the ratios are 2 and 1.8, so greedy takes $(1, 2)$ first, leaving capacity 4 in which the weight-5 item no longer fits, giving value 2; the optimum takes $(5, 9)$ alone for value 9. The gap can be made arbitrarily large by scaling the heavy item, so no constant-factor bound saves the ratio greedy on 0/1 knapsack – it requires DP (section 8). The fractional variant also exhibits a subtler trap: when several items share the same value-per-weight ratio, the exchange argument must compare the greedy choice against *any* optimum that ties on the ratio, not only those that pick the same physical item.

10 Amortised Analysis

For data structures deployed in long sequences of operations, the worst case for any single call is often rare: one resize buys many cheap insertions, one cascade of bit-flips buys many silent increments. *Amortised analysis* accounts for the cost of a sequence of n operations as a whole,

reporting the per-operation average as a worst-case guarantee over input sequences. This is *not* average-case analysis: no probability distribution on inputs is involved. Three techniques compute the bound – aggregate, accounting, and potential methods – which differ only in the bookkeeping.

10.1 Definitions

Throughout, fix a data structure and a sequence of operations $o_1, o_2, \dots, o_n$. Let t_i denote the *actual* (true) cost of o_i (e.g., the number of RAM steps), and let $T(n) = \sum_{i=1}^n t_i$ denote the total actual cost.

Definition 10.1 (Amortised cost). Given a sequence of n operations with actual costs $t_1, \dots, t_n$, the *amortised cost* of the sequence is the average actual cost

$$\frac{T(n)}{n} = \frac{1}{n} \sum_{i=1}^n t_i.$$

We say each operation has *amortised cost* $f(n)$ when $T(n) \leq n \cdot f(n)$ for every legal input sequence, equivalently when the per-operation average is $O(f(n))$ in the worst case over input sequences. The amortised cost is a *worst-case* guarantee on the average; no probability is involved. Different operations sharing the same data structure may be assigned *different* amortised costs (see the queue example below).

Remark 10.2 (Amortised is not average-case). Average-case analysis averages over a probability distribution on inputs; amortised analysis averages over a sequence of operations on a single (worst-case) input. There is no random variable, no $\mathbb{E}$, and no probability assumption – the bound holds against an adversarial sequence.

Definition 10.3 (Aggregate method). The *aggregate method* bounds the total cost $T(n)$ of any sequence of n operations directly, then divides by n to obtain a uniform per-operation amortised cost $T(n)/n$. The same amortised cost is assigned to every operation, regardless of which operation it is. The method is conceptually simple but inflexible: it cannot give different amortised costs to different operations.

Definition 10.4 (Accounting method). The *accounting method* (also *banker's method*) assigns to the i -th operation a *fictitious amortised cost* c_i , possibly different per operation type. The difference $c_i - t_i$ is treated as *credit* deposited in a bank account associated with the data structure (or with specific items inside it). Cheap operations are over-charged so they leave behind credit; expensive operations are under-charged and pay the deficit out of accumulated credit. The analysis must verify the *bank invariant*: the running total $\sum_{j=1}^i (c_j - t_j) \geq 0$ for every prefix i – not merely at $i = n$, since a transient deficit would mean an actual cost paid by credit not yet earned. When this holds,

$$\sum_{i=1}^n t_i \leq \sum_{i=1}^n c_i,$$

so the total amortised cost upper-bounds the total actual cost.

Definition 10.5 (Potential function). A *potential function* for a data structure is a map Φ from the set of states the structure can take to $\mathbb{R}_{\geq 0}$. We write $\Phi(i)$ for the potential after the i -th operation, and $\Phi(0)$ for the initial potential. Conventionally $\Phi(0) = 0$ and $\Phi(i) \geq \Phi(0)$ for all i , so the potential is always non-negative relative to the start.

Definition 10.6 (Potential method). Given a potential function Φ as above, the *potential method* defines the amortised cost of the i -th operation by

$$c_i = t_i + \Phi(i) - \Phi(i-1).$$

The change in potential $\Delta\Phi_i = \Phi(i) - \Phi(i-1)$ acts as a virtual deposit into ($\Delta\Phi_i > 0$) or withdrawal from ($\Delta\Phi_i < 0$) the data structure. Telescoping over $i = 1, \dots, n$,

$$\sum_{i=1}^n c_i = \sum_{i=1}^n t_i + \Phi(n) - \Phi(0).$$

Whenever $\Phi(n) \geq \Phi(0)$ (in particular, with $\Phi(0) = 0$ and $\Phi \geq 0$), the right-hand side dominates $\sum t_i$, so $\sum c_i$ is a valid upper bound on the total actual cost. The potential method is the most flexible of the three: choosing Φ cleverly often yields constant amortised costs even when actual costs are wildly variable.

Remark 10.7 (Keep the potential non-negative). The bound $\sum t_i \leq \sum c_i$ relies on $\Phi(n) \geq \Phi(0)$. With the standard convention $\Phi(0) = 0$, this requires Φ to stay non-negative on every state reachable by the operation sequence. A potential that can dip below its starting value silently invalidates the analysis; sign errors here are the most common defect in potential-method proofs.

10.2 The three methods

Theorem 10.8 (Amortised cost bounds total actual cost). For any sequence of n operations with actual costs $t_1, \dots, t_n$ and any assignment of amortised costs $c_1, \dots, c_n$ satisfying either

- (i) (accounting) $\sum_{j=1}^i c_j \geq \sum_{j=1}^i t_j$ for all $i \in \{1, \dots, n\}$, or
- (ii) (potential) $c_i = t_i + \Phi(i) - \Phi(i-1)$ for some Φ with $\Phi(n) \geq \Phi(0)$,

the total amortised cost upper-bounds the total actual cost: $\sum_{i=1}^n t_i \leq \sum_{i=1}^n c_i$. In particular, if every $c_i \leq \bar{c}$ then the per-operation amortised cost is at most $\bar{c}$, i.e. $T(n)/n \leq \bar{c}$.

Proof. Case (i) is immediate from the prefix inequality at $i = n$. For case (ii), telescope:

$$\sum_{i=1}^n c_i = \sum_{i=1}^n (t_i + \Phi(i) - \Phi(i-1)) = \sum_{i=1}^n t_i + \Phi(n) - \Phi(0) \geq \sum_{i=1}^n t_i,$$

where the last inequality uses $\Phi(n) \geq \Phi(0)$. Both forms therefore yield $\sum t_i \leq \sum c_i$. The aggregate method is the special case of (i) in which all c_i are equal to $T(n)/n$.

Remark 10.9 (Choosing a method). The aggregate method (Definition 10.3) is fastest when one can write down a clean closed form for $T(n)$, e.g. the doubling pattern below. The accounting method (Definition 10.4) shines when the structure suggests a natural “charge per item”; it and the potential method both permit different amortised costs for different operation types, whereas the aggregate method assigns a single uniform cost. The potential method (Definition 10.6) is the most powerful and the standard tool for non-trivial structures (Fibonacci heaps, splay trees), but it requires the right Φ , which is sometimes the whole problem. All three methods are bookkeeping reformulations of the same accounting identity (Theorem 10.8) and so produce the same asymptotic bound – a discrepancy in constants between two methods on the same problem signals an error in one of them.

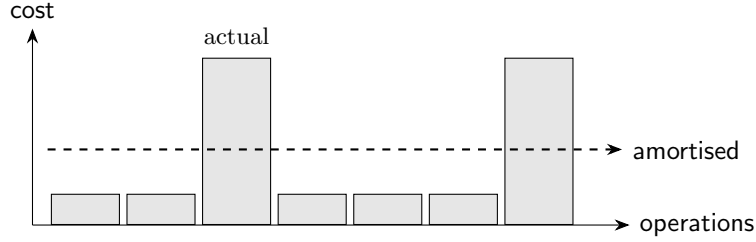


Figure 5: Amortised analysis smooths the spiky actual cost into a flat per-operation cost via a potential function. Solid bars are actual costs t_i ; the dashed line is the amortised cost c_i . Tall spikes (resizes, cascading bit-flips) are paid for by accumulated potential.

Example 10.10 (Queue with INSERT and EMPTY). A queue supports $\text{INSERT}(x)$ at $\Theta(1)$ actual cost and $\text{EMPTY}()$ which deletes all k current elements at $\Theta(k)$ actual cost. A naive worst-case bound gives $O(n)$ per EMPTY , so $O(n^2)$ for n operations. **Aggregate.** Each element is inserted at most once and deleted at most once, so over n operations the total work is $\leq 2n$, giving amortised cost $O(1)$. **Accounting.** Charge INSERT amortised cost 2 (true cost 1, with 1 stored on the inserted element) and EMPTY amortised cost 0 (the k deletions are paid by the k stored credits). **Potential.** Set $\Phi(i) = \text{number of elements in the queue}$. Then $c_{\text{INSERT}} = 1 + 1 = 2$ and $c_{\text{EMPTY}} = k + (0 - k) = 0$. All three give the same conclusion.

10.3 Binary counter

A k -bit binary counter supports INCREMENT , which scans the array $A[0..k-1]$ from the low end, flipping 1s to 0s until it finds a 0, then flips that 0 to 1. The actual cost is the number of bit-flips. A single INCREMENT on the all-ones counter flips k bits, so the worst-case per-operation cost is $\Theta(k) = \Theta(\log n)$ over n increments – giving the loose bound $O(n \log n)$. The amortised analysis sharpens this to $O(n)$.

Theorem 10.11 (Binary counter has $O(1)$ amortised increment). *Starting from the all-zeros counter, a sequence of n INCREMENT operations performs at most $2n$ bit-flips. Hence the amortised cost per increment is $O(1)$.*

Proof. We give the accounting and potential proofs; the aggregate proof is recorded as Example 10.12.

Accounting method. Charge each bit-flip $0 \rightarrow 1$ a fictitious cost of \$2: \$1 pays for the actual flip, and \$1 is deposited on the bit. Charge the bit-flip $1 \rightarrow 0$ nothing – it is paid by withdrawing the \$1 stored on that bit when it was last set. Bank invariant: at any point, every 1-bit of the counter holds exactly \$1 of credit (proof by induction on the number of increments: setting a bit deposits \$1, resetting it withdraws \$1). The bank balance equals the number of 1-bits in the counter, which is ≥ 0 always. Each INCREMENT performs exactly one $0 \rightarrow 1$ flip (the final bit it sets), so its amortised cost is exactly \$2 = $O(1)$. Total amortised cost over n increments is $2n$, which upper-bounds the total actual cost.

Potential method. Take $\Phi(i) = \text{number of 1-bits after the } i\text{-th increment}$. Then $\Phi(0) = 0$ and $\Phi(i) \geq 0$. Suppose the i -th increment flips r_i trailing 1s to 0s and then sets one 0 to 1. Its actual cost is $t_i = r_i + 1$ and the change in potential is $\Phi(i) - \Phi(i-1) = -r_i + 1$. Hence

$$c_i = t_i + \Phi(i) - \Phi(i-1) = (r_i + 1) + (-r_i + 1) = 2.$$

By Theorem 10.8, $\sum t_i \leq \sum c_i = 2n$, and the per-increment amortised cost is $O(1)$.

The two analyses are equivalent: the bank balance “\$1 per 1-bit” is exactly the potential function “number of 1-bits”.

Example 10.12 (Binary counter via aggregate method). Bit j flips every 2^{j+1} increments (it toggles when increment i has $2^j \mid i$ for the appropriate parity). Over n increments, bit j flips at most $\lfloor n/2^j \rfloor$ times. The total number of flips is therefore

$$T(n) = \sum_{j=0}^{k-1} \left\lfloor \frac{n}{2^j} \right\rfloor \leq n \sum_{j=0}^{\infty} \frac{1}{2^j} = 2n.$$

The geometric series swallows the cost of all higher-order bits: bit 0 flips n times, bit 1 flips $n/2$ times, bit 2 flips $n/4$ times, and so on. Dividing by n gives amortised cost $\leq 2 = O(1)$ per increment. The aggregate method recovers the same bound as the accounting and potential analyses, and is arguably the cleanest of the three for this problem.

10.4 Dynamic array

A *dynamic array* (Java `ArrayList`, C++ `std::vector`, Python `list`) supports `PUSH( $x$ )`, which appends x to the array. When the underlying buffer is full, a new buffer of *double* the capacity is allocated, all existing elements are copied over, and the old buffer is freed. We charge $\Theta(1)$ for an in-place push and $\Theta(n)$ for a doubling push that copies n elements. A single doubling push therefore costs $\Theta(n)$, but doublings are rare: between successive doublings the array absorbs $\Theta(n)$ cheap pushes.

Theorem 10.13 (Dynamic-array push has $O(1)$ amortised cost). *Starting from an empty dynamic array with doubling growth, a sequence of n PUSH operations costs at most $3n$ slot-writes total. Hence the amortised cost per push is $O(1)$, even though a single push may cost $\Theta(n)$ in the worst case.*

Remark 10.14 (Geometric growth is essential). Growing by a constant additive c each resize gives total cost $\Theta(n^2)$ and amortised cost $\Theta(n)$, not $O(1)$. Any constant geometric growth factor > 1 keeps the series convergent and yields $O(1)$ amortised cost; doubling is the standard choice. With deletions, halving the buffer when load drops below $1/4$ (rather than $1/2$) preserves $O(1)$ amortised: halving at $1/2$ admits an adversarial alternation of insert/delete that forces a resize on every other operation.

Proof. Three independent proofs follow, one per method. They are recorded as worked examples to make the proof patterns visible side-by-side.

Aggregate method (Example 10.15). Doubling occurs after pushes that fill the array exactly. Among n pushes, the doublings happen at sizes $1, 2, 4, 8, \dots$ up to the largest power of two $\leq n$, copying $1 + 2 + 4 + \dots + 2^{\lfloor \log_2(n-1) \rfloor} \leq 2(n-1) < 2n$ elements in total. Adding the n ordinary writes for the pushes themselves, $T(n) \leq n + 2n = 3n$, so the amortised cost is $\leq 3 = O(1)$.

Accounting method (Example 10.16). Charge each push \$3. The full proof is in the example.

Potential method (Example 10.17). Take $\Phi(D) = 2 \cdot \text{size}(D) - \text{capacity}(D)$, defined on states D where $\text{size} \geq \text{capacity}/2$ (the standard regime after the first doubling); set $\Phi = 0$ when $\text{size} < \text{capacity}/2$. The amortised cost works out to 3 in both cases. Details in the example.

Example 10.15 (Dynamic array, aggregate method). Suppose the array starts at capacity 1 and the buffer doubles whenever full. After n pushes the buffer has been doubled $\lceil \log_2 n \rceil$ times; the j -th doubling copies 2^{j-1} elements (since the new buffer has capacity 2^j and the old one is full at size 2^{j-1}). The copy cost is therefore

$$\sum_{j=1}^{\lceil \log_2 n \rceil} 2^{j-1} = 2^{\lceil \log_2 n \rceil} - 1 < 2n.$$

Add n for the slot-writes done by the n pushes themselves: $T(n) \leq n + 2n = 3n$. Dividing by n gives amortised cost $\leq 3 \in O(1)$. Doubling – as opposed to “add a constant” growth, which would give $\Theta(n^2)$ total – is essential: any growth factor $\geq 1 + \varepsilon$ for fixed $\varepsilon > 0$ keeps the geometric series convergent and gives $O(1)$ amortised; growth “by one” makes the series arithmetic and gives $\Theta(n)$ amortised.

Example 10.16 (Dynamic array, accounting method). Charge each PUSH a fictitious \$3:

- \$1 pays for the actual write of the new element into its slot.
- \$1 is deposited on the new element, to pay for moving it during the *next* doubling.
- \$1 is deposited on one of the older elements, to pay for moving it during the next doubling.

Bank invariant: just after a doubling at size m (so the new capacity is $2m$ and there are m filled slots), every one of the m “old” slots has zero credit and the array is half-full. As the next m pushes fill the second half, each new push deposits \$1 on itself and \$1 on a distinct old element, so by the time the buffer is full again (m pushes later), every one of the $2m$ elements carries \$1 of credit. The next doubling copies $2m$ elements, paid for entirely by the $2m$ stored credits. Bank balance is ≥ 0 at every prefix. Total amortised cost $\leq 3n$, which upper-bounds the total actual cost.

Example 10.17 (Dynamic array, potential method). Define

$$\Phi(D) = 2 \cdot \text{size}(D) - \text{capacity}(D),$$

which is non-negative whenever the array is at least half-full – the regime sustained by doubling. (The first few pushes can be handled by setting $\Phi = 0$ until size first reaches capacity, or simply by starting with capacity 1 and size 1 after the first push, so $\Phi = 1$.) Two cases for the i -th push:

Case 1 (no doubling). The push writes one slot. Size grows by 1, capacity is unchanged, so $\Delta\Phi = 2$. Amortised cost $c_i = 1 + 2 = 3$.

Case 2 (doubling). Just before the push, size = capacity = $i-1$, so $\Phi(i-1) = 2(i-1) - (i-1) = i-1$. The push copies $i-1$ elements (cost $i-1$) into a buffer of capacity $2(i-1)$, then writes the new element (cost 1); total $t_i = i$. After the push, size = i , capacity = $2(i-1)$, so $\Phi(i) = 2i - 2(i-1) = 2$. Hence $\Delta\Phi = 2 - (i-1) = 3 - i$, and

$$c_i = t_i + \Delta\Phi = i + (3 - i) = 3.$$

In both cases $c_i = 3$, so the per-push amortised cost is exactly $3 = O(1)$. The choice $\Phi = 2 \cdot \text{size} - \text{capacity}$ is tuned so that the potential lost at a doubling exactly cancels the linear copy cost.

Part III

Limits of Efficient Computation

11 Reductions and Intractability

A *reduction* from problem A to problem B is a transformation of A -instances into B -instances that preserves the answer, and so witnesses “ A is no harder than B ”. Two consequences follow: if B is easy then so is A , and contrapositively, if A is hard then so is B . This chapter develops the framework – decision problems, the classes P and NP, polynomial-time mapping reductions, NP-hardness and NP-completeness – and proves the structural theorems (transitivity, P-preservation, contrapositive) that the catalogue in section 12 relies on.

11.1 Decision problems and complexity classes

Definition 11.1 (Decision problem). A *decision problem* is a function $\Pi : I \rightarrow \{\text{YES}, \text{NO}\}$ from an instance space I (the set of valid encodings of inputs) to a binary answer. An instance $\alpha \in I$ with $\Pi(\alpha) = \text{YES}$ is a *YES-instance*; otherwise it is a *NO-instance*. Decision problems are the standard form for complexity-theoretic comparisons: every optimisation problem (“find the minimum-weight spanning tree”) has a decision counterpart (“is there a spanning tree of weight $\leq k$?”), and the optimisation version is at least as hard as its decision version, since one call to the optimisation oracle answers the decision question by comparing the optimum to k . Hardness lower bounds therefore lose nothing by restricting to decision problems.

Definition 11.2 (Language). The *language* associated with a decision problem Π is the set

$$L_{\Pi} = \{ \alpha \in I : \Pi(\alpha) = \text{YES} \}$$

of YES-instances. “Solving Π ” and “deciding membership in L_{Π} ” are interchangeable phrasings; the language view lets us treat complexity classes as sets of languages.

Definition 11.3 (Encoding and input size). The complexity of an algorithm is measured as a function of the *length of the encoding* of the input, not its numeric value. We adopt the standard conventions: integers are encoded in binary (so an integer W has encoding length $\Theta(\log W)$), and structured objects (graphs, matrices, lists) are encoded as sequences of parameters in a fixed scheme. Input size is denoted n throughout; for a graph $G = (V, E)$ this is $\Theta(|V| + |E|)$, and for a list of n integers each at most W it is $\Theta(n \log W)$.

Definition 11.4 (Deterministic polynomial time). An algorithm runs in (*deterministic*) *polynomial time* if there is a constant $c \geq 0$ such that on every input of size n it halts in time $O(n^c)$. A decision problem Π is *decidable in polynomial time* if there is a polynomial-time algorithm that on input α outputs $\Pi(\alpha)$. Polynomial time is the formal stand-in for “efficiently solvable”: it is robust under reasonable changes to the computing model (RAM vs. Turing machine alters the exponent but not polynomiality), and the polynomials that arise in practice are small (n , $n \log n$, n^2 , n^3), almost never n^{100} .

Remark 11.5 (Polynomial vs. pseudo-polynomial). An algorithm whose running time is polynomial in the *numeric value* of an input number, rather than in its encoding length, is called *pseudo-polynomial*. The textbook DP for Knapsack runs in $O(nW)$ time on n items with weight bound W ; the input has size $\Theta(n \log W)$, so $O(nW)$ is exponential in the encoding length. Pseudo-polynomial algorithms do *not* certify membership in P.

Definition 11.6 (Class P).

$$P = \{ L \subseteq \Sigma^* : L \text{ is decidable in polynomial time} \}.$$

That is, P is the collection of decision problems for which a deterministic polynomial-time algorithm exists. Membership in P is the formal capture of “tractable”.

Definition 11.7 (Certificate / witness). For a decision problem Π and a YES-instance α , a *certificate* (or *witness*) for α is a string w that proves $\Pi(\alpha) = \text{YES}$ to a polynomial-time verifier. For example, a satisfying assignment is a certificate for SAT; a Hamiltonian cycle is a certificate for HAM-CYCLE; a subset summing to T is a certificate for SUBSET-SUM. The certificate must be polynomial-size in $|\alpha|$, and the verification procedure must run in polynomial time in $|\alpha|$.

Definition 11.8 (Class NP). A decision problem Π belongs to NP if there exist a polynomial p and a deterministic polynomial-time algorithm V (the *verifier*) such that for every instance α ,

$$\Pi(\alpha) = \text{YES} \iff \exists w \in \{0, 1\}^{p(|\alpha|)} \text{ with } V(\alpha, w) = \text{accept}.$$

In words: YES-instances admit a polynomial-size certificate that a polynomial-time verifier accepts; NO-instances admit no such certificate. The two clauses – polynomial *size* of w and polynomial *time* of V – are both required; either alone is insufficient.

Remark 11.9 (Non-deterministic-machine view of NP). Equivalently, NP is the class of decision problems decidable by a *non-deterministic* Turing machine in polynomial time: such a machine, on input α , may guess a string w of polynomial length and then verify it deterministically in polynomial time, accepting α iff some guess leads to acceptance. The two views are interchangeable: the certificate is what the non-deterministic machine guesses, and the verifier is the deterministic post-guess computation. The acronym NP stands for *Non-deterministic Polynomial time*, not “not polynomial”.

Proposition 11.10 ($P \subseteq NP$). Every decision problem decidable in deterministic polynomial time is in NP.

Proof. Let $\Pi \in P$, witnessed by a polynomial-time algorithm A that on input α outputs $\Pi(\alpha)$. Define the verifier $V(\alpha, w) := A(\alpha)$, ignoring the certificate w entirely; let the certificate length be $p(n) = 1$ (any polynomial works). Then V runs in polynomial time and $V(\alpha, w) = \text{accept}$ iff $\Pi(\alpha) = \text{YES}$, regardless of w . So $\Pi \in NP$.

Whether the inclusion is strict – the celebrated P vs. NP question – is open. The widely held conjecture is $P \neq NP$.

11.2 Reductions

The intuition is: a reduction from A to B shows that A is no harder than B , by exhibiting a way to “solve A using B ”. For decision problems the cleanest formulation is a *mapping* (or *many-one*) reduction: a function on instances that preserves YES/NO.

Definition 11.11 (Polynomial-time mapping reduction). Let A and B be decision problems. A *polynomial-time mapping reduction* from A to B , written $A \leq_p B$, is a function f from instances of A to instances of B such that

- (i) **Computability:** f is computable in polynomial time in the size of its input;

(ii) **Correctness:** for every instance α of A ,

$$A(\alpha) = \text{YES} \iff B(f(\alpha)) = \text{YES}.$$

The biconditional in (ii) is essential: both YES-instances must map to YES-instances and NO-instances to NO-instances. The YES/NO answer is then simply “copied” from the B -call back to A .

Remark 11.12 (Direction of $\leq_p$). The notation $A \leq_p B$ should be read “ A is no harder than B ”, or equivalently “ B is at least as hard as A ”. The reduction lets us solve A *using* B : an algorithm for B becomes an algorithm for A by composing with f . Reading the inequality backwards is the most frequent error in this material: to show B is hard, one reduces a known-hard problem *to* B (so the easy side, the source A , sits on the left of $\leq_p$), never the other way around.

Remark 11.13 (Both directions of the biconditional). Clause (ii) is a biconditional, and both halves carry weight: a YES-instance must map to a YES-instance (*forward direction*), and a NO-instance must map to a NO-instance (*reverse direction*). Equivalently: $x \in A \Rightarrow f(x) \in B$, and $f(x) \in B \Rightarrow x \in A$. Forgetting one direction is a frequent slip, and a one-sided proof can hide a reduction that turns NO-instances into YES-instances and so destroys the answer. Note that witness (certificate) translation is a *separate* property sometimes needed to argue NP membership of the target problem; it is not part of the mapping-reduction correctness criterion.

Definition 11.14 (NP-hard). A decision problem B is *NP-hard* if every problem in NP reduces to it in polynomial time:

$$\forall A \in \text{NP}, \quad A \leq_p B.$$

NP-hard problems are at least as hard as every problem in NP. They are not required to lie in NP themselves – some NP-hard problems are even undecidable.

Definition 11.15 (NP-complete). A decision problem B is *NP-complete* if (i) $B \in \text{NP}$ and (ii) B is NP-hard. NP-complete problems are the “hardest” problems in NP: a polynomial-time algorithm for any one of them would yield polynomial-time algorithms for *every* problem in NP, collapsing $\text{P} = \text{NP}$. The Cook–Levin theorem (section 12) establishes that 3-SAT is NP-complete; subsequent NP-completeness proofs then proceed by reduction from 3-SAT (or one of its descendants) to the target.

11.3 Properties of reductions

The three structural theorems below are what make reductions useful. They are the engine that turns a single hardness result (Cook–Levin) into a catalogue of hundreds.

Theorem 11.16 (Transitivity of $\leq_p$). If $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$.

Proof. Let f witness $A \leq_p B$, computable in time $p(n)$ on inputs of size n , with $A(\alpha) = \text{YES} \iff B(f(\alpha)) = \text{YES}$. Let g witness $B \leq_p C$, computable in time $q(m)$ on inputs of size m , with $B(\beta) = \text{YES} \iff C(g(\beta)) = \text{YES}$. Define $h(\alpha) := g(f(\alpha))$.

Correctness: chaining the biconditionals,

$$A(\alpha) = \text{YES} \iff B(f(\alpha)) = \text{YES} \iff C(g(f(\alpha))) = \text{YES} \iff C(h(\alpha)) = \text{YES}.$$

Running time: computing $f(\alpha)$ takes time $p(n)$ and produces an output of size at most $p(n)$ (an algorithm cannot write more than its running time). Then computing g on an input of size

$p(n)$ takes time $q(p(n))$. Total time $p(n) + q(p(n))$. Since the composition of two polynomials is a polynomial, h is computable in polynomial time. Hence $A \leq_p C$.

Remark 11.17 (The polynomial-time clause is not free). The composition argument above relies essentially on clause (i) of Definition 11.11: only because f runs in polynomial time does its output have polynomial size, and only then is g applied to a polynomial-size input. A reduction whose transformation alone runs in superpolynomial time invalidates every consequence drawn from $\leq_p$, even if the YES/NO biconditional is correct. Verifying the running time of the transformation is part of every reduction proof, not an afterthought.

Remark 11.18 (Reflexivity). $A \leq_p A$ trivially, via the identity reduction $f(\alpha) = \alpha$ (computable in linear time, correctness immediate). Together with transitivity, $\leq_p$ is a preorder on decision problems.

Theorem 11.19 (Reductions preserve P). *If $A \leq_p B$ and $B \in P$, then $A \in P$.*

Proof. Let f witness $A \leq_p B$, computable in time $p(n) = O(n^{c_1})$ on inputs of size n . Let A_B be a polynomial-time algorithm for B running in time $q(m) = O(m^{c_2})$ on inputs of size m . Define an algorithm for A as follows: on input α of size n , compute $\beta := f(\alpha)$ in time $p(n)$, then run $A_B(\beta)$ and output its answer.

Correctness: by definition of the reduction, $A(\alpha) = \text{YES} \iff B(\beta) = \text{YES}$, and A_B correctly outputs $B(\beta)$.

Running time: step 1 costs $p(n) = O(n^{c_1})$ and the output β has size at most $p(n)$. Step 2 then costs $q(p(n)) = O((n^{c_1})^{c_2}) = O(n^{c_1 c_2})$. Total $O(n^{c_1}) + O(n^{c_1 c_2}) = O(n^{c_1 c_2})$ (assuming $c_1, c_2 \geq 1$), which is polynomial. So $A \in P$.

Theorem 11.20 (Hardness propagates along reductions). *If $A \leq_p B$ and A is NP-hard, then B is NP-hard. Consequently, if additionally $B \in \text{NP}$, then B is NP-complete.*

Proof. This is the contrapositive of Theorem 11.19 stated at the level of NP-hardness, and it is what every NP-completeness proof uses.

Let C be any problem in NP. Since A is NP-hard, $C \leq_p A$. By hypothesis $A \leq_p B$. By transitivity (Theorem 11.16), $C \leq_p B$. Since $C \in \text{NP}$ was arbitrary, every problem in NP reduces to B , i.e. B is NP-hard.

For the second clause: if in addition $B \in \text{NP}$, then B satisfies both clauses of NP-completeness, so B is NP-complete.

Remark 11.21 (The recipe for NP-completeness proofs). Theorem 11.20 dictates the standard pattern for showing B is NP-complete. (1) Show $B \in \text{NP}$ by exhibiting a polynomial-size certificate and a polynomial-time verifier. NP membership is a separate obligation from NP-hardness and is often the easier half; skipping it leaves only an NP-hardness claim, which is strictly weaker. (2) Pick a problem A already known to be NP-hard, typically 3-SAT or one of its descendants. Picking the wrong source problem – one that is in P, or whose hardness is unknown – proves nothing. (3) Construct f in polynomial time and prove the biconditional $x \in A \iff f(x) \in B$ in both directions: the forward direction ($x \in A \Rightarrow f(x) \in B$) and the reverse direction ($f(x) \in B \Rightarrow x \in A$). Witness translation (showing a certificate for $f(x)$ decodes into one for x) is a separate concern relevant to NP membership of the target, not to the reduction correctness itself. The catalogue of canonical reductions is in section 12.

11.4 Worked example

We give a complete decision-problem reduction in detail; the more involved canonical 3-SAT $\leq_p$ Independent-Set reduction is deferred to section 12.

Example 11.22 (Independent-Set $\leq_p$ Vertex-Cover). Recall the two problems:

- **INDEPENDENT-SET:** given an undirected graph $G = (V, E)$ and an integer k , does G contain a set $S \subseteq V$ of k or more vertices such that no edge of E has both endpoints in S ?
- **VERTEX-COVER:** given $G = (V, E)$ and an integer k' , does G contain a set $C \subseteq V$ of at most k' vertices such that every edge has at least one endpoint in C ?

The reduction. Given an instance (G, k) of INDEPENDENT-SET with $n = |V|$, output the instance $(G, n - k)$ of VERTEX-COVER. The graph is unchanged; only the threshold flips from k to $n - k$.

Polynomial-time computability. The transformation copies G and computes the integer $n - k$. This runs in linear time in the input size, hence in polynomial time.

Correctness, forward direction ($\Rightarrow$). Suppose (G, k) is a YES-instance of INDEPENDENT-SET, witnessed by an independent set S with $|S| \geq k$. We claim $C := V \setminus S$ is a vertex cover of size $\leq n - k$. Size: $|C| = n - |S| \leq n - k$. Cover property: take any edge $(u, v) \in E$. If both $u, v \in S$, then the edge is internal to S , contradicting independence. So at least one of u, v lies in $V \setminus S = C$, i.e. the edge is covered. Hence $(G, n - k)$ is a YES-instance of VERTEX-COVER.

Correctness, reverse direction ($\Leftarrow$). Suppose $(G, n - k)$ is a YES-instance of VERTEX-COVER, witnessed by a vertex cover C with $|C| \leq n - k$. We claim $S := V \setminus C$ is an independent set of size $\geq k$. Size: $|S| = n - |C| \geq n - (n - k) = k$. Independence: suppose for contradiction that some edge $(u, v) \in E$ has both endpoints in $S = V \setminus C$. Then $u, v \notin C$, so C does not cover (u, v) , contradicting that C is a vertex cover. Hence S is independent. So (G, k) is a YES-instance of INDEPENDENT-SET.

Conclusion. Both directions hold and the transformation is polynomial-time, so INDEPENDENT-SET $\leq_p$ VERTEX-COVER. By symmetry of the construction (the same map $(G, k) \mapsto (G, n - k)$ also reduces VERTEX-COVER to INDEPENDENT-SET), the two problems are polynomially equivalent: any polynomial-time algorithm for one yields one for the other. The construction also illustrates the general principle from Theorem 11.20: if either problem is NP-hard, so is the other.

12 NP-Completeness

Building on the machinery of section 11 – decision problems, the classes P and NP, polynomial-time many-one reductions $\leq_p$, and NP-hardness/completeness – this chapter is the operational catalogue. The Cook–Levin theorem seeds the entire NP-complete club at 3-SAT; thereafter, every NP-hardness proof reduces *from* a previously-known NP-hard problem *to* the new one, following the template of input transformation, certificate translation, both-directions correctness, and polynomial-time check. The canonical reductions worked through here are INDEPENDENT-SET $\leq_p$ CLIQUE, INDEPENDENT-SET $\leq_p$ VERTEX-COVER, 3-SAT $\leq_p$ DIRECTED-HAM-CYCLE, and PARTITION-SPECIFIC FANTASTIC-HALF – the last accompanied by an instructive sequence of failed attempts.

12.1 Definitions

The canonical NP-complete problems referenced in this chapter are catalogued below. Each is a decision problem: input as stated, YES/NO output. Witness sizes (for NP membership) are noted in remarks where non-obvious.

Definition 12.1 (3-SAT). Given a Boolean formula φ in conjunctive normal form (CNF) with exactly three literals per clause, decide whether some assignment of truth values to the variables makes $\varphi = 1$.

Definition 12.2 (Circuit-SAT). Given a Boolean circuit C with one output, decide whether some assignment of truth values to the input wires makes the output 1.

Definition 12.3 (Clique). Given an undirected graph $G = (V, E)$ and an integer k , decide whether G contains a *clique* of size k – a subset $S \subseteq V$ with $|S| = k$ and every pair in S joined by an edge.

Definition 12.4 (Independent-Set). Given an undirected graph $G = (V, E)$ and an integer k , decide whether G contains an *independent set* of size k – a subset $S \subseteq V$ with $|S| = k$ and no two vertices of S joined by an edge.

Definition 12.5 (Vertex-Cover). Given an undirected graph $G = (V, E)$ and an integer k , decide whether G has a *vertex cover* of size k – a subset $C \subseteq V$ with $|C| = k$ such that every edge has at least one endpoint in C .

Definition 12.6 (Hamiltonian-Cycle). Given an undirected graph $G = (V, E)$, decide whether G contains a cycle that visits every vertex exactly once.

Definition 12.7 (Directed-Hamiltonian-Cycle). Given a directed graph $G = (V, E)$, decide whether G contains a directed cycle that visits every vertex exactly once.

Definition 12.8 (Subset-Sum). Given non-negative integers $a_1, \dots, a_n$ and a target W , decide whether some subset $S \subseteq \{1, \dots, n\}$ satisfies $\sum_{i \in S} a_i = W$.

Definition 12.9 (Partition). Given non-negative integers $a_1, \dots, a_n$, decide whether $\{1, \dots, n\}$ can be split into two disjoint sets $S, \bar{S}$ with $\sum_{i \in S} a_i = \sum_{i \in \bar{S}} a_i$.

Definition 12.10 (Partition-Special). Given n items, each with a positive integer weight x_i , decide whether the items can be partitioned into two groups of equal cardinality and equal weight.

Definition 12.11 (Fantastic-Half). Given n items with positive integer weights $b_1, \dots, b_n$ summing to B , decide whether some subset of size $n/2$ has weight exactly $B/2$.

12.2 Cook–Levin theorem

Theorem 12.12 (Cook–Levin). 3-SAT is NP-complete. More strongly, every problem in NP poly-time reduces to CIRCUIT-SAT, and CIRCUIT-SAT poly-time reduces to 3-SAT, so 3-SAT is NP-hard. Combined with the fact that 3-SAT $\in$ NP (a satisfying assignment is a polynomial-size, polynomial-time-verifiable certificate), this gives NP-completeness.

Proof. (sketch.) Fix an arbitrary problem $L \in$ NP. By Definition 11.8, there is a polynomial-time verifier $V(x, y)$ and a polynomial p such that $x \in L$ iff there exists a certificate y with $|y| \leq p(|x|)$ and $V(x, y) = 1$. Since V runs in polynomial time, it is computable by a Turing machine M_V with running time bounded by some polynomial $q(|x|)$.

Step 1 (every $L \in$ NP reduces to CIRCUIT-SAT). The computation of M_V on input (x, y) for $q(|x|)$ steps can be unrolled into a Boolean circuit C_x whose inputs are the bits of y . The circuit simulates the tape, head, and state of M_V at each time step using a polynomial-size collection

of gates – one “slice” per time step, each slice computing the next-step configuration from the previous slice using the (constant-size) transition table of M_V . The total size is polynomial in $|x|$ and $q(|x|)$, hence polynomial in $|x|$. The circuit outputs 1 iff $V(x, y) = 1$. Thus $x \in L$ iff C_x has some satisfying input y , i.e. $C_x \in \text{CIRCUIT-SAT}$. The map $x \mapsto C_x$ is computable in polynomial time.

Step 2 ($\text{CIRCUIT-SAT} \leq_p \text{3-SAT}$). Given a Boolean circuit, introduce one variable per wire. For each gate, write a small CNF formula encoding “output wire equals gate-of-input-wires” (e.g. for an AND gate with inputs a, b and output c , the clauses $(\bar{a} \vee \bar{b} \vee c) \wedge (a \vee \bar{c}) \wedge (b \vee \bar{c})$ enforce $c \leftrightarrow a \wedge b$). Conjoin all these per-gate clauses with a single unit clause forcing the output wire to 1. The resulting CNF has clauses of size ≤ 3 ; pad shorter clauses with auxiliary variables to obtain exactly 3 literals per clause. The construction has size polynomial in the circuit, and the formula is satisfiable iff the circuit is.

Composing the two steps gives $L \leq_p \text{3-SAT}$ for every $L \in \text{NP}$, which by Definition 11.14 is the definition of 3-SAT being NP-hard.

Remark 12.13 (What Cook–Levin buys). After Cook–Levin, every subsequent NP-hardness proof follows the pattern: pick a problem A already known NP-hard (typically 3-SAT, but later INDEPENDENT-SET, VERTEX-COVER, SUBSET-SUM, PARTITION, PARTITION-SPECIAL, etc. once they have been proven NP-hard), and exhibit $A \leq_p B$. By transitivity of $\leq_p$ (Theorem 11.16), every problem in NP then reduces to B .

A complete NP-completeness proof has two halves: exhibit a polynomial-size, polynomial-time-verifiable certificate (membership in NP), *and* reduce a known NP-hard problem to the target (hardness). Skipping the membership step shows only NP-hardness, not NP-completeness. The reduction direction is also a perennial trap: to prove B is NP-hard one reduces $A \leq_p B$ for A *already* known hard; reducing $B \leq_p \text{3-SAT}$ instead would only show $B \in \text{NP}$. Mnemonic: the previously-known-hard problem reduces *to* the new problem, so that solving the new one also solves the old.

12.3 Canonical reductions

Theorem 12.14 (Independent-Set $\leq_p$ Clique). CLIQUE is NP-complete. The hardness is established by the reduction: given an instance (G, k) of INDEPENDENT-SET (where $G = (V, E)$), output the instance $(\bar{G}, k)$ of CLIQUE, where $\bar{G} = (V, \bar{E})$ is the complement graph: $(u, v) \in \bar{E}$ iff $u \neq v$ and $(u, v) \notin E$.

Proof. *Membership in NP.* A clique $C \subseteq V$ of size $\geq k$ is itself a polynomial-size certificate; the verifier checks $|C| \geq k$ and $(u, v) \in E$ for every distinct pair $u, v \in C$ in $O(k^2)$ time.

Reduction is polynomial. Constructing $\bar{G}$ enumerates all $\binom{|V|}{2}$ pairs and includes them in $\bar{E}$ iff they are not in E , taking $O(|V|^2)$ time.

$(\Rightarrow)$. Suppose G has an independent set $X \subseteq V$ of size k . By definition, no two vertices in X are adjacent in G , so for every distinct $u, v \in X$ we have $(u, v) \notin E$, hence $(u, v) \in \bar{E}$. Thus X is a clique of size k in $\bar{G}$.

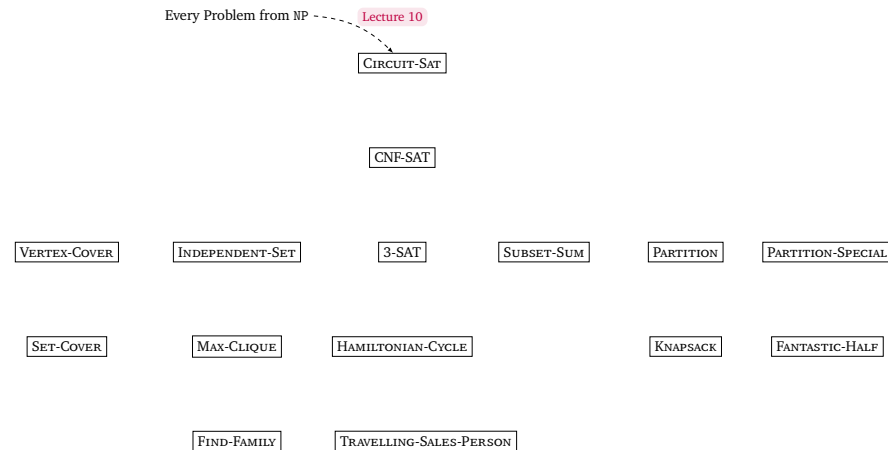
$(\Leftarrow)$. Suppose $\bar{G}$ has a clique X of size k . Then for every distinct $u, v \in X$, $(u, v) \in \bar{E}$, so $(u, v) \notin E$, so X is an independent set of size k in G .

Since INDEPENDENT-SET is NP-hard (by reducing 3-SAT to INDEPENDENT-SET; see section 11), so is CLIQUE.

Reductions

Last Update: – (Initial Version)

We will be seeing many (polynomial-time) reductions throughout the course. An arrow $A \rightarrow B$ indicates that we have proved $A \leq_p B$.
Exercise: Update this diagram as the semester progresses!



5

Figure 6: Canonical NP-complete reductions covered in this chapter. An arrow $A \rightarrow B$ means $A \leq_p B$; the dashed arrow from “Every Problem from NP” to CIRCUIT-SAT is the Cook–Levin theorem.

Remark 12.15 (Certificates have to translate). The reduction is correct precisely because the same vertex subset witnesses both sides: a YES-witness C for CLIQUE on $\overline{G}$ is also a YES-witness for INDEPENDENT-SET on G . Arguing only that “the optimal value of A on x equals the optimal value of B on $f(x)$ ” is necessary but not sufficient for $\leq_p$ -correctness; one must additionally exhibit a polynomial-time map taking a YES-witness for $f(x)$ back to a YES-witness for x .

Theorem 12.16 (Independent-Set $\leq_p$ Vertex-Cover). VERTEX-COVER is NP-complete. The hardness uses the duality: $X \subseteq V$ is a vertex cover of $G = (V, E)$ iff $V \setminus X$ is an independent set of G . Reduction: given an instance (G, k) of INDEPENDENT-SET, output $(G, |V| - k)$ as an instance of VERTEX-COVER.

Proof. NP membership. A subset $X \subseteq V$ with $|X| \leq k'$ is a certificate; the verifier checks for every edge $(u, v) \in E$ whether $u \in X$ or $v \in X$, in $O(|E|)$ time.

Reduction is polynomial. The output is just $(G, |V| - k)$, computed in $O(|V|)$ time.

Correctness via the duality. For any $X \subseteq V$, the following are equivalent:

- (i) X is a vertex cover, i.e. for every edge $(u, v) \in E$, $u \in X$ or $v \in X$;
- (ii) no edge has both endpoints in $V \setminus X$;
- (iii) $V \setminus X$ is an independent set.

The first $\iff$ second is contraposition; the second $\iff$ third is the definition of an independent set.

($\Rightarrow$). If G has an independent set Y of size $\geq k$, take $X = V \setminus Y$. By the duality, X is a vertex cover; and $|X| = |V| - |Y| \leq |V| - k$.

($\Leftarrow$). If G has a vertex cover X of size $\leq |V| - k$, take $Y = V \setminus X$. By the duality, Y is an independent set; and $|Y| = |V| - |X| \geq k$.

Corollary 12.17 (Three-way equivalence). INDEPENDENT-SET, CLIQUE, and VERTEX-COVER are pairwise polynomial-time equivalent: each reduces to each in polynomial time, via complementation of the graph (Theorem 12.14) and complementation of the vertex set (Theorem 12.16).

Theorem 12.18 (3-Sat $\leq_p$ Directed-Ham-Cycle). DIRECTED-HAM-CYCLE (and hence HAMILTONIAN-CYCLE on undirected graphs, by a further reduction) is NP-complete. Given a 3-SAT instance Φ over variables $x_1, \dots, x_n$ with clauses $C_1, \dots, C_m$, construct a directed graph G as follows.

- For each variable x_i , build a horizontal path gadget P_i : a chain of $3m + 3$ vertices with antiparallel edges between consecutive vertices (so the path can be traversed left-to-right or right-to-left). Left-to-right traversal of P_i encodes $x_i = \text{True}$; right-to-left encodes $x_i = \text{False}$.
- Add two special vertices s and t . Connect s to the left and right ends of P_1 , and connect both ends of P_n to t . Each P_i 's endpoints are connected to both endpoints of P_{i+1} , so any Hamiltonian cycle traverses the path gadgets one by one, choosing a direction independently for each.
- For each clause $C_j = \ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3}$, add a clause vertex c_j . For each literal $\ell_{j,k}$, place edges between c_j and a designated pair $(u_{i,j}, v_{i,j})$ of consecutive vertices in P_i (where i is the variable underlying $\ell_{j,k}$): specifically, if $\ell_{j,k} = x_i$, add edge $u_{i,j} \rightarrow c_j$ and $c_j \rightarrow v_{i,j}$ (so a left-to-right traversal of P_i can “detour” through c_j); if $\ell_{j,k} = \overline{x_i}$, reverse the orientation. Different clauses use disjoint pairs of consecutive vertices in each P_i , which is why each P_i needs $\Theta(m)$ vertices.

The output G has $\Theta(nm)$ vertices and edges, constructed in polynomial time.

Proof. (sketch.) NP membership. A Hamiltonian cycle is itself a polynomial-size certificate; the verifier checks that it visits each vertex exactly once and uses only edges of G , in $O(|V|)$ time.

Reduction is polynomial. The graph has $\Theta(nm)$ vertices and edges, all constructible in polynomial time.

($\Rightarrow$). Suppose Φ is satisfiable, witnessed by an assignment α . Build a Hamiltonian cycle: start at s ; for each $i = 1, \dots, n$, traverse P_i left-to-right if $\alpha(x_i) = \text{True}$ and right-to-left otherwise. Since α satisfies Φ , each clause C_j contains some literal $\ell_{j,k}$ that is true under α ; the orientation of P_i (where i is the variable of $\ell_{j,k}$) chosen above is precisely the one that lets the traversal detour through c_j via the dedicated $(u_{i,j}, v_{i,j})$ pair. Pick one such satisfying literal per clause and detour through the corresponding c_j ; finally connect to t and back to s . Each clause vertex is visited exactly once because we detour through exactly one literal per clause.

($\Leftarrow$). Suppose G has a Hamiltonian cycle H . The key claim (proved by case analysis of the local edges around each clause vertex's two attachment points u, v in P_i) is: *after H enters a clause vertex c_j from some $u_{i,j} \in P_i$, it must leave c_j back to the immediately following vertex $v_{i,j}$ in the same P_i – it cannot “escape” to a different $P_{i'}$.* The argument: if H entered c_j from u and exited to a vertex v' in some other path, then the predecessor of the next vertex v in the original P_i would have no remaining valid in-neighbour, leading H to get stuck. Granting the claim, contracting all detours through clause vertices yields a Hamiltonian cycle of the auxiliary graph consisting of

just the path gadgets and s, t . That cycle must traverse each P_i in a single direction; read off $\alpha(x_i) = \text{True}$ if left-to-right and False otherwise. Each clause vertex c_j is visited via some literal $\ell_{j,k}$ which (by construction of the detour edges) is true under α , so α satisfies every clause, hence Φ .

Theorem 12.19 (Partition-Special $\leq_p$ Fantastic-Half). FANTASTIC-HALF is NP-complete.

Recall: PARTITION-SPECIAL asks, given non-negative integers $x_1, \dots, x_n$, whether $\{1, \dots, n\}$ can be partitioned into $I_1 \sqcup I_2$ with $\sum_{i \in I_1} x_i = \sum_{i \in I_2} x_i$ and $|I_1| = |I_2|$; FANTASTIC-HALF asks, given non-negative integer pairs $(a_1, b_1), \dots, (a_n, b_n)$ with $A = \sum a_i$, $B = \sum b_i$, whether some $S \subseteq \{1, \dots, n\}$ with $|S| = n/2$ satisfies $\sum_{i \in S} a_i \geq A/2$ and $\sum_{i \in S} b_i \geq B/2$. *Reduction:* given $(x_1, \dots, x_n)$, output $a_i := x_i$ and $b_i := X - x_i$ for all i , where $X = \sum_i x_i$.

Proof. NP membership. The set S is itself a polynomial-size certificate; the verifier checks $|S| = n/2$, $\sum_{i \in S} a_i \geq A/2$, and $\sum_{i \in S} b_i \geq B/2$ in linear time.

Reduction validity and runtime. Each $b_i = X - x_i \geq 0$ since $x_i \leq X$, so the constructed instance is valid. The construction takes $O(n)$ time.

($\Rightarrow$). If $\{1, \dots, n\}$ partitions into I_1, I_2 of equal size and equal sum, take $S = I_1$. Then $|S| = n/2$, and $\sum_{i \in S} a_i = \sum_{i \in S} x_i = X/2 = A/2$. For the b -sums, $\sum_{i \in S} b_i = \sum_{i \in S} (X - x_i) = (n/2)X - X/2$, while $B = \sum_i (X - x_i) = nX - X$, so $B/2 = (nX - X)/2 = (n/2)X - X/2$, giving $\sum_{i \in S} b_i = B/2$.

($\Leftarrow$). Suppose S satisfies $|S| = n/2$, $\sum_{i \in S} a_i \geq A/2$, and $\sum_{i \in S} b_i \geq B/2$. The first inequality reads $\sum_{i \in S} x_i \geq \sum_{i \notin S} x_i$. The second reads $\sum_{i \in S} (X - x_i) \geq \sum_{i \notin S} (X - x_i)$, i.e. $(n/2)X - \sum_{i \in S} x_i \geq (n/2)X - \sum_{i \notin S} x_i$, hence $\sum_{i \in S} x_i \leq \sum_{i \notin S} x_i$. The two inequalities together force equality: $\sum_{i \in S} x_i = \sum_{i \notin S} x_i$. So $(I_1, I_2) := (S, \{1, \dots, n\} \setminus S)$ is a valid PARTITION-SPECIAL witness.

Remark 12.20 (Why the obvious reductions $a_i = b_i = x_i$, $b_i = x_{n+1-i}$, and $b_i = -x_i$ all fail). Three natural attempts fail before the correct reduction $b_i = X - x_i$. The “copy” attempt $a_i = b_i = x_i$ breaks ($\Leftarrow$): the inequality $\sum_{i \in S} a_i \geq A/2$ alone is not enough to conclude equality with $\sum_{i \notin S} a_i$ – the counterexample $(1, 2, 3, 5)$ is a NO-instance of PARTITION-SPECIAL but YES under this map (take $S = \{3, 4\}$ with a -sum $8 \geq 11/2$ and b -sum $8 \geq 11/2$). The “reverse” attempt $b_i = x_{n+1-i}$ permutes but does not negate; the same counterexample still works. The “negate” attempt $b_i = -x_i$ would force equality (the two reverse inequalities cancel) but violates the non-negativity requirement of FANTASTIC-HALF, producing an invalid instance. Replacing $-x_i$ by $X - x_i$ preserves the cancellation property up to the additive constant X while keeping values non-negative; this is the trick.

The first two failures illustrate that *both directions* of correctness are mandatory: $x \in A \Rightarrow f(x) \in B$ is easy, but the reverse can fail on a small counterexample. The third illustrates that the reduction’s output must be a syntactically valid instance of the target – always check that $f(x)$ satisfies the input constraints of B . A separate concern is the runtime bound: the construction must be polynomial in $|x|$, not in the value encoded by x . For SUBSET-SUM-flavour reductions, “pseudo-polynomial” time in the magnitude of weights is *not* polynomial time.

12.4 Worked example

Example 12.21 (Concrete reduction Independent-Set $\leq_p$ Clique). Take the small graph G with vertex set $V = \{a, b, x, y, z, t\}$ and edge set

$$E = \{(a, b), (b, x), (x, y), (y, t), (y, z)\}.$$

We ask: does G have an independent set of size ≥ 3 ? We solve it via the `CLIQUE` oracle.

Step 1 – apply the reduction. Build $\overline{G}$ on the same vertex set; an edge (u, v) is in $\overline{G}$ iff $u \neq v$ and $(u, v) \notin E$. The total number of unordered pairs is $\binom{6}{2} = 15$. Subtracting the 5 edges of E , the complement has 10 edges:

$$\overline{E} = \{(a, x), (a, y), (a, z), (a, t), (b, y), (b, z), (b, t), (x, z), (x, t), (z, t)\}.$$

Step 2 – ask the target problem. We ask whether $\overline{G}$ has a *clique* of size ≥ 3 .

Step 3 – inspect. The triple $\{a, x, t\}$ has all three pairs $(a, x), (a, t), (x, t) \in \overline{E}$, so it is a clique of size 3 in $\overline{G}$. By Theorem 12.14 (the $\Leftarrow$ direction), this same triple is an independent set of size 3 in G : indeed, none of $(a, x), (a, t), (x, t)$ is in E .

Step 4 – certificate translation. Critically, the reduction’s correctness is not just that “YES corresponds to YES”; it is that the *same set of vertices* is the witness on both sides. A black-box `CLIQUE` solver returning the clique $\{a, x, t\}$ of $\overline{G}$ *directly* hands back the desired independent set of G . This is the typical structure: the certificate transforms via the same map (or a simple inverse) as the instance itself.

Polynomial time. Building $\overline{G}$ took $O(n^2)$ time. So a hypothetical polynomial-time `CLIQUE` solver would yield a polynomial-time `INDEPENDENT-SET` solver: one $\overline{G}$ -construction followed by one `CLIQUE` call, then trivially read off the witness.

A Selected Problems

How to use this appendix

Each entry: terse problem statement, the key insight (the “trick”), and the standard pattern it instantiates. When a problem of the listed *shape* arises, the listed *trick* applies.

Asymptotic analysis

A.1 $\log f \in o(\log g)$ does not follow from $f \in o(g)$

Problem. If $f \in o(g)$ with both increasing, must $\lceil \log f(n) \rceil \in o(\lceil \log g(n) \rceil)$?

Trick. Take $f(n) = 2^n$, $g(n) = 4^n$: $f \in o(g)$ but $\log f = n = \frac{1}{2} \log g$. Logs compress exponential gaps to constant factors, breaking little- o .

Pattern. Big- O/Θ are preserved by logarithms (up to constants), but o and ω are not. When asked whether an asymptotic relation survives a transformation, check whether the relation tolerates constant multiplicative slack. See section 2.

Recurrences

A.2 Recurrence with two unequal recursive calls

Problem. Solve $T(n) = T(n/2) + T(\sqrt{n}) + 2n$ tightly.

Trick. Master theorem does not apply (two recursive calls with different shrink rates). Naive bound $T(n) \leq 2T(n/2) + 2n$ gives $O(n \log n)$, which is loose. Guess $T(n) \leq cn$ and verify by substitution: the $\sqrt{n}$ branch contributes only $c\sqrt{n} \leq (c-4)n/2$ for $n \geq 10$ with $c \geq 12$. So $T(n) \in \Theta(n)$.

Pattern. When master theorem fails, try the substitution method with a linear (or otherwise sharp) guess; the $\sqrt{n}$ subproblem is asymptotically negligible compared to $T(n/2) + n$. See section 3.

Divide and conquer**A.3 H-index by unbounded (galloping) binary search**

Problem. Given a non-increasing array $A[1..n]$, find the largest i with $A[i] \geq i$ (the H-index).

Trick. The plain binary search costs $O(\log n)$ regardless of where the answer lies. To get $O(\log H)$ when the H-index H is small, first *gallop*: probe $i = 1, 2, 4, \dots, 2^k$ until $A[2^k] < 2^k$, then binary-search inside $[2^{k-1}, 2^k]$. Total: $O(\log H)$.

Pattern. “Output-sensitive search”: when the answer might be much smaller than n , exponentially expand the search range before binary-searching. Same idea as searching in an unbounded sorted stream. See section 5.

A.4 Two-scanner search with a moving target

Problem. A fighter hides in one of n bunkers; two scanners may be queried once per day at 6 AM (each tests an interval); after each scan, the fighter may move to an adjacent bunker. Minimise the number of days.

Trick. With two simultaneous interval scanners, you can quarter the uncertainty per day (four outcome patterns), giving $\log_4 n$ days statically. The mobility adds at most $+2$ to the interval each night, so the recurrence becomes $u_{d+1} \leq u_d/4 + 2$, whose fixed-point is constant; $\lceil \log_4 n \rceil + O(1)$ days still suffice.

Pattern. k parallel binary tests divide the search space by $k + 1$, not by 2. With perturbations between rounds, set up a recurrence whose dominant term is the $1/(k + 1)$ shrink and absorb the additive perturbation as a geometric tail. See section 5.

Sorting and lower bounds**A.5 Sorting pairs (x, y) by $x + n^y$ in $O(n)$**

Problem. Sort n pairs (x_i, y_i) with $1 \leq x_i, y_i < n^3$ by $\text{val}(x, y) = x + n^y$ in $O(n)$ time.

Trick. The $\Omega(n \log n)$ comparison lower bound does not apply because we can read digits, so radix sort is fair game. *Splitting on y is essential*: when $y_j > 3$, n^{y_j} dominates x_j ; for $y \leq 3$, val fits in $2n^3$ and a single radix sort works. Otherwise, two passes (radix-sort by x , then stable radix-sort by y) finish the rest. Each pass is $O(n)$ since values are $\leq n^3$ (constant number of digits in base n).

Pattern. When values are polynomial in n , view them as $O(1)$ -digit base- n numbers and apply radix sort for $O(n)$ sorting. Beware mixing scales: split the input into “small” and “large” regimes when one component dominates the other. See section 6.

Randomised algorithms**A.6 Existence proofs via the probabilistic method**

Problem. Show every graph has an independent set of size at least $\sum_v \frac{1}{d(v)+1}$.

Trick. Pick a uniform random vertex order; let I contain each v all of whose neighbours come after it. I is independent by construction. By linearity of expectation, $\Pr[v \in I] = \frac{1}{d(v)+1}$, so $\mathbb{E}[|I|] = \sum_v \frac{1}{d(v)+1}$. Some realisation must achieve at least the expectation.

Pattern. *Probabilistic method*: to prove existence of a structure of size $\geq L$, design a random construction whose expectation is L . Indicator + linearity of expectation does the heavy lifting; conditional independence is not needed. See section 7.

Dynamic programming

A.7 Minimise $|S_X - 2S_Y|$ via subset-sum DP

Problem. Partition indices into X, Y minimising $|\sum_X A[i] - 2\sum_Y A[i]|$, where $0 \leq A[i] \leq M$.

Trick. Let $S = \sum A[i]$ and $t = \sum_Y A[i]$. Then the objective rewrites as $|S - 3t|$. So we just need a subset-sum closest to $S/3$: a standard pseudo-polynomial $O(nS) = O(n^2M)$ DP table, then scan for the best feasible t .

Pattern. Algebraically reduce a multi-set objective to a single-variable one (here, the target sum t); then the problem becomes a known DP. The space of achievable subset sums is the canonical reduction target. See section 8.

Greedy algorithms

A.8 Interval cover: pick the latest finish among feasible starts

Problem. Cover days $1..M$ with the fewest guard intervals $[a_i, b_i]$.

Trick. Sweep: maintain frontier m (next uncovered day, initially 1). Among guards with $a_i \leq m$, pick the one with the largest b_i , then advance $m \leftarrow b_i + 1$. Exchange argument: any optimal solution must include some guard active on day m ; swap that guard for the latest-finishing such guard without losing feasibility.

Pattern. *Greedy choice with exchange argument*: identify a structural “pinch point” (here, day m must be covered *somehow*), then show the most aggressive feasible choice at that point is safe. Compare to activity selection. Note: this stops being greedy once weights enter (a weighted variant requires DP). See section 9.

Amortised analysis

A.9 Repeated set unions cost $O(n \log n)$

Problem. Start with n singletons. Repeatedly union two sets at cost $\min(|A|, |B|)$ until one set remains. Show total cost $\leq \frac{n}{2} \log n$ regardless of strategy.

Trick. Charge to elements, not operations. Define $\phi_i = \log |S(i)|$ where $S(i)$ is element i 's current set. Merging sets of sizes $a \geq b$: each element on the small side gains $\log \frac{a+b}{b} \geq \log 2 = 1$, each on the large side gains $\log(1 + b/a) \geq b/a$. Summing: total potential rises by $\geq 2b$, twice the cost. Since $\phi_i \in [0, \log n]$, the total potential is at most $n \log n$, giving cost $\leq \frac{n}{2} \log n$.

Pattern. *Potential per element* (not per operation) is the right granularity for size-based merge costs. The estimate $\log(1 + x) \geq x$ for $x \in [0, 1]$ converts a logarithmic potential change into a linear lower bound, matching the merge cost. Same scheme as weighted-union-find analysis. See section 10.

Reductions and NP-completeness

A.10 Decision-to-search via self-reduction (Hamiltonian cycle, 3-SAT)

Problem. Given a polynomial-time *decider* for Hamiltonian cycle (resp. 3-SAT), find an actual Hamiltonian cycle (resp. a satisfying assignment) in polynomial time.

Trick. Iteratively commit and re-test. For HC: at each vertex, try keeping one outgoing edge and deleting the rest; if the decider still says YES, commit. For 3-SAT: set $x_1 = 1$, ask the decider; if YES, fix it, else set $x_1 = 0$; repeat for $x_2, \dots, x_n$. Each commit preserves the YES-instance invariant,

and only $\text{poly}(n)$ decider calls are made.

Pattern. *Self-reduction:* when the search problem and the decision problem share the structure “A solution exists in G iff a solution exists in G restricted by one bit”, reduce search to a polynomial sequence of decisions. This is why search and decision are polynomially equivalent for NP-complete problems. See section 11, section 12.

A.11 Reducing CLIQUE to Half Clique by padding

Problem. Show Half Clique (does G on $|V|$ vertices have a clique of size exactly $|V|/2$?) is NP-complete.

Trick. The size constraint is rigid. From $(\hat{G}, k)$ with $|V(\hat{G})| = n$, build G by adding $n - k$ new vertices forming a clique fully connected to $\hat{G}$, plus k isolated vertices. Total $2n$ vertices; a clique of size n exists iff $\hat{G}$ has a k -clique.

Pattern. *Padding reductions:* when the target problem fixes a size to half (or another constant fraction) of $|V|$, add cliques to “promote” a small clique and isolated vertices to control the vertex count. Same flavour as reducing one fixed-size variant to another. See section 12.

Index of Definitions

- 3-SAT, 59
- accounting method, 49
- aggregate method, 49
- amortised cost, 49
- approximation ratio, 47
- banker's method, 49
- Big-O notation, 9
- Big-Omega notation, 9
- Big-Theta notation, 9
- certificate, 55
- Circuit-SAT, 59
- class NP, 55
- class P, 55
- Clique, 59
- comparison-based sort, 26
- decision problem, 54
- decision tree, 27
- deterministic polynomial time, 54
- Directed-Hamiltonian-Cycle, 59
- divide and conquer, 22
- encoding, 54
- event, 6
- exchange argument, 42
- expectation, 6
- expected running time, 33
- Fantastic-Half, 59
- greedy algorithm, 42
- greedy-choice property, 42
- Hamiltonian-Cycle, 59
- in-place sort, 26
- Independent-Set, 59
- indicator variable, 6
- input size, 54
- language (decision problem), 54
- Las Vegas algorithm, 32
- little-o notation, 9
- little-omega notation, 10
- loop invariant, 17
- memoisation, 37
- Monte Carlo algorithm, 32
- NP-complete, 56
- NP-hard, 56
- one-sided error, 33
- optimal substructure, 37
- optimal substructure (greedy), 42
- overlapping subproblems, 37
- partial correctness, 17
- Partition, 59
- Partition-Special, 59
- polynomial-time reduction, 55
- potential function, 49
- potential method, 50
- principle of deferred decisions, 34
- probability space, 6
- random variable, 6
- randomised algorithm, 32
- recurrence relation, 13
- recursion-tree method, 13
- sample space, 6
- stable sort, 26
- standard deviation, 6
- state space, 37
- strong induction, 18
- structural induction, 18
- subproblem, 37
- Subset-Sum, 59
- substitution method, 13
- tabulation, 37
- termination measure, 17
- total correctness, 17
- transition (DP), 37
- two-sided error, 33
- variance, 6
- Vertex-Cover, 59

List of Theorems and Lemmas

1.1	Theorem (Linearity of expectation)	6
1.2	Theorem (Markov's inequality)	6
1.4	Theorem (Chebyshev's inequality)	6
1.6	Theorem (Union bound)	6
2.15	Theorem (Equivalence of definitional and limit forms)	10
2.17	Theorem (O – Ω duality)	10
2.18	Theorem (Transitivity)	11
2.19	Theorem (Sum and max rules)	11
2.20	Corollary (Reflexivity, symmetry of Θ)	11
3.6	Theorem (Master theorem)	15
4.8	Theorem (Loop-invariant theorem)	18
4.13	Theorem (Strong induction principle)	21
5.4	Theorem (Correctness template for D&C)	23
5.6	Proposition (Worst-case time of a D&C algorithm)	23
6.5	Theorem (Comparison-sorting lower bound)	27
6.6	Corollary (Sharp counting consequence)	28
6.8	Theorem (Average-case running time of quicksort)	28
6.12	Theorem (Counting sort)	30
6.15	Theorem (Radix sort)	31
6.17	Theorem (Median-of-medians selection)	31
7.8	Theorem (Correctness of Freivalds' algorithm)	34
7.11	Theorem (Error amplification)	35
7.14	Proposition (Randomised quicksort)	36
8.10	Theorem (Correctness template for DP)	38
8.11	Proposition (Time complexity of a DP algorithm)	39
9.6	Theorem (Correctness of earliest-finish-first scheduling)	43
9.9	Theorem (Correctness of Huffman coding)	44
9.12	Theorem (Cut property)	45
9.14	Theorem (Correctness of Prim)	46
9.15	Theorem (Correctness of Kruskal)	46
9.19	Theorem (Maximal matching gives a 2-approximation for Vertex Cover)	47
10.8	Theorem (Amortised cost bounds total actual cost)	50
10.11	Theorem (Binary counter has $O(1)$ amortised increment)	51
10.13	Theorem (Dynamic-array push has $O(1)$ amortised cost)	52
11.10	Proposition ($P \subseteq NP$)	55
11.16	Theorem (Transitivity of $\leq_p$)	56
11.19	Theorem (Reductions preserve P)	57
11.20	Theorem (Hardness propagates along reductions)	57
12.12	Theorem (Cook–Levin)	59
12.14	Theorem ($INDEPENDENT-SET \leq_p CLIQUE$)	60
12.16	Theorem ($INDEPENDENT-SET \leq_p VERTEX-COVER$)	61
12.17	Corollary (Three-way equivalence)	62
12.18	Theorem ($3-SAT \leq_p DIRECTED-HAM-CYCLE$)	62
12.19	Theorem ($PARTITION-SPECIAL \leq_p FANTASTIC-HALF$)	63